Rethinking Java Performance Analysis

Stephen M. Blackburn
steveblackburn@google.com
Google
Sydney, Australia
Australian National University
Canberra, Australia

Zixian Cai
zixian.cai@anu.edu.au
Australian National University
Canberra, Australia

Rui Chen
chenrui.crc@bytedance.com
Independent
Beijing, China

Xi Yang
xi@iop.systems
IOP Systems
Sydney, Australia

John Zhang
john.z@canva.com
Canva
Sydney, Australia

John Zigman
john.zigman@sydney.edu.au
The University of Sydney
Sydney, Australia

Abstract

Representative workloads and principled methodologies are the foundation of performance analysis, which in turn provides the empirical grounding for much of the innovation in systems research. However, benchmarks are hard to maintain, methodologies are hard to develop, and our field moves fast. The tension between our fast-moving fields and their need to maintain their methodological foundations is a serious challenge. This paper explores that challenge through the lens of Java performance analysis. Lessons we draw extend to other languages and other fields of computer science.

In this paper we: i) introduce a complete overhaul of the DaCapo benchmark suite [7], characterizing 22 new and/or refreshed workloads across 47 dimensions, using principal components analysis to demonstrate their diversity, ii) demonstrate new methodologies and how they are integrated into an easy to use framework, iii) use this framework to conduct an analysis of the state of the art in production Java performance, and iv) motivate the need to invest in renewed methodologies and workloads, using as an example a review of contemporary production garbage collector performance.

We highlight the danger of allowing methodologies to lag innovation and respond with a suite and new methodologies that nudge forward some of our field's methodological foundations. We offer guidance on maintaining the empirical rigor we need to encourage profitable research directions and quickly identify unprofitable ones.

CCS Concepts: • **Software and its engineering → Software performance**.

Keywords: Performance Analysis; Java; Garbage Collection

ASPLOS '25, Rotterdam, Netherlands

ACM ISBN 979-8-4007-0698-1/25/03
https://doi.org/10.1145/3669940.3707217

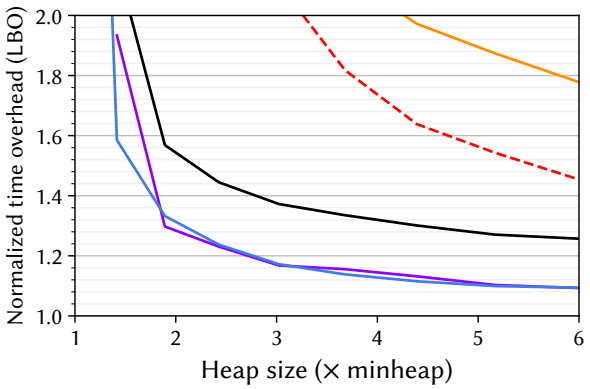
ACM Reference Format:
Stephen M. Blackburn, Zixian Cai, Rui Chen, Xi Yang, John Zhang, and John Zigman. 2025. Rethinking Java Performance Analysis. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25), March 30-April 3, 2025, Rotterdam, Netherlands.* ACM, New York, NY, USA, 72 pages. https://doi.org/10.1145/3669940.3707217

1 Introduction

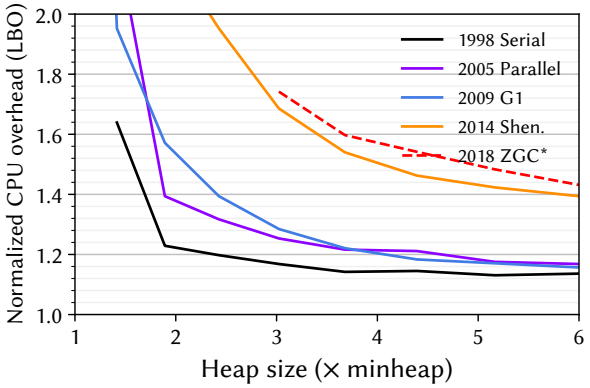
Building and maintaining benchmarks is a Sisyphean task, yet our field depends critically on them. Methodological innovation seems to be uncommon, yet our field is fast moving, so demands it. These two contradictions pose an enduring risk that we misdirect our field, abandoning promising ideas while pursuing ideas that we should have abandoned [6].

DaCapo is a broadly-focussed benchmark suite for Java heavily used in diverse domains within academia and industry [8, 20, 27, 45]. We motivate this work with a case study using DaCapo Chopin to explore overheads of contemporary production garbage collectors. We find that garbage collectors are consuming 15 % of CPU cycles even in the most favorable situations and that newer garbage collectors incur even higher overheads — as high as 17× in small heaps and 63 % in generous ones. What is interesting and relevant to our paper is not these overheads, but that they've largely gone unnoticed. Given the scale at which Java is deployed, the likely impact is substantial. We suggest that this lack of awareness is an example of collective methodological inattention.

One contribution of this paper is that we highlight the importance of the systems community continuously improving and evolving our methodologies. The heart of contribution, though, is DaCapo Chopin, a major release of the DaCapo benchmark suite for Java [7] that took fourteen years to develop, with eight entirely new workloads, all other workloads fully refreshed, a new methodology for measuring and reporting latency, nine latency-sensitive workloads, novel integrated workload characterization, and applications targeting the phone and the server, with minimum heap sizes from



(a) Lower bound wall clock time overheads.



(b) Lower bound total CPU overheads (Linux TASK_CLOCK).

Figure 1. Lower bounds on the overheads of five OpenJDK 21 production garbage collectors with their default settings, as a function of heap size, showing the geometric mean of overhead over all 22 DaCapo Chopin benchmarks. We only plot data points where the respective collector can run all 22 benchmarks to completion. In the best case, wall clock overheads are 9 % (G1 and Parallel) and total CPU overheads are 15 % (Serial). At smaller heaps, overheads exceed 2×.

5 MB to 20 GB. We offer methodological guidance, including a description of DaCapo Chopin's new latency metrics, and offer insights that we glean from evaluating DaCapo Chopin using OpenJDK 21.

We hope that the methodological guidance we offer will be used, that it might fuel a spirit of methodological critique and development, and that it might also inspire others to develop diverse, open, standardized workloads and methodologies for Java, for other languages, and in other areas.

2 Motivation

Figure 1 shows the overhead of five OpenJDK 21 garbage collectors as a function of heap size, taking the geometric mean over all 22 DaCapo Chopin benchmarks, using the lower bound overhead (LBO)[1] methodology [11].[2] The graphs show the time-space tradeoff inherent to garbage collection: CPU resources consumed by garbage collection rise as the available memory shrinks. Figure 1(a) shows the wall clock time overhead while Figure 1(b) uses Linux perf TASK_CLOCK, which sums the running time of all threads in the process, indicating the total computational overhead. All of the collectors except ZGC use compressed pointers by default. Because ZGC does not support compressed pointers, care should be taken when comparing it with the other collectors.

First, note that the CPU overhead of garbage collection is 15 % *in the best case*. Second, note that there is a regression when we consider collector designs in terms of when they were introduced into the JVM: Serial (1998), Parallel (2005), G1 (2009) [15], Shenandoah (2014) [17, 18, 43], and ZGC (2018) [29, 30, 51]. Comparison with the previous analysis by Cai et al. [11] indicates that these results are robust across JVM versions and benchmark versions. If we accept for a moment that our analysis is sound and that these figures accurately reflect the state of the art, this should give researchers pause.

What's going on? The relevant context is the rise of parallelism and latency as foremost concerns in application domains from mobile to the data center; parallelism because of the ubiquity of parallel hardware, latency because of the increasing use of garbage collection in latency-sensitive settings. The evolution in collector designs reflects this. Serial uses a single collection thread, while Parallel uses all of the available hardware parallelism, so runs faster than Serial. However, parallelism is never perfectly efficient, so Parallel tends to have larger total overhead when considering the task clock (Figure 1(b)). G1 performs work concurrently with the application and works in smaller regions at a time, so offers better latency than Parallel, and sometimes incurs a performance overhead in doing so. Shenandoah and ZGC go a step further and perform almost all collector work concurrently with the application, promising to offer better latency still, but incurring additional overhead in doing so. The problem depicted in Figure 1 was raised in 2022 by Cai et al. [11].

How did this happen? We don't improve what we don't measure. Specifically: i) **Failure to expose garbage collection's time-space tradeoff.** Despite the importance of systematically exploring this most basic tradeoff having been laid down more than twenty years ago [9, 10, 25], work routinely ignores this advice, neither systematically identifying minimum heap sizes [9], nor varying the available memory [10, 25] when evaluating collectors.[3] We respond to this in Section 4.2. ii) **Failure to appropriately evaluate latency.** Two decades ago, Cheng and Blelloch [13] clearly

[1] Pronounced *elbow*.
[2] Details of the methodology are in Section 6.1 and Section 6.2.
[3] We're pointing to the broader research community (the authors included) and its collective culture, not to individual researchers.

explained why garbage collection pause times should never be used as a proxy for user-experienced latency, but GC pauses continue to be (mis)used this way. We respond to this in Section 4.4. iii) **Failure to expose total computational overheads.** Despite the ubiquity of hardware parallelism and the importance of multi-tenanted platforms such as mobile devices, browsers, and data centers, evaluations rarely measure the total computational cost of systems, focusing instead on wall clock time. The situation is worse still with garbage collection, where costs can be hard to attribute [11]. We respond to this in Section 4.5. iv) **Failure to evaluate using diverse, appropriate workloads.** The cost of creating and maintaining benchmarks means that often there are not good, representative, workloads available. DaCapo Chopin is our response.

Objections to our motivating analysis might include:

Q: Shouldn't the latency advantages of the various collectors be presented here too?
A: We investigate latency in Section 4.4 and Section 6.3.
Q: Weren't some of these collectors designed (only) to be used with generous heaps?
A: Our analysis extends to 6× the minimum required memory. Given the cost of memory across mobile, desktop and data centers, a 6× memory overhead is generous. Consider Figure 6(d), showing h2 at 4 GB.
Q: What about an application domain that is not memory or compute-constrained?
A: In this situation, it may be best not to garbage collect at all [33, 49]. Otherwise, the time and space overheads introduced by a system remain important.

Although our motivating example is concretely based on garbage collection, it need not be. Garbage collection is an example of a methodologically challenging subject. Our point here is not to paint a negative picture of production garbage collectors, but to cast a light on overheads that have gone largely unnoticed yet affect production systems. We don't attribute this to the designers and engineers, but to our broader research community and our methodological inattention.

Our hope is that the new workloads, new methodologies, and new tooling we provide will nudge the field forward again, making it easier for researchers to better measure, analyze, and understand the likely impact of their work.

3 Background and Related Work

3.1 Modern Virtual Machines and OpenJDK 21

Modern language virtual machines are large and complex. According to estimates published by Synopsys Open Hub [46], the OpenJDK runtime includes 12 M lines of code, took 3193 person years to develop, and cost approximately $215 M to build. The complexity of these runtimes and their wide use compounds the methodological challenges of measuring them well while raising the stakes of not doing so.

We focus our attention on OpenJDK 21 and the compilers and collectors that ship with it. OpenJDK 21 was released in September 2023, a recent stable release of the most widely used runtime for Java [38], grounding our analysis in a state of the art production environment. It is not our goal to create benchmarks or methodologies for other languages, although some of our findings apply broadly to garbage-collected languages, and others to performance analysis more broadly. Nor is it our goal to evaluate other Java runtimes or garbage collectors, although the work we present should be directly applicable to them.

OpenJDK 21 was built over more than two decades, with a sophisticated multi-tier compiler [39] and a suite of highly-tuned production garbage collectors [15, 18, 24, 30]. The system is constantly under development, with contributions from the research community and industry. This analysis is not a commentary on developers of OpenJDK 21 but a critique of the broader community from which it emerged, and particularly our research community, which includes the authors.

3.2 Benchmarks and Benchmark Suites

The history of using benchmarks to evaluate systems performance dates at least to the early 1970's, when Curnow and Wichmann [14] discuss the use of a *"clearly defined task"* to compare the speed of various CPUs in their paper describing the Whetstone FORTRAN benchmark. They note that

> *unless such a program is carefully constructed it is unlikely to be typical of the many thousands of programs run at an installation.*

This observation cuts to heart of the problems outlined in Section 2. Ensuring benchmarks are representative makes them expensive to create and maintain, yet an absence of representative benchmarks is typically the justification researchers give for their use of ad hoc workloads and all the methodological problems that follow.

Benchmarks such as Whetstone and more recent examples such as gcbench [16], tests for Java Concurrency JSR166 [28], and the computer language benchmark game [3] are examples of micro benchmarks. These are easy to use, easy to measure, but far from realistic. They are nonetheless valuable tools. Simple, deterministic workloads can be particularly helpful in identifying and attributing specific performance regressions with high fidelity.

The DaCapo benchmark suite [7] was developed nineteen years ago to provide a realistic setting for JVM development and performance analysis. It was the result of a broad collaboration among industrial and academic researchers. The first-order design goals were diverse real-world applications, and ease of use. These led to the following criteria: i) open source workloads, ii) maximizing coverage of application

domains and behaviors, iii) easy to measure self-contained workloads, iv) exclusion of GUI workloads, and v) provision of a range of inputs. The authors also outlined a series of methodological recommendations, with a particular focus on JIT compilation and garbage collection, which differentiated Java performance analysis from that of C and FORTRAN which had dominated the decades prior to DaCapo's release [8].

The Renaissance suite [41] was developed to fill an absence of Java workloads that adequately exercised Java's newer parallel programming abstractions and concurrency primitives. The authors built a rich suite of programs following similar principles to DaCapo and used the suite to evaluate the Graal compiler [50] against the HotSpot C2 compiler [39], evaluating four new compiler optimizations and a number of other existing optimizations. In addition to Renaissance, there are other broadly-focussed suites developed with similar principles to DaCapo [42], but none target Java. There are many other suites with more narrow objectives, such as JaConTeBe which targets concurrency bugs [31].

SPECjvm and SPECjbb are produced by SPEC, a non-profit corporation guided by a desire to provide industry-standard benchmarks with which products can be fairly compared. SPECjvm was last released in 2008 and is not widely used by researchers. SPECjbb2015 [44] is a synthetic benchmark that executes requests over a simple model of a business, reporting both throughput and latency statistics. The business model can be scaled, generating large workloads. The benchmark measures *jOPS* (operations), and reports a *critical-jOPS* metric which is the geometric mean of the number of jOPS across different service-level agreements (SLAs) where the 99th percentile latency meets the respective SLA.

We present the new DaCapo Chopin benchmark suite. It differs from the prior work in significant ways: i) it is comprised entirely of real world workloads, ii) it includes workloads with application domains all the way from mobile to server, iii) it introduces a novel integrated latency measure and nine latency-sensitive workloads which report rich latency statistics, iv) its workloads have minimum heap sizes ranging from 5 MB to 20 GB, v) it includes 47 per-benchmark statistics characterizing and ranking the workloads, including nominal minimal heap sizes and various performance metrics, to help researchers understand workload behavior and to facilitate sound methodology, and vi) it introduces eight completely new workloads and updates all others. All benchmarks run on OpenJDK 21. We rely on community input to gain assurance of the representativeness of the suite. The composition of DaCapo Chopin was heavily guided by feedback from industry, with more than half of the workloads proposed by and/or co-developed with industrial users.

3.3 Empirical Evaluation

There is a large body of work on empirical evaluation [5, 6, 8, 21, 23, 48]. The SIGPLAN Empirical Evaluation Checklist [5] provides seven checklist items and 22 counter-examples designed to guide researchers to conduct sound evaluations. It presents both principles (seven checklist items) and concreteness (via counterexamples). The principles include that a paper's claims must be explicit and supported by their evaluation, and that there must be an appropriate and clear experimental design. A group of SIGPLAN researchers developed a pragmatic guide to assessing empirical evaluations, which includes extensive guidance and references [6]. Their opening sentence resonates with our goals:

> *An unsound claim can misdirect a field, encouraging the pursuit of unworthy ideas and the abandonment of promising ideas.*

The original DaCapo release came with methodological recommendations, including how to control for JIT compiler warmup and how to evaluate the time-space tradeoff of garbage collectors [7, 8]. We build on that work in Section 4.

There are numerous papers on how to improve fidelity in empirical evaluations of managed languages. Huang et al. [22] developed a methodology for *replaying* dynamic optimization plans to reduce measurement noise. Georges et al. [19] proposed refinements to this approach and made recommendations on how many measures should be taken in any given experiment. Cai et al. [11] describe the lower bound overhead (LBO) methodology evaluating garbage collector overheads. We use LBO throughout this paper and discuss it further in Section 4.5. Mytkowicz et al. [37] highlight alarming pitfalls for those conducting empirical evaluations. Papadakis et al. [40] conducted a broad study of the memory sensitivity of a range of Java benchmarks. We include some similar metrics here, but our goal is a broader characterization of workloads and we focus on the new OpenJDK 21 and DaCapo Chopin. Carpen-Amarie et al. [12] use cache coloring to artificially restrict the size of the last level cache in order to study the sensitivity of concurrent garbage collectors to last level cache size. We apply the same technique in our workload characterization (Section 5.1). Others such as Barrett et al. [4] have focussed on how to understand minute changes in performance, techniques which can be invaluable when attempting to identify and attribute small performance regressions. Our focus is in evaluating large production runtimes such as OpenJDK 21 in the face of large multi-threaded workloads with complex time-varying inputs that often exhibit complex behaviors.

4 Methodology

We made the case at the start of this paper that upholding sound methodological principles is important to the health of the field, and we summarized some of the wealth of resources available to researchers [4–6, 8, 11, 12, 19, 21, 23, 37, 40, 48].

Here we focus on: i) aspects of existing empirical evaluation advice that we think need renewed attention, and ii) new methodological features that DaCapo Chopin makes available. We believe that together, the following offers a strong response to each of the four major points of methodological failure that we outlined in Section 2.

4.1 Methodological Principles

While concrete recommendations are often most helpful, their concreteness means that their utility has a limited lifetime. We therefore start with some principles what provide a higher level framework from within which researchers can view methodology for empirical evaluation.

Researchers should **follow established methodology** right up to the very point where their evaluation moves beyond what the state of the art can support. Then they must **identify new methodologies** that correctly measure the subject of their evaluation. The SIGPLAN empirical evaluation checklist captures this by noting that the checklist *is meant to* support *informed judgement, not* supplant *it* [5]. Doing this well is hard. It requires the **integrity** to earnestly want to reveal the empirical truth, in the full knowledge that the results may not support an idea that has been years in development. It requires a high degree of **rigor**, because attention to detail is essential when evaluating complex systems in complex environments. Finally, it requires **discernment**, since one must be able to discern when existing methodologies are adequate (and should be rigorously adhered to), and when new methodologies must be developed.

4.2 The Time–Space Tradeoff

Figure 1 clearly illustrates the time–space tradeoff underpinning garbage collection, something all evaluations of garbage collected languages should either explore or control for. Although this has been widely understood for two decades or more, it remains commonplace for this aspect of managed language performance evaluation to be ignored.

Recommendation H1. Garbage collectors should be evaluated across a range of heap sizes to demonstrate the sensitivity of the collector to the time–space tradeoff [8].

We use a generous 6× upper bound in Figure 1 but a smaller upper bound might be more reasonable in many circumstances. Because the time-space tradeoff is not linear (see Figure 1), larger heap sizes yield smaller and smaller amounts of information. We therefore suggest selecting heap sizes in a distribution that gives more resolution to small heap sizes. Because the heap size requirements of different benchmarks vary greatly,[4] it is essential that the heap sizes used in an evaluation are chosen on a benchmark-by-benchmark basis, rather than applying the same heap sizes to all benchmarks.

Recommendation H2. Heap sizes should be expressed in terms of multiples of the minimum heap size in which a baseline collector can run that workload [8, 9].

To assist with this, DaCapo Chopin includes nominal statistics[5] which capture minimum heap sizes for its small, default, large and vlarge benchmark configurations with compressed pointers, as well as for the default configuration when compressed pointers are disabled. These all use the baseline OpenJDK 21 configuration we describe in Section 6.1.

Note that methodologies like this which control the memory available to the garbage collector (e.g. via `-Xmx`) *do not* necessarily provide a clear measure of how efficiently a collector reclaims space. This is because the minimum heap size in which a workload can run reflects the workload's *peak* memory usage, not its average usage. A metric which reflected the 'area under the memory use curve' might better reflect the net memory footprint of a workload.

4.3 Compilers, Warmup and Performance Analysis

Aside from the challenges that garbage collection brings to measuring a runtime, the just-in-time compiler and the broader experimental environment can create confounding effects. Mytkowicz et al. explain how small details such as the length of strings in environment variables can confound findings, advice researchers should heed [37].

While some researchers (implicitly) take the position that more iterations and more invocations are better, we make two contrary points: i) resources are finite and there is an opportunity cost associated with running more executions—it is quite possible that an experiment that tests more features is empirically stronger than one that yields very high precision on fewer dimensions, and ii) contrived experimental environments run a risk of irrelevance by compromising realism.

Recommendation P1. Researchers should be cautious of naïvely following methodological prescriptions. Instead they should be guided by: i) the coherence of their experimental design with respect to the claims they plan to make (which, for example, may determine whether to time the first iteration capturing JIT overheads, class loading, etc., or one that is well warmed up), and ii) the statistical significance of their findings (ensuring that there are sufficient data points such that a statistically sound conclusion can be drawn).

[4] For example, the minimum heap sizes range from 5 MB (avrora) to 681 MB (h2) in the default benchmark sizes settings, and up to 20 GB in the vlarge setting (h2).

[5] We use the term 'nominal' in the sense of *'being, or relating to a designated or theoretical size that may vary from the actual'* [32]. We use the word to emphasize that these measures are only intended to provide broad characterizations of the workloads in some default context and should not be viewed as a concrete, definitive measures defining the workload.

Figure 2. Cheng and Blelloch [13] used a figure like this to illustrate the problem with using GC pauses as a measure of responsiveness and proposed the minimum mutator utilization (MMU) metric as a response. GC pause time continues to be widely (mis)used as a proxy for responsiveness more than twenty years later.

The DaCapo Chopin suite comes with detailed measures of warmup time for each workload. In practice we found that the fifth iteration (-n 5) for default workload sizes and the first iteration for large and vlarge sizes exhibit well-warmed up behavior for our baseline configuration of OpenJDK 21.

4.4 User-Experienced Latency

Latency-sensitive applications are an increasingly important consideration for developers of managed languages, since garbage collectors and JIT compilers are capable of generating latency that is visible to application users. This applies to mobile devices, where refresh rates and smooth scrolling affect the user experience, to the desktop where browser responsiveness is important, and in request-based services that run on servers. DaCapo Chopin includes nine latency-sensitive workloads. These include jme, which is based on the jMonkey Engine, a popular video game engine, spring, which is a microservices workload built on the Spring web framework, and seven other request-based services.

A naïve approach to measuring latency is to simply measure the length of pauses created by the runtime that lock out the application (*stop the world* pauses). However, as Cheng and Blelloch [13] pointed out, this is a poor measure since several short pauses may have a similar or worse effect than a long pause (Figure 2). They proposed the notion of *minimum mutator utilization* (MMU) metric to reflect how much CPU was available to the mutator over a sliding window of time, for various window sizes. Despite this clear insight and guidance two decades ago, it remains common for GC pauses to be used as a proxy for user-experienced latency. Even so, MMU is not ideal since it is a single-threaded measure, cannot capture throughput reductions due to expensive barriers embedded within the mutator, and requires instrumenting the garbage collector. Zhao et al. [52] show that reliance on simple GC pause times has led to designs with surprisingly poor latency responsiveness even in modern collectors that specifically target latency. DaCapo Chopin addresses the problem by *directly reporting user-experienced latency*.

Simple Latency. DaCapo times every event: frame renders for jme and client requests for the other eight latency-sensitive workloads. As the workload progresses, DaCapo stores event start and end times in an array. Careful engineering ensures that the cost of recording these measurements is low. Once the workload completes, DaCapo determines the distribution of latencies, reporting the distribution in terms of percentiles, from median to 99.99, as well as optionally saving the complete data to file for offline analysis. We call this metric *Simple Latency*.

Request Queuing and Metered Latency. Most real-world request-based services implement a queuing system. Requests enter the queuing system at some externally determined rate, dictated by factors such as when customers make purchases or when people launch queries. These systems typically adjust server capacity as demand changes. One of the DaCapo design principles is that each benchmark will run in a single JVM on a single machine. Thus, attempting to implement a realistic distributed load balancing system was out of scope for DaCapo Chopin. Sacrificing some realism for determinism, the DaCapo request-based workloads are driven by a pre-determined set of requests, with each worker consuming consecutive requests until all have been completed. Within each thread, the start time of each request is thus dictated by the completion of the request before.

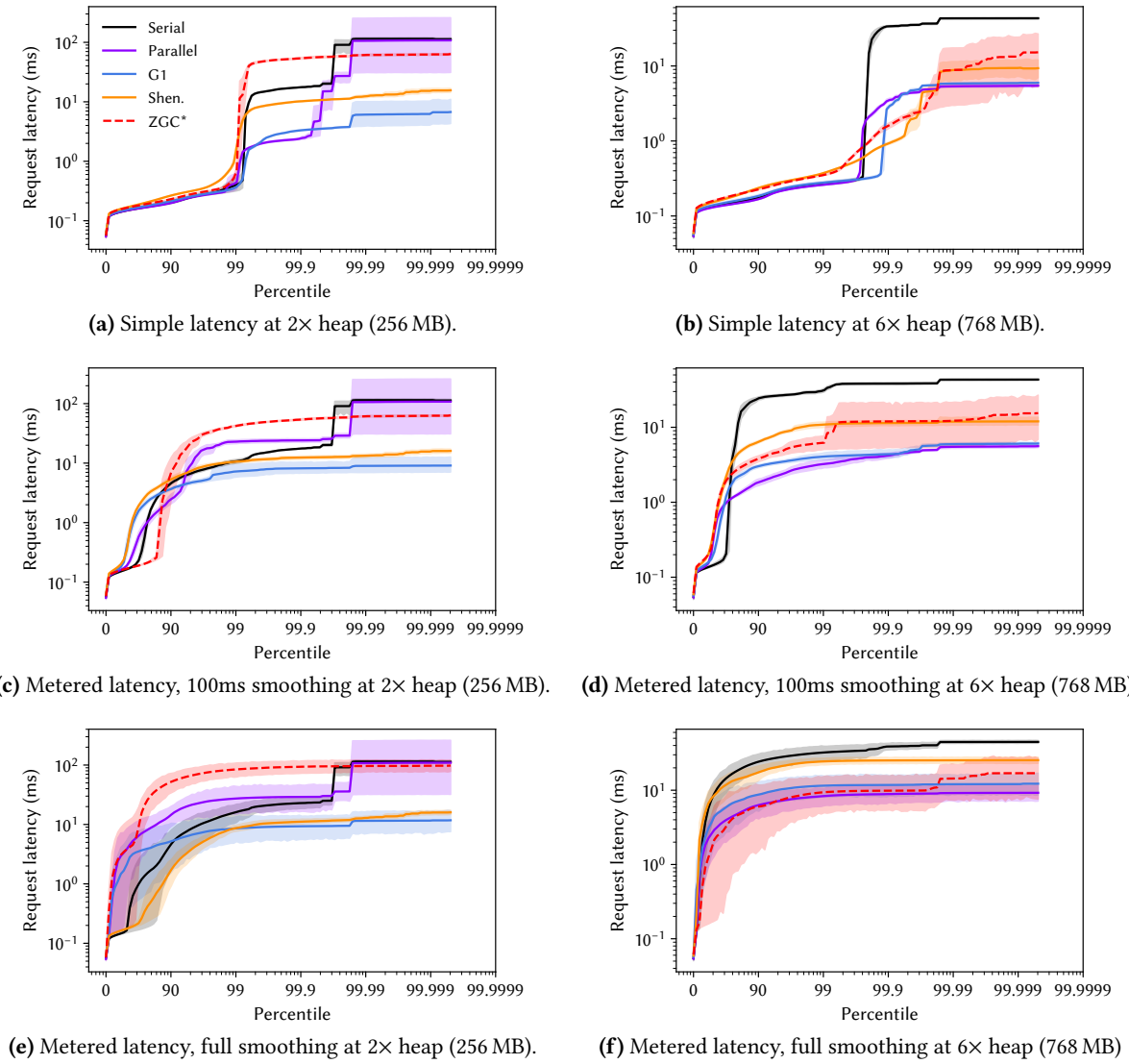
In a real system, request/event start times are *externally defined*, so a delay will affect not only all running events, but all subsequent events that are forced to wait in the queue due to the backlog of work. Without a queue, DaCapo's workloads cannot directly model the cascading effect of delays.

Instead, we model a similar effect with what we call *Metered Latency*. At the completion of the workload, we assign each event an assumed start time based on all events having been hypothetically received at *uniform* intervals throughout the execution of the benchmark. We then determine the metered latency for each event as the time between its end time and the earlier of its actual and assumed start times. Thus, when the application is paused, the effect of the pause is not felt just by those events that were running when the pause occurred, but also by those whose end time was delayed.

We implement the uniform synthetic start times by applying a smoothing function to the actual start times, using a sliding average. A window size of one affords no smoothing, so is identical to simple latency, reflecting no queueing effect. On the other hand, an arbitrarily large window gives all events uniformly distributed synthetic start times. DaCapo reports metered latency using window sizes from 1 ms up to the length of the benchmark execution, in powers of ten. We suggest that a smoothing window of 100 ms is a reasonable middle ground, allowing for variation in request completion rate over the whole execution (e.g. due to compiler or file cache warm up), while exposing effects of disruptions due garbage collection etc.

Recommendation L1. Researchers should report user-experienced latency, not weak proxies such as GC pauses.

Recommendation L2. Researchers should report distribution statistics and/or plot CDFs as illustrated in Figure 3,



(a) Simple latency at 2× heap (256 MB).

(b) Simple latency at 6× heap (768 MB).

(c) Metered latency, 100ms smoothing at 2× heap (256 MB).

(d) Metered latency, 100ms smoothing at 6× heap (768 MB).

(e) Metered latency, full smoothing at 2× heap (256 MB).

(f) Metered latency, full smoothing at 6× heap (768 MB)

Figure 3. DaCapo Chopin records the time for each event for its latency-sensitive workloads, avoiding the need for users to resort to using misleading proxies such as GC pause times. These figures plot the distribution of request latencies for cassandra for each of OpenJDK 21's five production collectors, with the 95th percentile indicated by the shaded area. Even at the generous 6.0× heap, the newer collectors do not deliver better latency than G1 on this workload.

rather than reporting singular latency metrics.

The inclusion of a realistic and diverse set of latency-sensitive workloads based on modern widely-used frameworks such as Spring, Cassandra, Kafka, Lucene, Tomcat, and Wildfly, and built-in latency metrics allow the community to easily and systematically measure user-experienced latency.

4.5 Lower Bound Garbage Collection Overheads

Understanding the real cost of garbage collection is a long-standing problem. The root of the problem is that garbage collection costs (and benefits) can be hard to measure. The difficulty of attribution is due to some costs being finely woven into the fabric of the application, such as the cost of the allocator or the cost of read and write barriers, while other costs are indirect, such as the locality effects of various allocation strategies or copying orders.

Cai et al. [11] exploit two simple observations to develop an easy-to-use and transparent measure of GC overheads: 1. If a perfect zero-cost GC existed, it could be used as the baseline with which to measure the overhead of concrete collectors. The overhead of a collector would simply be the difference between a benchmark run using that collector and the benchmark using the ideal collector. 2. Although the ideal

collector by definition does not exist, it can be *approximated* by running with a real garbage collector and subtracting costs easily attributable to the GC from the total costs.

The methodology thus requires taking measurements using multiple collectors, subtracting the costs attributable to the GC in each case. The system with the lowest net cost is the best approximation to ideal, and thus forms the baseline. The difference between the total cost for a concrete system and the baseline is thus an approximation to its overhead. Since the baseline is always an overestimate of the ideal, this overhead measure is always an *underestimate* of the overhead, and is thus a *lower bound on overhead*. This methodology is transparent and simple to implement, yielding a clear insight into the real overheads of garbage collection.

Recommendation O1. Researchers should report GC overheads when evaluating garbage collectors, using a methodology such as LBO [11], as illustrated in Figure 1.

Recommendation O2. Researchers should report both wall clock and total CPU overheads.[6]

5 DaCapo Chopin Benchmarks

The DaCapo Chopin suite replaces DaCapo Bach, adding eight new benchmarks and removing one. The composition of the suite, the retirement of old benchmarks and the addition of new ones is driven by community engagement, with the goal of keeping the suite relevant, diverse, and representative. Section 5.2 explains how we use principal components analysis (PCA) to quantify the diversity of the benchmarks we include.

5.1 Nominal Statistics

DaCapo Chopin comes with a large and diverse set of precomputed analyses and statistics, including bytecode execution and allocation size statistics as well as various performance metrics, such as ones measuring sensitivity to heap and cache size.[7] The statistics are included as part of the suite because they are methodologically and computationally non-trivial to calculate, yet provide considerable insight into *how* each of the benchmarks behave and *why* they behave that way.

DaCapo's *nominal statistics*[5] are derived from these precomputed metrics and are available, with brief descriptions, at the command line (-p). Their purpose is to provide benchmark users with a rich *qualitative* characterization of each workload with respect to a fixed hardware and software setting. Each benchmark is scored out of ten against each metric. The score is a simple linear mapping of the benchmark's rank among all benchmarks. 1 indicates the lowest ranked, while 10 indicates the highest ranked. We characterize each benchmark in the DaCapo Chopin suite across at least 35 dimensions[8].

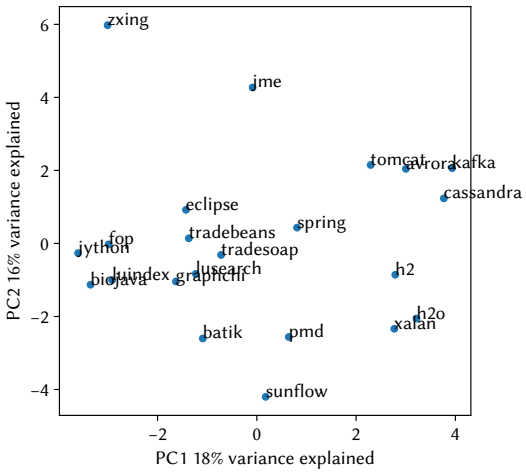
The inclusion of such metrics in a benchmark suite is novel as far as we know. We believe that they will help improve methodology (for example, nominal minimum heap sizes are among the statistics), and help researchers readily reason about behaviors they observe (such as why a compiler optimization appears to be less effective on some benchmarks).

The original DaCapo paper made the choice to only characterize statistics using JVM-neutral measures, such as total bytes allocated and total objects allocated [7]. We found that approach overly constraining as it precluded using metrics such as architectural sensitivity, which require measurements on a real JVM. Instead we chose to use OpenJDK 21 with its most basic configurations (such as the default G1 garbage collector), and describe the measures we made as *nominal*. This is to make clear that we were not attempting to *evaluate* the benchmarks or the JVM but to *characterize* the benchmarks within the suite in meaningful ways that would help users of the suite better understand the benchmarks and their sensitivity to various aspects of the execution environment. Our metrics include measures such as sensitivity to last level cache size, sensitivity to compiler configuration, sensitivity to heap size, etc. The focus of the nominal statistics is the *rank* among the benchmarks. We gather the statistics using a variety of techniques including time-consuming bytecode instrumentation and (separate) performance measurements. The bytecode instrumentation tools are included as part of the suite, allowing others to reproduce our measurements.

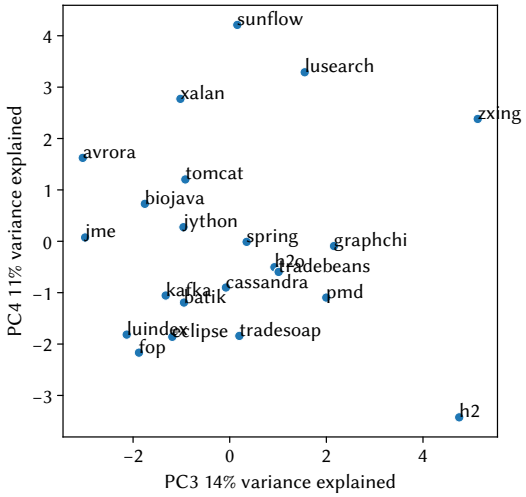
We give each nominal statistic a three-letter acronym and assign each benchmark a rank and a score from 1 to 10 on every metric, according to its sensitivity. For example, the lusearch workload has a nominal allocation rate (ARA) of 23556 MB/sec based on its total allocation and execution time. This places it first in the suite, yielding a score of 10. On the other hand, its sensitivity to aggressive (-comp) C2 compilation (PCC), is close to average, yielding a score of 4. These scores hold no meaning beyond allowing users to assess the *relative* sensitivities of the workloads.

We cluster the statistics into five groups, indicated by the first letter in the metric's acronym. The first four allocation metrics (AOA, AOL, AOM & AOS) are based on data gathered via bytecode-instrumented executions of the benchmarks, and the fifth (ARA) combines a measure of bytes allocated with a separately measured, uninstrumented time for benchmark execution. The seven bytecode metrics (BAL, BAS, BEF, BGF, BPF, BUB & BUF) are also gathered via bytecode instrumentation. Four of these (BAL, BAS, BGF & BPF) combine

[6] Note that naïvely counting cycles on a heterogenous platform unsound.
[7] From DaCapo Chopin MR1 onward, these statistics are available within the `stats` folder, e.g.: `dacapo-23.11-MR1-chopin/stats`.
[8] Most benchmarks have 47 dimensions, but not every dimension is available or relevant to each benchmark. tradebeans and tradesoap have the fewest, at 35, while h2 has the most at 47.



(a) PC1 and PC 2



(b) PC3 and PC4.

Figure 4. Principal components analysis of the 22 DaCapo workloads with respect to the 33 nominal statistics which had non-null results for all benchmarks.

bytecode counts with uninstrumented execution time to produce a rate. The twelve garbage collection metrics (GCA, GCC, GCM, GCP, GLK, GMD, GML, GMS, GMU, GMV, GSS, & GTO) use telemetry from the runtime's garbage collector. The eleven performance metrics (PCC, PCS, PET, PFS, PIN, PKP, PLS, PMS, PPE, PSD, & PWU) measure the execution time of the benchmark under various hardware and software configurations. Eleven of the microarchitectural metrics (UBM, UBR, UBS, UDC, UDT, UIP, ULL, USC, & USF) use hardware performance counters to measure performance characteristics, while the remaining two, UAI and UAA, measure the sensitivity of the benchmarks to running on entirely different processor designs (Intel and ARM).

5.2 Principal Component Analysis

We use the nominal statistics for each benchmark to conduct a principal component analysis of the workloads in the suite. In the analysis we use the 33 nominal metrics where all benchmarks have data points. We use raw values rather than scores, and apply standard scaling (linear scaling with 0 mean and unit variance). Figure 4 shows scatter plots of the twenty two workloads with respect to the top four principal components, with PC1 being the most determinative component (18 %) and PC4 being the least (11 %). Together, these four principal components account for over 50 % of the variance between benchmarks. Intuitively, the further apart the workloads are in the scatter graph, the greater the difference between them with respect to the nominal statistics. When designing a suite, diversity is important for coverage, and to avoid implicit duplication and thus over-emphasizing certain features. Figure 4 shows that the workloads that make up the DaCapo Chopin suite are well distributed, exhibiting substantial variation.

6 Analysis

We now use the new DaCapo Chopin and the methodologies we have discussed in the previous sections to conduct a detailed analysis of the workloads. The detailed results and statistics that underpin the following analysis are available within the benchmark suite[7] and as an appendix to this paper. In this section we will explain the methodology we've used throughout our analysis and highlight key results.

6.1 Methodology

The large number of dimensions available to our analysis prevents us from exploring the cross product of all variations across each dimension. Instead we identify a single baseline and explore variations with respect to it. We chose as our baseline the default configuration of the most recently shipping OpenJDK release, running on a recent x86 processor.

6.1.1 Benchmark Suite. We use version 23.11-chopin-MR2 of the DaCapo benchmark suite [2, 7], the most recent release of the suite at the time of writing.

6.1.2 JVM, Compilers and Garbage Collectors. We use the OpenJDK 21 runtime 2024-07-16 LTS shipped as the Temurin-21.0.4+7 distribution. Unless otherwise stated: for compatibility and consistency when evaluating across multiple OpenJDK versions, we used -server to select the runtime's compiler behavior; we ran 5 iterations of each benchmark, timing the last; and we used a 2× the benchmark's GMD nominal statistic, which is the minimum heap size in which that application will run 5 iterations with the default collector and the -server flag. When controlling for heap size, we used the -Xms and -Xmx flags. We run 10 invocations of each benchmark and show or plot the 95 % confidence intervals.

In practice, 10 invocations is sufficient to produce results with sufficiently tight confidence intervals.

6.1.3 Hardware and Operating System.

Our default hardware configuration is an AMD Ryzen 9 7950X Zen4 with 16 cores and 32 hardware threads, a 4.5 GHz base clock (with frequency scaling turned off), and 64 MB of last level cache. The system has 2 × 32 GB of DDR5-4800 with the standard JEDEC 40-39-39-77 timing profile.

We used the Ubuntu 22.04.4 distribution with the Linux 6.8.0-40 kernel, with the scaling governor set to performance. We turned off NMI watchdog to fully use all 6 performance monitor counters on the CPU. The system firmware is AMD AGESA 1.1.0.0.

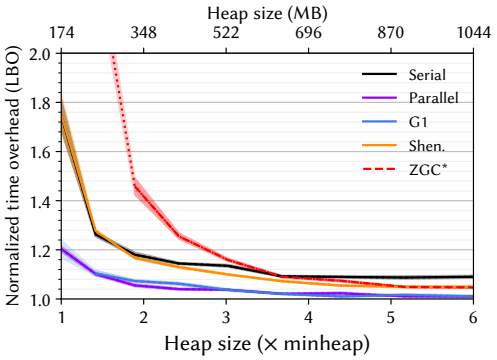
When testing benchmarks' sensitivity to memory speed, we configure the memory to the equivalent of DDR5-2000 32-32-32-64. When testing benchmarks' sensitivity to frequency scaling, we enable Core Performance Boost. When testing benchmarks' sensitivity to LLC size, we use AMD's PQOS L3 Cache Alloction Enforcement through Linux's resctrl interface.

6.2 Lower Bound Overheads

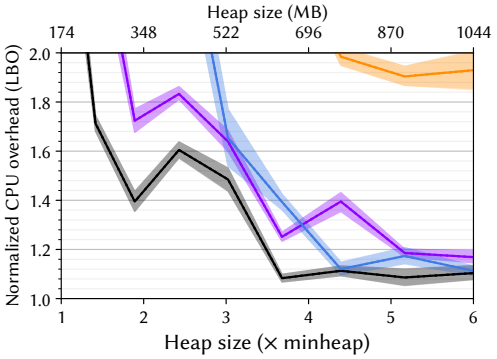
We now use the lower bound overhead (LBO) methodology introduced by Cai et al. [11]. The key to LBO is that it exposes the *total overheads* of a garbage collector relative to a conservative approximation to the ideal. LBO is methodologically straightforward, yet captures difficult-to-attribute overheads which had previously gone largely unmeasured. We discussed the geometric mean of LBO results in Section 2, and per-benchmark LBO results are included in the appendix. Here we analyze cassandra and lusearch as examples.

LBO Methodology. The key idea is to 'distill' a baseline that conservatively approximates the ideal GC. The distilled baseline is then used as the denominator in the LBO graphs, while the measured system forms the numerator [11]. We use Java's JVMTI interface to capture the easily-attributable stop-the-world periods of the collectors. The remainder is an approximation to the application costs. We then find the lowest approximated application cost from among all collectors and all heap sizes, and use that as the distilled cost, our denominator. Note that the simpler the collector, the more likely it is that the stop-the-world period captures most of the collector's cost. The simplest of the OpenJDK 21 collectors are Serial and Parallel, but even these have write barriers embedded within the application. So our distilled cost is clearly short of the ideal, and as a consequence the LBO overheads are *systematically conservative* estimates.

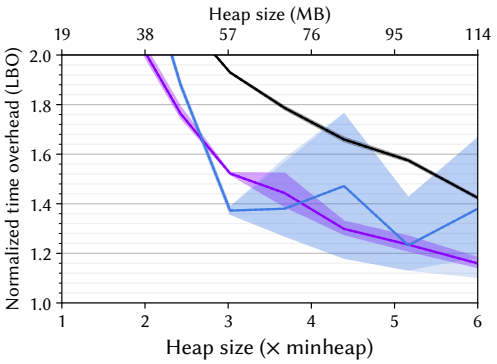
We use all of the garbage collectors that ship with OpenJDK 21 in our analysis. We evaluate them with respect to wall clock and task clock, at heap sizes from 1–6× the minimum heap size. The task clock captures total CPU use, across all threads. We plot each curve and indicate 95 % confidence intervals with shading.



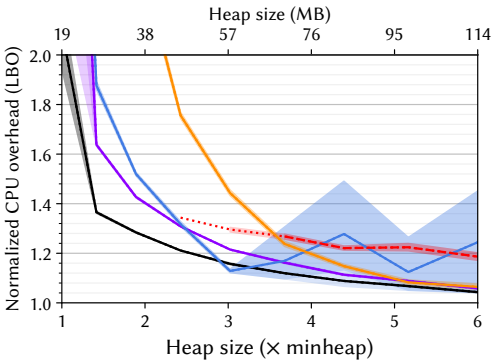
(a) Wall clock overheads for cassandra.



(b) Total CPU overheads (task clock) for cassandra.



(c) Wall clock overheads for lusearch.



(d) Total CPU overheads (task clock) for lusearch.

Figure 5. LBO overheads for cassandra and lusearch.

LBO Analysis. Figure 5(a) and Figure 5(b) show overheads for cassandra. The wall clock and task clock results are strikingly different. Above 4× the minimum heap size, all collectors have modest wall clock overheads, and right down to a modest 2× heap, Shenandoah's overhead remains below 20 %, while G1 and Parallel remain below about 10 %. However, the task clock tells a different story. G1 has a 60 % overhead even at a moderate 3× heap, while other collectors have overheads greater than a factor of two. This is most likely due to the collectors successfully making use of unused cores, since cassandra itself is not fully utilizing the available hardware. Although in the case of our experimental setup, the cores not being used by cassandra were free, in general there is an opportunity cost associated with using computational resources like this. This highlights the importance of taking both wall clock and task clock measures.

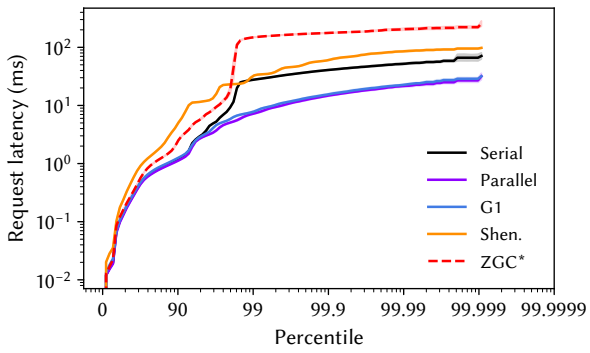
Figure 5(c) and Figure 5(d) show the overheads for lusearch. Wall clock overheads for Shenandoah are very high, greater than the 2.0× y-axis limit for all values of x. However, task clock overheads are significantly *lower*. This is counter intuitive since Shenandoah is a concurrent collector. However, note that lusearch has a very high allocation rate (ARA). Collectors like Shenandoah throttle the application in cases where the collector can't free memory fast enough to satisfy the application's demand. In this case, Shenandoah is throttling lusearch's 32 rapidly allocating client threads. This has the effect of much worse wall clock time (Figure 5(c)), and the side effect that the application can run efficiently due to less synchronization and contention (Figure 5(d)).

6.3 User-Experienced Latency

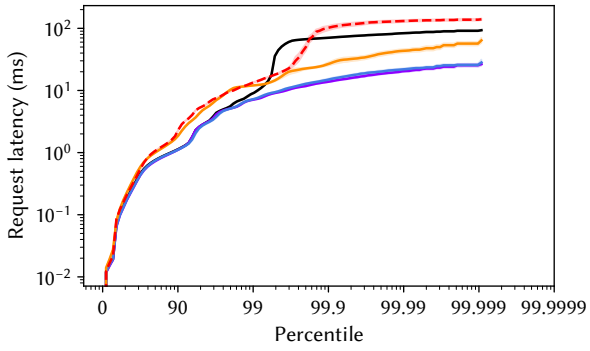
In Section 4.4 we described how DaCapo Chopin directly measures user-experienced latency, and discussed simple and metered latency for cassandra.

Figure 6 shows user-experienced latency for the h2 workload. The graphs are remarkably consistent. Above the 99.9th percentile the graphs are almost identical, with plateaus that reflect pauses in the range of 10–200 ms. The effect of the larger heap size is for most collectors to push the curves to the right. These graphs raise four questions: 1. Why are metered and simple latency almost identical at 2×? 2. Why do the latency-sensitive collectors (Shenandoah, ZGC and GenZGC) perform worse than Parallel and G1 in all cases, and worse than Serial in the 2× heap? 3. Why do all collectors have slightly *worse* tail latency when the heap is larger? 4. Why does Shenandoah's metered latency get *worse* at the larger heap size?

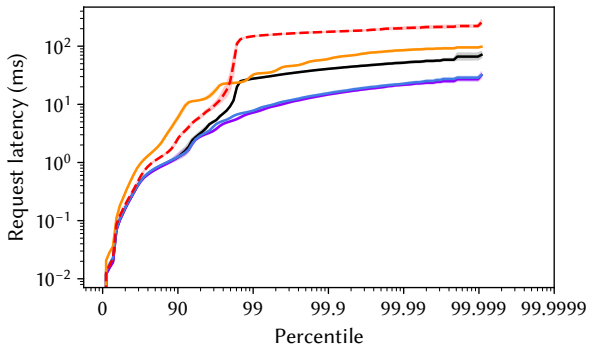
Answering these questions requires more understanding of the h2 workload. h2 is a database benchmark. It first creates a large in-memory database and then conducts queries over that database using the TPC-C workload [47]. It is the latency distributions of those queries that is depicted Figure 6. As a result of its design, h2 has a large heap size (GMD) but a low memory turnover (GTO), and each GC tends to



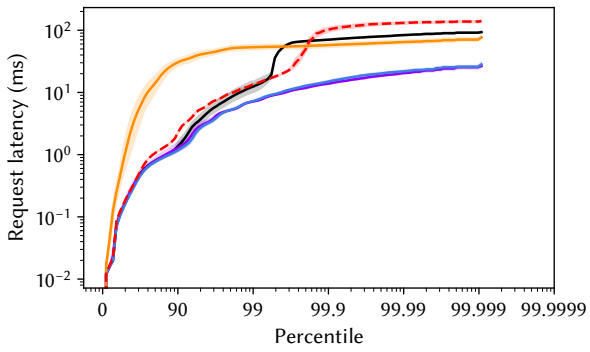
(a) Simple latency, 2× heap (1.36 GB).



(b) Simple latency, 6× heap (4 GB).



(c) Metered latency, full smoothing, 2× heap (1.36 GB).



(d) Metered latency, full smoothing, 6× heap (4 GB).

Figure 6. User-experienced latency for h2, plotting the latency distribution for 100000 requests using each collector.

yield a lot (GCM). Although its allocation rate (ARA) is high, much of that allocation occurs in the database construction phase. As a result of these factors, h2 has low sensitivity to heap size (GSS).

Recall that because metered latency takes the *earlier* of the actual and notional uniform start times, but leaves the end time unchanged, it can never be lower than the simple latency. The similarity between the metered and simple latency in Figure 6 is consistent with the pauses due to garbage collection being small relative to the query execution time. In fact the 90th percentile latency of the h2 queries is around 3 ms. The heap profile of h2 means that it needs few GCs, and those are performed very quickly and productively, having little impact on the query latency, which is why the metered latency is almost identical to the simple latency, and why a simple collector like Serial is able to perform relatively well.

The reason for the poor latency for the newer collectors can be explained by h2's LBO graph, Figure 16 of the appendix. The new collectors have task clock overheads of around 70 % at the 6×, rapidly rising above 100 %. This means that the collectors are consuming roughly half or more of the available CPU cycles, with the result that individual queries are running noticeably slower.

Looking closely at the 6× heaps, all collectors have slightly worse latency at the far right of the graph (i.e. the tail). This effect is particularly noticeable for Serial. This is because when the heap is a lot larger, although collections will be less frequent, each collection will likely take a little longer since the larger heap will likely hold more live data than the smaller heap. Thus pauses tend to be larger when the heap is larger.

The noticeably worse metered latency for Shenandoah at the 6× heap is explained by Shenandoah's mutator throttling. When it cannot collect fast enough to keep up with application's allocation needs, Shenandoah will throttle application threads to make more hardware available for concurrent garbage collection work. It does this aggressively in the 2× heap, resulting in less application parallelism, and it is able to keep up. However, at 6.0× it is less aggressive, and as a consequence exposes the application threads that run to more GC-induced overheads. This is evident from h2's time LBO, which shows time overheads well over 100 % at 2× due to the mutators being throttled. We also confirm this by reviewing Shenandoah's GC log.

This analysis of h2's latency highlights the importance of a multi-faceted performance analysis, including user-experienced latency, wall clock and task clock LBO, as well as insights into the workload's characteristics revealed by DaCapo's nominal statistics.

6.4 Architectural Sensitivity

Among the twelve nominal statistics most dominant in the PCA analysis are six microarchitectural features: instructions per cycle (UIP), level one data cache miss rate (UDC), last level cache miss rate (ULL), front and back end processor boundedness (USF, USB) and SMT contention (USC). This illustrates that the workloads in the suite exhibit microarchitectural diversity and substantially different degrees of architectural sensitivity. To explore this further and to highlight the analytical utility of the microarchitectural measures included with the DaCapo nominal statistics, we consider four workloads in more detail.

Instructions executed per clock (IPC) indicates how effectively an out of order processor is being utilized. The maximum IPC of any workload is bounded by the number of issue slots in the CPU, which is 6 on the AMD Zen4 machine we use. The IPCs of the DaCapo Chopin workloads range from biojava and jython with high IPCs of 4.76 and 2.76 to h2o and xalan with IPCs of just 0.92 and 0.94 respectively. We will use these four workloads as examples in our exploration of architectural sensitivity. The complete nominal statistics for each of these workloads is included in the `stats` folder of the benchmark suite and the appendix to this paper.

biojava. The high IPC (4.67) of biojava, which analyzes protein sequences, reflects that it is a highly-tuned computational workload. The nominal stats reveal that with the exception of pipeline restarts (UBR), the other nine 'negative' microarchitectural measures are among the lowest in the suite. biojava is fairly insensitive to memory slowdown (PMS) and last level cache size reduction (PLS). Consistent with this, it has above average sensitivity to CPU frequency scaling (PFS) and compiler configuration (PCC, PCS).

jython. The high IPC of jython (2.68) is less impressive than biojava's and has a different explanation. It is a python language implementation that implements an interpreter. It scores very well in most of the metrics, but suffers from very high stalls due to bad speculation due to misprediction (UBP & UBS). It is insensitive to memory speed (PMS) and last level cache size (PLS). This is all consistent with jython spending most of its time in a small but somewhat unpredictable interpreter loop.

xalan. The low IPC of xalan (0.98) is due to a mix of factors, but poor locality is key. It has very high data cache, last level cache, and DTLB miss rates (UDC, ULL, UDT), and is sensitive to last level cache size (PLS).

h2o. Memory performance is an even bigger contributor to h2o's low IPC (0.89). It has the highest back end stalls (USB) and last level cache misses (ULL), and very high data cache and DTLB misses (UDC & UDT). It also has high sensitivity to memory speed (PMS). These are consistent with h2o being a memory-intensive machine learning workload.

7 Conclusion

This paper outlines a problem: when methodological norms cannot keep pace with innovation, we lack the tools we need to notice important regressions. We motivate the problem concretely and then respond to it three ways. First, we contribute a benchmark suite that embodies fourteen years of work, adding eight completely new workloads and bringing existing workloads up to date. The workloads are rich, together constituting a codebase of roughly 16 MLOC. They are diverse, as shown by our principal components analysis. Second, we contribute methodological innovations as part of the suite, allowing researchers to easily analyze their workloads' characteristics, and to measure user-experienced latency. Finally, we contribute a number of methodological recommendations. Although no benchmark suite or methodology can ever be *complete* in any sense, we hope that together, these contributions will strengthen our field's methodological grounding and in doing so help address the problem we used to motivate the paper.

Acknowledgments

The development of the Bach and Chopin releases of the DaCapo benchmark suite was supported by generous donations from Google, Intel, and Oracle. This material is partially based upon work supported by the Australian Research Council under Grant No. DP190103367. The ARM microarchitectural evaluation uses the hardware donated by Ampere Computing. Zixian Cai is supported by an Australian Government Research Training Program Scholarship.

References

[1] 2016. The AVR Simulation and Analysis Framework. https://github.com/avrora-framework

[2] 2023. DaCapo 23.11-chopin. https://github.com/dacapobench/dacapobench/releases/tag/v23.11-chopin

[3] Doug Bagley, Brent Fulgham, and Isaac Gouy. 2004. The Computer Language Benchmarks Game. https://benchmarksgame-team.pages.debian.net/benchmarksgame

[4] Edd Barrett, Carl Friedrich Bolz-Tereick, Rebecca Killick, Sarah Mount, and Laurence Tratt. 2017. Virtual machine warmup blows hot and cold. *Proc. ACM Program. Lang.* 1, OOPSLA (2017), 52:1–52:27. https://doi.org/10.1145/3133876

[5] Emery. D. Berger, Stephen. M. Blackburn, Matthias Hauswirth, and Michael Hicks. 2018. SIGPLAN Empirical Evaluation Checklist. http://www.sigplan.org/Resources/EmpiricalEvaluation/

[6] Stephen M. Blackburn, Amer Diwan, Matthias Hauswirth, Peter F. Sweeney, José Nelson Amaral, Tim Brecht, Lubomír Bulej, Cliff Click, Lieven Eeckhout, Sebastian Fischmeister, Daniel Frampton, Laurie J. Hendren, Michael Hind, Antony L. Hosking, Richard E. Jones, Tomas Kalibera, Nathan Keynes, Nathaniel Nystrom, and Andreas Zeller. 2016. The Truth, The Whole Truth, and Nothing But the Truth: A Pragmatic Guide to Assessing Empirical Evaluations. *ACM Trans. Program. Lang. Syst.* 38, 4 (2016), 15:1–15:20. http://dl.acm.org/citation.cfm?id=2983574

[7] Stephen M. Blackburn, Robin Garner, Chris Hoffmann, Asjad M. Khan, Kathryn S. McKinley, Rotem Bentzur, Amer Diwan, Daniel Feinberg, Daniel Frampton, Samuel Z. Guyer, Martin Hirzel, Antony L. Hosking, Maria Jump, Han Bok Lee, J. Eliot B. Moss, Aashish Phansalkar, Darko Stefanovic, Thomas VanDrunen, Daniel von Dincklage, and Ben Wiedermann. 2006. The DaCapo benchmarks: java benchmarking development and analysis. In *Proceedings of the 21th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2006, October 22-26, 2006, Portland, Oregon, USA*, Peri L. Tarr and William R. Cook (Eds.). ACM, 169–190. https://doi.org/10.1145/1167473.1167488

[8] Stephen M. Blackburn, Kathryn S. McKinley, Robin Garner, Chris Hoffmann, Asjad M. Khan, Rotem Bentzur, Amer Diwan, Daniel Feinberg, Daniel Frampton, Samuel Z. Guyer, Martin Hirzel, Antony L. Hosking, Maria Jump, Han Lee, J. Eliot B. Moss, Aashish Phansalkar, Darko Stefanovic, Thomas VanDrunen, Daniel von Dincklage, and Ben Wiedermann. 2008. Wake up and smell the coffee: evaluation methodology for the 21st century. *Commun. ACM* 51, 8 (2008), 83–89. https://doi.org/10.1145/1378704.1378723

[9] Stephen M. Blackburn, Sharad Singhai, Matthew Hertz, Kathryn S. McKinley, and J. Eliot B. Moss. 2001. Pretenuring for Java. In *Proceedings of the 2001 ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications, OOPSLA 2001, Tampa, Florida, USA, October 14-18, 2001*, Linda M. Northrop and John M. Vlissides (Eds.). ACM, 342–352. https://doi.org/10.1145/504282.504307

[10] Tim Brecht, Eshrat Arjomandi, Chang Li, and Hang Pham. 2001. Controlling Garbage Collection and Heap Growth to Reduce the Execution Time of Java Applications. In *Proceedings of the 2001 ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications, OOPSLA 2001, Tampa, Florida, USA, October 14-18, 2001*, Linda M. Northrop and John M. Vlissides (Eds.). ACM, 353–366. https://doi.org/10.1145/504282.504308

[11] Zixian Cai, Stephen M. Blackburn, Michael D. Bond, and Martin Maas. 2022. Distilling the Real Cost of Production Garbage Collectors. In *International IEEE Symposium on Performance Analysis of Systems and Software, ISPASS 2022, Singapore, May 22-24, 2022*. IEEE, 46–57. https://doi.org/10.1109/ISPASS55109.2022.00005

[12] Maria Carpen-Amarie, Georgios Vavouliotis, Konstantinos Tovletoglou, Boris Grot, and René Müller. 2023. Concurrent GCs and Modern Java Workloads: A Cache Perspective. In *Proceedings of the 2023 ACM SIGPLAN International Symposium on Memory Management, ISMM 2023, Orlando, FL, USA, 18 June 2023*, Stephen M. Blackburn and Erez Petrank (Eds.). ACM, 71–84. https://doi.org/10.1145/3591195.3595269

[13] Perry Cheng and Guy E. Blelloch. 2001. A Parallel, Real-Time Garbage Collector. In *Proceedings of the 2001 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), Snowbird, Utah, USA, June 20-22, 2001*, Michael Burke and Mary Lou Soffa (Eds.). ACM, 125–136. https://doi.org/10.1145/378795.378823

[14] H. J. Curnow and Brian A. Wichmann. 1976. A Synthetic Benchmark. *Comput. J.* 19, 1 (1976), 43–49. https://doi.org/10.1093/COMJNL/19.1.43

[15] David Detlefs, Christine H. Flood, Steve Heller, and Tony Printezis. 2004. Garbage-first garbage collection. In *Proceedings of the 4th International Symposium on Memory Management, ISMM 2004, Vancouver, BC, Canada, October 24-25, 2004*, David F. Bacon and Amer Diwan (Eds.). ACM, 37–48. https://doi.org/10.1145/1029873.1029879

[16] John Ellis, Pete Kovac, Hans Boehm, and Will Clinger. 1997. An Artificial Garbage Collection Benchmark. https://hboehm.info/gc/gc_bench.html

[17] Christine H. Flood and Roman Kennke. 2014. JEP 189: Shenandoah: A Low-Pause-Time Garbage Collector (Experimental). https://openjdk.org/jeps/189

[18] Christine H. Flood, Roman Kennke, Andrew E. Dinn, Andrew Haley, and Roland Westrelin. 2016. Shenandoah: An open-source concurrent compacting garbage collector for OpenJDK. In *Proceedings of the 13th International Conference on Principles and Practices of Programming on the Java Platform: Virtual Machines, Languages, and Tools, Lugano, Switzerland, August 29 - September 2, 2016*, Walter Binder and Petr

Tuma (Eds.). ACM, 13:1–13:9. https://doi.org/10.1145/2972206.2972210

[19] Andy Georges, Lieven Eeckhout, and Dries Buytaert. 2008. Java performance evaluation through rigorous replay compilation. In *Proceedings of the 23rd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2008, October 19-23, 2008, Nashville, TN, USA*, Gail E. Harris (Ed.). ACM, 367–384. https://doi.org/10.1145/1449764.1449794

[20] Bhargav Reddy Godala, Sankara Prasad Ramesh, Gilles A. Pokam, Jared Stark, André Seznec, Dean M. Tullsen, and David I. August. 2024. PDIP: Priority Directed Instruction Prefetching. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 846–861. https://doi.org/10.1145/3620665.3640394

[21] Phillip I. Good and James W. Hardin. 2012. *Common Errors in Statistics (And How to Avoid Them)* (4th ed.). John Wiley & Sons. https://doi.org/10.1002/9781118360125

[22] Xianglong Huang, Stephen M. Blackburn, Kathryn S. McKinley, J. Eliot B. Moss, Zhenlin Wang, and Perry Cheng. 2004. The garbage collection advantage: improving program locality. In *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2004, October 24-28, 2004, Vancouver, BC, Canada*, John M. Vlissides and Douglas C. Schmidt (Eds.). ACM, 69–80. https://doi.org/10.1145/1028976.1028983

[23] Schuyler W. Huck. 2009. *Statistical Misconceptions*. Routledge.

[24] Stefan Karlsson. 2021. JEP 439: Generational ZGC. https://openjdk.org/jeps/439

[25] Jin-Soo Kim and Yarsun Hsu. 2000. Memory system behavior of Java programs: methodology and analysis. In *Proceedings of the 2000 ACM SIGMETRICS international conference on Measurement and modeling of computer systems, Santa Clara, CA, USA, June 18-21, 2000*, Alexandre Brandwajn, Jim Kurose, and Philippe Nain (Eds.). ACM, 264–274. https://doi.org/10.1145/339331.339422

[26] Aapo Kyrola, Guy E. Blelloch, and Carlos Guestrin. 2012. GraphChi: Large-Scale Graph Computation on Just a PC. In *10th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2012, Hollywood, CA, USA, October 8-10, 2012*, Chandu Thekkath and Amin Vahdat (Eds.). USENIX Association, 31–46. https://www.usenix.org/conference/osdi12/technical-sessions/presentation/kyrola

[27] Michael Larabel. 2024. Java Throughput/Latency & Power Efficiency Tuning For AMD EPYC Turin. https://www.phoronix.com/review/java-optimizations-epyc-turin

[28] Doug Lea. 2011. JSR166 software repository. https://gee.cs.oswego.edu/cgi-bin/viewcvs.cgi/jsr166/jsr166/src/test/

[29] Per Liden. 2018. JEP 377: ZGC: A Scalable Low-Latency Garbage Collector (Production). https://openjdk.org/jeps/377

[30] Per Liden and Stefan Karlsson. 2018. JEP 333: ZGC: A Scalable Low-Latency Garbage Collector (Experimental). https://openjdk.org/jeps/333

[31] Ziyi Lin, Darko Marinov, Hao Zhong, Yuting Chen, and Jianjun Zhao. 2015. JaConTeBe: A Benchmark Suite of Real-World Java Concurrency Bugs (T). In *30th IEEE/ACM International Conference on Automated Software Engineering, ASE 2015, Lincoln, NE, USA, November 9-13, 2015*, Myra B. Cohen, Lars Grunske, and Michael Whalen (Eds.). IEEE Computer Society, 178–189. https://doi.org/10.1109/ASE.2015.87

[32] Merriam-Webster.com. 2023. "nominal". https://www.merriam-webster.com/dictionary/nominal

[33] Kent Mitchell. 1995. Re: Does memory leak? comp.lang.ada 1995/03/31. https://archive.legitdata.co/comp.lang.ada/3lhdjd$l6h@rational.rational.com/

[34] Nick Mitchell, Edith Schonberg, and Gary Sevitsky. 2009. Making Sense of Large Heaps. In *ECOOP 2009 - Object-Oriented Programming, 23rd European Conference, Genoa, Italy, July 6-10, 2009. Proceedings (Lecture Notes in Computer Science, Vol. 5653)*, Sophia Drossopoulou (Ed.). Springer, 77–97. https://doi.org/10.1007/978-3-642-03013-0_5

[35] Nick Mitchell, Edith Schonberg, and Gary Sevitsky. 2010. Four Trends Leading to Java Runtime Bloat. *IEEE Softw.* 27, 1 (2010), 56–63. https://doi.org/10.1109/MS.2010.7

[36] Nick Mitchell, Gary Sevitsky, and Harini Srinivasan. 2006. Modeling Runtime Behavior in Framework-Based Applications. In *ECOOP 2006 - Object-Oriented Programming, 20th European Conference, Nantes, France, July 3-7, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 4067)*, Dave Thomas (Ed.). Springer, 429–451. https://doi.org/10.1007/11785477_25

[37] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. 2009. Producing wrong data without doing anything obviously wrong!. In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2009, Washington, DC, USA, March 7-11, 2009*, Mary Lou Soffa and Mary Jane Irwin (Eds.). ACM, 265–276. https://doi.org/10.1145/1508244.1508275

[38] OpenJDK. 2023. JDK 21. https://openjdk.org/projects/jdk/21/

[39] Michael Paleczny, Christopher A. Vick, and Cliff Click. 2001. The Java HotSpot Server Compiler. In *Proceedings of the 1st Java Virtual Machine Research and Technology Symposium, April 23-24, 2001, Monterey, CA, USA*, Saul Wold (Ed.). USENIX. http://www.usenix.org/publications/library/proceedings/jvm01/paleczny.html

[40] Orion Papadakis, Andreas Andronikakis, Nikos Foutris, Michail Papadimitriou, Athanasios Stratikopoulos, Foivos S. Zakkak, Polychronis Xekalakis, and Christos Kotselidis. 2023. A Multifaceted Memory Analysis of Java Benchmarks. In *Proceedings of the 20th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes, MPLR 2023, Cascais, Portugal, 22 October 2023*, Rodrigo Bruno and Eliot Moss (Eds.). ACM, 70–84. https://doi.org/10.1145/3617651.3622978

[41] Aleksandar Prokopec, Andrea Rosà, David Leopoldseder, Gilles Duboscq, Petr Tuma, Martin Studener, Lubomír Bulej, Yudi Zheng, Alex Villazón, Doug Simon, Thomas Würthinger, and Walter Binder. 2019. Renaissance: benchmarking suite for parallel applications on the JVM. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2019, Phoenix, AZ, USA, June 22-26, 2019*, Kathryn S. McKinley and Kathleen Fisher (Eds.). ACM, 31–47. https://doi.org/10.1145/3314221.3314637

[42] Andreas Sewe, Mira Mezini, Aibek Sarimbekov, and Walter Binder. 2011. Da capo con scala: design and analysis of a scala benchmark suite for the java virtual machine. In *Proceedings of the 26th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2011, part of SPLASH 2011, Portland, OR, USA, October 22 - 27, 2011*, Cristina Videira Lopes and Kathleen Fisher (Eds.). ACM, 657–676. https://doi.org/10.1145/2048066.2048118

[43] Aleksey Shipilev. 2020. JEP 379: Shenandoah: A Low-Pause-Time Garbage Collector (Production). https://openjdk.org/jeps/379

[44] SPEC Standard Performance Evaluation Corporation. 2015. The SPECjbb 2015 benchmark. https://www.spec.org/jbb2015/

[45] Phoronix Test Suite. 2025. DaCapo Benchmark. https://openbenchmarking.org/test/pts/dacapobench

[46] Synopsys. 2023. OpenJDK Estimated Cost. https://openhub.net/p/openjdk/estimated_cost

[47] TPC. 1992. TPC-C on-line transaction processing benchmark. https://www.tpc.org/tpcc/

[48] Andrew J. Vickers. 2009. *What is a p-value anyway? 34 Stories to Help You Actually Understand Statistics*. Pearson.

[49] Chenyang Wu and Min Ni. 2017. Dismissing Python Garbage Collection at Instagram. Instagram Engineering. https://instagram-engineering.com/dismissing-python-garbage-collection-at-instagram-4dca40b29172

[50] Thomas Würthinger. 2014. Graal and truffle: modularity and separation of concerns as cornerstones for building a multipurpose runtime. In *13th International Conference on Modularity, MODULARITY '14, Lugano, Switzerland, April 22-26, 2014*, Walter Binder, Erik Ernst, Achille Peternier, and Robert Hirschfeld (Eds.). ACM, 3–4. https://doi.org/10.1145/2584469.2584663

[51] Albert Mingkun Yang and Tobias Wrigstad. 2022. Deep Dive into ZGC: A Modern Garbage Collector in OpenJDK. *ACM Trans. Program. Lang. Syst.* 44, 4 (2022), 22:1–22:34. https://doi.org/10.1145/3538532

[52] Wenyu Zhao, Stephen M. Blackburn, and Kathryn S. McKinley. 2022. Low-latency, high-throughput garbage collection. In *PLDI '22: 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, San Diego, CA, USA, June 13 - 17, 2022*, Ranjit Jhala and Isil Dillig (Eds.). ACM, 76–91. https://doi.org/10.1145/3519939.3523440

A Artifact Appendix

A.1 Abstract

In this artifact, we provide DaCapo 23.11-Chopin release—an overhaul of the DaCapo benchmark suite [7]. As a case study, we use the new suite to measure the contemporary production Java garbage collector performance. The results from the experiments drive the methodological recommendations for performance analysis in the paper.

A.2 Artifact Checklist

- **Program:** DaCapo Chopin benchmarks, which is a contribution of this paper.
- **Run-time environment:** Recent Linux kernel (tested with Ubuntu 20.04 LTS's `5.15.0-113-generic` kernel), OpenJDK 21 (tested with Eclipse Temurin `21.0.3_9`) for running benchmarks, and Python 3.7 or newer for experiment automation (tested with Python 3.10.12). `perf_event_open` syscall access required. sudo access required for Docker.
- **Hardware:** An x86_64 host. 16 cores/32 threads, and 32 GB RAM are recommended. AMD Ryzen 9 7950X with frequency scaling off, and 2 × 32 GB DDR5-4800 DIMM usng JEDEC 40-39-39-77 timing, are required to reproduce the performance results.
- **Execution:** The machine should be otherwise idle.
- **Metrics:** Execution time, task clock, simple and metered user-experience latency in various quantiles.
- **Output:** Console logs with performance metrics, and raw latency CSVs for latency-sensitive benchmarks.
- **Experiments:** Experiments are automated via running-ng v0.4.6, which provides rich customization options.
- **How much disk space required (approximately)?:** 30 GB: a 17 GB image, which is 7 GB after compression.
- **How much time is needed to prepare workflow (approximately)?:** 1 hour
- **How much time is needed to complete experiments (approximately)?:** 1 hour for a simiplified run, and 1 week for a full run.
- **Publicly available?:** Yes
- **Code licenses (if publicly available)?:** Apache License, Version 2.0
- **Workflow framework used?:** running-ng v0.4.6
- **Archived (provide DOI)?:** 10.5281/zenodo.12682890

A.3 Description

A.3.1 How to Access. Download from the Zenodo archive.

A.3.2 Hardware Dependencies. Please refer to the above checklist.

A.3.3 Software Dependencies. Docker Engine required on the host, and all other dependencies are provided in the Docker image.

A.4 Installation

Import the Docker image using `docker load < dacapo-asplos-2025-artifact.tar.gz`. A container can be launched using `docker run -it --cap-add PERFMON --rm -v ./results:/dacapo/results dacapo`. All the below commands are to be run inside the container under `/dacapo` as the working directory.

A.5 Basic Test

`running runbms ./results/ ./experiments/kick-the-tires.yml -p "kick-the-tires" -s 2`.

A.6 Experiment Workflow

Use `running runbms` and provide a folder to store results and the path to the experiment definition file. Please refer to the `README.md` of the artifact for more details.

A.7 Evaluation and Expected Results

Note that the full experiment run is time consuming, and we provide a smaller subset (see `README.md`).

To reproduce the results for the time-space tradeoff (Section 4.2) and lower bound garbage collector overheads (Section 4.5), use `running runbms ./results/ ./experiments/lbo.yml 8 -p "lbo"`. The results can reproduce Figure 1 and Figure 5.

To reproduce the results for user-experienced latency (Section 4.4), use `running runbms ./results/ ./experiments/latency.yml -s 6,2 -p "latency"`. The results can reproduce Figure 3 and Figure 6.

To view the nominal statistics for each benchmark, pass `-p` to the benchmark.

Please refer to the `README.md` of the artifact for sample outputs, and other details.

A.8 Experiment Customization

We use `running-ng` to systematically run experiments, which provides rich customization options. Experiments are defined using composable and customizable YAML files. Please refer to the `README.md` of the artifact for more details.

A.9 Methodology

Submission, reviewing and badging methodology:

- https://www.acm.org/publications/policies/artifact-review-badging
- http://cTuning.org/ae/submission-20201122.html
- http://cTuning.org/ae/reviewing-20201122.html

B Appendix: Benchmark Descriptions and Statistics

In this appendix, we include the complete nominal statistics, LBO graphs, and post-GC heap size graphs for each benchmark in the 23.11-MR2 release. For the latency-sensitive benchmarks, we also include the simple and metered latency graphs for 2× and 6× heaps.

Table 1. The 47 nominal statistics used to characterize the DaCapo Chopin workloads. Not every statistic is available on or applicable to every workload. We use these to conduct principal components analysis of the diversity of the suite, and to inform our performance analysis of the workloads. The nominal statistics for each workload can be printed using DaCapo Chopin's '-p' command line option. The first letter of the metric name reflects its grouping: **A**llocation, **B**ytecode, **G**arbage collection, **P**erformance, and **U**(μ)-architecture.

Metric	Description
AOA	nominal average object size (bytes)
AOL	nominal 90-percentile object size (bytes)
AOM	nominal median object size (bytes)
AOS	nominal 10-percentile object size (bytes)
ARA	nominal allocation rate (bytes / μsec)
BAL	nominal aaload per usec
BAS	nominal aastore per usec
BEF	nominal execution focus / dominance of hot code
BGF	nominal getfield per usec
BPF	nominal putfield per usec
BUB	nominal thousands of unique bytecodes executed
BUF	nominal thousands of unique function calls executed
GCA	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	nominal GC count at 2X minimum heap size (G1)
GCM	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	nominal percentage of time spent in GC pauses at 2X minimum heap size (G1)
GLK	nominal percent 10th iteration memory leakage (10 iterations / 1 iterations)
GMD	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	nominal minimum heap size (MB) for default size *without* compressed pointers
GMV	nominal minimum heap size (MB) for vlarge size configuration (with compressed pointers)
GSS	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	nominal percentage slowdown due to forced c2 compilation compared to tiered baseline (compiler cost)
PCS	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivity to compiler)
PET	nominal execution time (sec)
PFS	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	nominal percentage slowdown due to using the interpreter (sensitivity to interpreter)
PKP	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	nominal percentage slowdown due to slower DRAM (memory speed sensitivity)
PPE	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	nominal iterations to warm up to within 1.5 % of best
UAA	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	nominal backend bound (memory)
UBP	nominal 1000 x bad speculation: mispredicts
UBR	nominal 1000000 x bad speculation: pipeline restarts
UBS	nominal 1000 x bad speculation
UDC	nominal data cache misses per K instructions
UDT	nominal DTLB misses per M instructions
UIP	nominal 100 x instructions per cycle (IPC)
ULL	nominal LLC misses M instructions
USB	nominal 100 x back end bound
USC	nominal 1000 x SMT contention
USF	nominal 100 x front end bound

Table 2. The twelve most determinant nominal statistics as revealed by our principal components analysis, and their values for each of the DaCapo Chopin benchmarks. Each cell presents the rank of the respective benchmark with respect to that nominal statistic (black) and the concrete value reported (grey).

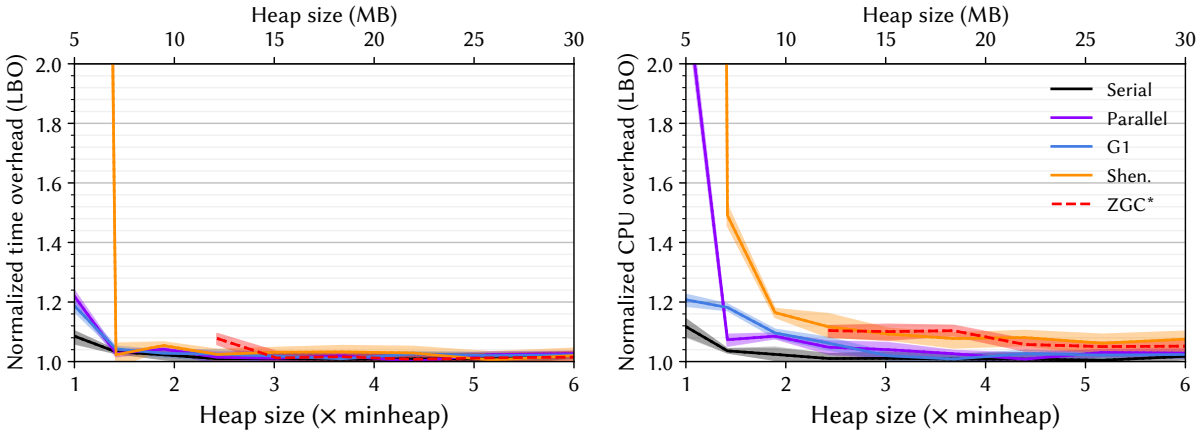
Benchmark	GLK	GMU	PET	PFS	PKP	PWU	UAA	UAI	UBP	UBR	UBS	USF
avrora	9 0	22 7	6 4	4 18	1 56	13 2	19 53	22 -19	21 19	22 164	21 20	1 51
batik	9 0	3 229	13 2	1 20	21 0	9 4	16 80	12 25	8 52	4 2388	8 55	19 10
biojava	9 0	5 183	5 5	3 19	16 1	20 1	5 121	16 14	17 29	1 3487	16 33	20 6
cassandra	2 46	8 142	3 6	18 2	5 11	13 2	1 168	21 -9	15 37	17 619	15 38	4 40
eclipse	8 1	7 167	1 8	4 18	9 6	11 3	12 92	5 36	3 97	13 994	3 98	11 30
fop	9 0	20 17	17 1	11 13	12 2	2 8	18 76	6 35	1 134	3 2653	1 137	8 32
graphchi	9 0	6 179	8 3	10 14	16 1	13 2	6 112	6 35	22 5	16 704	22 5	22 4
h2	9 0	1 903	13 2	17 5	21 0	13 2	4 127	14 24	17 29	14 920	18 30	17 17
h2o	4 17	12 73	8 3	15 9	11 4	9 4	8 102	9 32	17 29	10 1126	18 30	15 18
jme	9 0	17 29	2 7	21 0	6 8	20 1	22 2	20 1	4 89	8 1226	4 90	8 32
jython	9 0	14 31	8 3	1 20	16 1	1 9	8 102	9 32	5 85	11 1105	5 86	13 21
kafka	9 0	4 208	3 6	20 1	2 25	11 3	20 19	17 13	16 30	20 547	17 31	3 43
luindex	9 0	14 31	8 3	4 18	12 2	13 2	13 90	12 25	2 109	2 3280	2 112	18 12
lusearch	9 0	19 21	13 2	13 11	7 7	2 8	14 87	1 56	11 40	18 596	11 41	12 23
pmd	7 5	2 269	17 1	13 11	16 1	4 7	6 112	2 47	13 38	7 1295	12 39	13 21
spring	9 0	13 70	13 2	16 8	7 7	13 2	14 87	11 30	7 60	6 1475	7 61	8 32
sunflow	9 0	14 31	8 3	8 16	16 1	6 6	11 98	15 19	20 21	5 2380	20 24	21 5
tomcat	9 0	18 24	6 4	18 2	3 19	13 2	21 14	19 4	10 44	19 584	10 45	2 45
tradebeans	3 26	9 141	17 1	7 17	12 2	6 6	3 144	3 42	13 38	9 1187	12 39	5 38
tradesoap	6 6	11 115	17 1	8 16	12 2	8 5	2 147	8 34	6 73	12 1087	6 74	7 35
xalan	5 7	20 17	17 1	12 12	4 14	20 1	10 101	17 13	12 39	15 785	12 39	6 36
zxing	1 120	10 127	17 1	22 -1	10 5	4 7	17 77	3 42	8 52	21 374	9 52	15 18

B.1 Avrora

This workload is based on the AVRORA simulation and analysis framework for AVR micro-controllers [1]. It is one of the most unusual workloads in DaCapo Chopin. Each simulated entity in the microcontroller is represented by a thread, so there is a high degree of fine-grained concurrency. It has the second lowest allocation rate in the suite (ARA), the highest percentage of time spent in the kernel (PKP), is very insensitive to compiler selection (PCS), and is the most front end-bound workload (USF). The last three of these are likely due to its very heavy use of locking primitives. It has low back end stalls (USB), and low bad speculation (UBS). Although avrora is highly concurrent, it has very low parallel efficiency (PPE).

Table 3. Complete nominal statistics for avrora. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	1	34	18	28	58	211	nominal average object size (bytes)
AOL	1	32	19	24	56	200	nominal 90-percentile object size (bytes)
AOM	9	32	2	24	32	48	nominal median object size (bytes)
AOS	10	24	1	16	24	24	nominal 10-percentile object size (bytes)
ARA	1	56	19	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	4	31	13	0	34	2204	nominal aaload per usec
BAS	4	0	13	0	1	126	nominal aastore per usec
BEF	6	5	9	1	4	29	nominal execution focus / dominance of hot code
BGF	5	692	10	0	527	32087	nominal getfield per usec
BPF	7	206	7	0	83	3863	nominal putfield per usec
BUB	4	33	12	1	34	177	nominal thousands of unique bytecodes executed
BUF	5	4	10	0	4	29	nominal thousands of unique function calls
GCA	1	80	20	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	2	551	18	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	2	80	19	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	4	1	15	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	6	0	9	0	0	120	nominal percent 10th iteration memory leakage
GMD	0	5	22	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	1	15	19	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	2	5	18	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	0	7	22	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	3	18	17	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	3	33	14	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	2	83	19	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	1	7	21	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	8	4	6	1	3	8	nominal execution time (sec)
PFS	9	18	4	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	1	7	21	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	10	56	1	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	3	2	16	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	6	6	10	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	2	3	19	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	10	4	2	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	5	2	13	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	2	53	19	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	0	-19	22	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	5	23	12	5	23	41	nominal backend bound (memory)
UBP	1	19	21	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	0	164	22	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	1	20	21	5	39	137	nominal 1000 x bad speculation
UDC	8	18	5	2	12	27	nominal data cache misses per K instructions
UDT	4	131	14	14	174	576	nominal DTLB misses per M instructions
UIP	3	113	17	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	6	3398	9	335	2645	8506	nominal LLC misses per M instructions
USB	4	26	15	7	29	53	nominal 100 x back end bound
USC	1	7	20	1	52	351	nominal 1000 x SMT contention
USF	10	51	1	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 7. Lower bounds on the overheads [11] for avrora for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

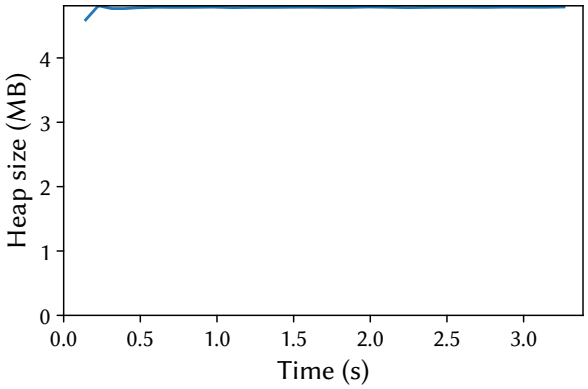


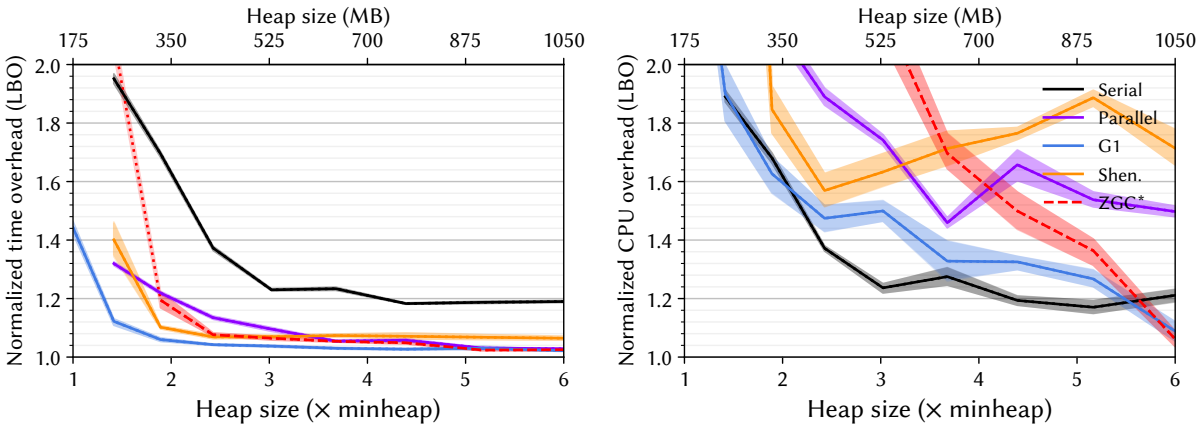
Figure 8. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.

B.2 Batik

This workload uses the Batik Apache scalable vector graphics (SVG) toolkit to render a number of svg files. Batik consists of nearly 400 K lines of Java code. It has very low allocation rate (ARA). It has the lowest memory turnover (GTO) and is the most sensitive workload to CPU frequency scaling (PFS). It is one of the most back end bound (USB) and one of the highest pipeline restarts (UBR), yet has one of the highest IPCs (UIP). Its large configuration has a 1.7 GB minimum heap size.

Table 4. Complete nominal statistics for batik. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	5	58	10	28	58	211	nominal average object size (bytes)
AOL	6	72	9	24	56	200	nominal 90-percentile object size (bytes)
AOM	9	32	2	24	32	48	nominal median object size (bytes)
AOS	10	24	1	16	24	24	nominal 10-percentile object size (bytes)
ARA	1	506	18	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	6	41	9	0	34	2204	nominal aaload per usec
BAS	4	0	13	0	1	126	nominal aastore per usec
BEF	5	4	11	1	4	29	nominal execution focus / dominance of hot code
BGF	1	126	18	0	527	32087	nominal getfield per usec
BPF	2	28	17	0	83	3863	nominal putfield per usec
BUB	4	32	13	1	34	177	nominal thousands of unique bytecodes executed
BUF	5	4	10	0	4	29	nominal thousands of unique function calls
GCA	10	121	2	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	1	111	20	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	10	132	2	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	7	9	8	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	6	0	9	0	0	120	nominal percent 10th iteration memory leakage
GMD	8	175	5	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	8	1759	4	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	5	19	11	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	9	229	3	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	4	40	15	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	0	3	20	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	8	306	5	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	3	24	17	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	5	2	13	1	3	8	nominal execution time (sec)
PFS	10	20	1	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	3	24	17	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	1	0	21	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	2	0	19	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	4	2	15	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	3	4	16	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	7	1	7	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	6	4	9	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	3	80	16	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	5	25	12	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	9	37	3	5	23	41	nominal backend bound (memory)
UBP	7	52	8	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	9	2388	4	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	7	55	8	5	39	137	nominal 1000 x bad speculation
UDC	2	4	19	2	12	27	nominal data cache misses per K instructions
UDT	2	50	19	14	174	576	nominal DTLB misses per M instructions
UIP	8	228	5	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	3	1872	16	335	2645	8506	nominal LLC misses per M instructions
USB	9	46	3	7	29	53	nominal 100 x back end bound
USC	2	16	19	1	52	351	nominal 1000 x SMT contention
USF	2	10	19	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 9. Lower bounds on the overheads [11] for batik for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

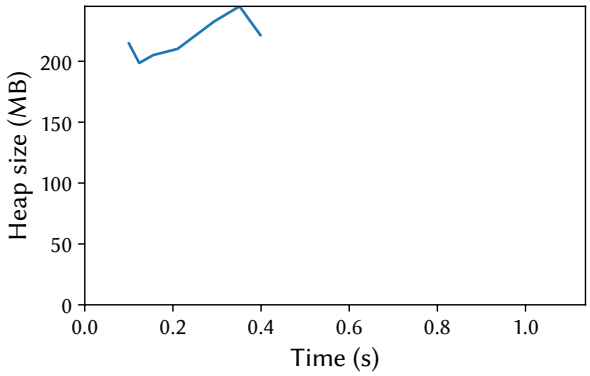


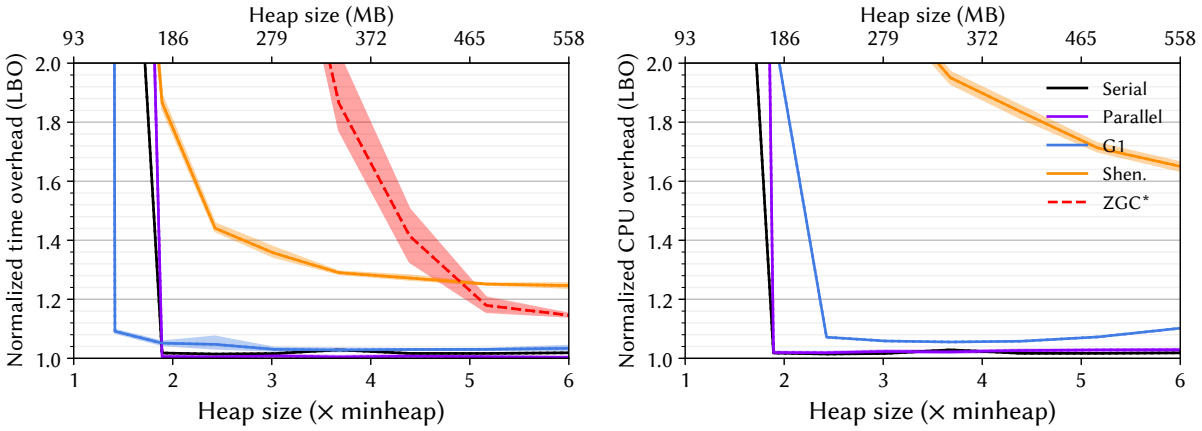
Figure 10. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.

B.3 Biojava

(**New**) This workload uses the BioJava framework to generate ten physico-chemical properties of protein sequences of different sizes. BioJava consists of over 300 K lines of Java code. The workload has the tightest hot code focus in the suite (BEF), the highest IPC (UIP), the lowest data cache misses (UDC), very low DTLB misses (UDT), last level cache misses (ULL), front and back end stalls (USB, USF), and SMT contention (USC), as well as the smallest average object size (AOA) and the largest 10th percentile object size (AOS). It is one of the most sensitive benchmarks to heap size (GSS). Its large configuration has a 1 GB minimum heap size.

Table 5. Complete nominal statistics for biojava. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	0	28	20	28	58	211	nominal average object size (bytes)
AOL	0	24	20	24	56	200	nominal 90-percentile object size (bytes)
AOM	3	24	14	24	32	48	nominal median object size (bytes)
AOS	10	24	1	16	24	24	nominal 10-percentile object size (bytes)
ARA	4	2041	12	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	1	0	18	0	34	2204	nominal aaload per usec
BAS	4	0	13	0	1	126	nominal aastore per usec
BEF	9	28	2	1	4	29	nominal execution focus / dominance of hot code
BGF	2	171	17	0	527	32087	nominal getfield per usec
BPF	1	2	19	0	83	3863	nominal putfield per usec
BUB	2	18	17	1	34	177	nominal thousands of unique bytecodes executed
BUF	3	2	15	0	4	29	nominal thousands of unique function calls
GCA	7	106	8	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	7	2172	8	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	5	98	12	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	4	1	15	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	6	0	9	0	0	120	nominal percent 10th iteration memory leakage
GMD	6	93	10	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	7	1027	6	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	3	7	16	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	8	183	5	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	10	7107	2	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	6	102	8	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	6	224	9	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	6	106	9	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	8	5	5	1	3	8	nominal execution time (sec)
PFS	9	19	3	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	6	106	9	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	3	1	16	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	3	1	17	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	2	0	19	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	5	5	13	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	3	0	16	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	1	1	20	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	8	121	5	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	3	14	16	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	1	15	21	5	23	41	nominal backend bound (memory)
UBP	3	29	17	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	10	3487	1	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	3	33	16	5	39	137	nominal 1000 x bad speculation
UDC	0	2	22	2	12	27	nominal data cache misses per K instructions
UDT	1	30	21	14	174	576	nominal DTLB misses per M instructions
UIP	10	476	1	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	2	1427	19	335	2645	8506	nominal LLC misses per M instructions
USB	1	19	21	7	29	53	nominal 100 x back end bound
USC	4	41	14	1	52	351	nominal 1000 x SMT contention
USF	1	6	20	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 11. Lower bounds on the overheads [11] for biojava for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

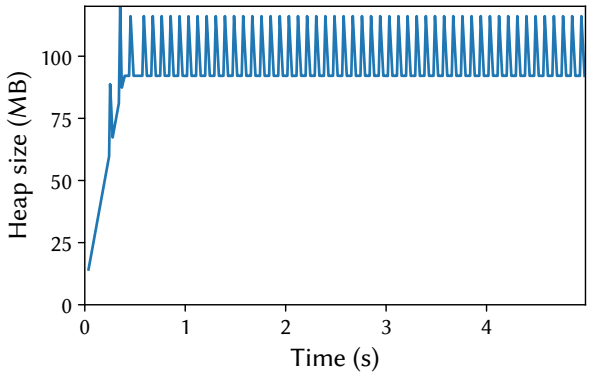


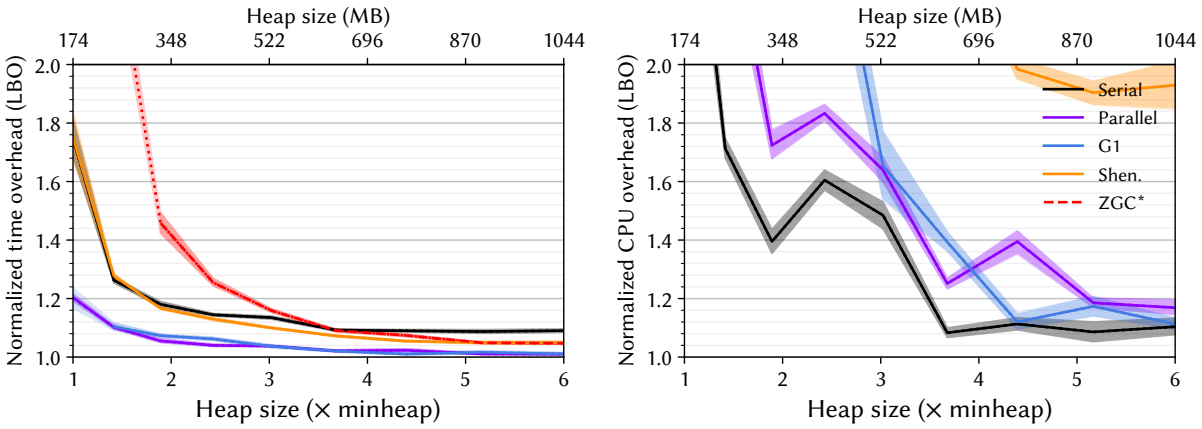
Figure 12. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.

B.4 Cassandra

(**New**) This workload executes the Yahoo! Cloud Serving Benchmark (YCSB) over the Apache Cassandra NoSQL database management system, which consists of nearly 700 K lines of Java code. It is a request-based workload, reporting request latencies. cassandra is one of the least GC-intensive workloads in the suite (GCP), but it suffers memory leakage (GLK). It has the highest DTLB miss rage rate (UDT), one of the highest data cache miss rates (UDC), one of the highest last level cache miss rates (ULL), and is one of the most front end bound (USF), yielding low IPC (UIP).

Table 6. Complete nominal statistics for cassandra. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	3	40	15	28	58	211	nominal average object size (bytes)
AOL	5	56	11	24	56	200	nominal 90-percentile object size (bytes)
AOM	9	32	2	24	32	48	nominal median object size (bytes)
AOS	10	24	1	16	24	24	nominal 10-percentile object size (bytes)
ARA	3	890	15	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	3	9	15	0	34	2204	nominal aaload per usec
BAS	6	1	9	0	1	126	nominal aastore per usec
BEF	3	3	15	1	4	29	nominal execution focus / dominance of hot code
BGF	4	314	13	0	527	32087	nominal getfield per usec
BPF	4	57	13	0	83	3863	nominal putfield per usec
BUB	7	114	6	1	34	177	nominal thousands of unique bytecodes executed
BUF	8	18	5	0	4	29	nominal thousands of unique function calls
GCA	6	103	10	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	3	659	16	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	7	101	8	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	4	1	15	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	10	46	2	0	0	120	nominal percent 10th iteration memory leakage
GMD	7	174	7	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	5	174	10	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	9	77	3	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	7	142	8	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	2	14	19	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	4	34	13	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	1	60	21	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	3	31	16	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	9	6	3	1	3	8	nominal execution time (sec)
PFS	2	2	18	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	3	31	16	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	8	11	5	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	4	3	15	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	4	2	15	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	7	13	7	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	3	0	16	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	5	2	13	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	10	168	1	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	1	-9	21	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	6	26	9	5	23	41	nominal backend bound (memory)
UBP	4	37	15	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	3	619	17	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	4	38	15	5	39	137	nominal 1000 x bad speculation
UDC	10	24	2	2	12	27	nominal data cache misses per K instructions
UDT	10	576	1	14	174	576	nominal DTLB misses per M instructions
UIP	2	108	19	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	9	5719	3	335	2645	8506	nominal LLC misses per M instructions
USB	5	29	11	7	29	53	nominal 100 x back end bound
USC	6	92	10	1	52	351	nominal 1000 x SMT contention
USF	9	40	4	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 13. Lower bounds on the overheads [11] for cassandra for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

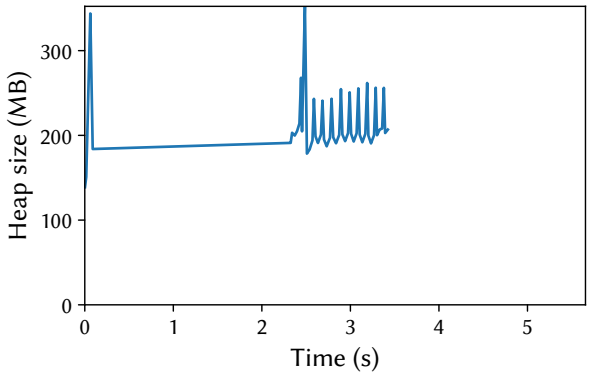
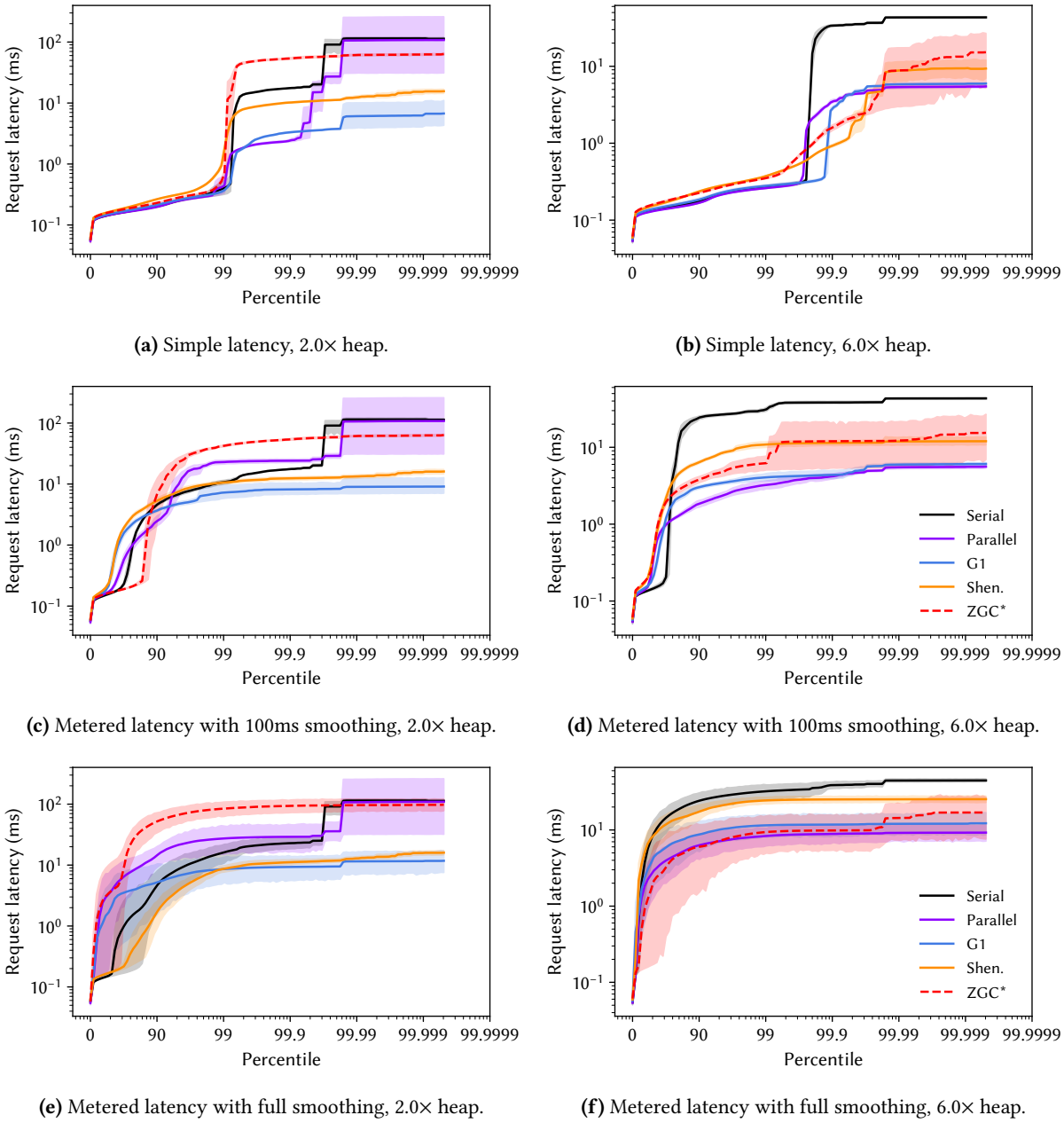


Figure 14. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.



(a) Simple latency, 2.0× heap.

(b) Simple latency, 6.0× heap.

(c) Metered latency with 100ms smoothing, 2.0× heap.

(d) Metered latency with 100ms smoothing, 6.0× heap.

(e) Metered latency with full smoothing, 2.0× heap.

(f) Metered latency with full smoothing, 6.0× heap.

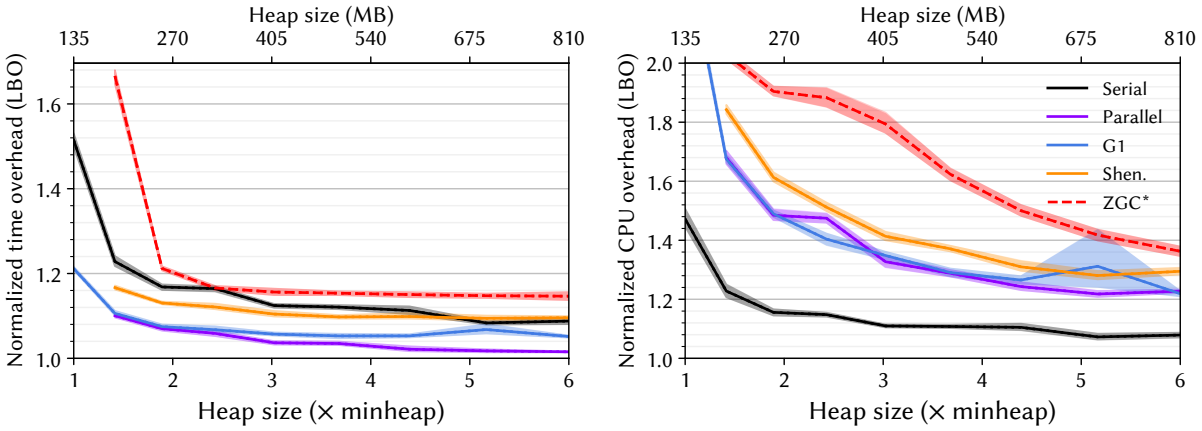
Figure 15. Distribution of request latencies for cassandra for each of OpenJDK 21's six production collectors. The figures in the left top row simply plot the request latencies, while the figures in the middle and the bottom rows use DaCapo's metered latency, which models a request queue and the cascading effect of delays.

B.5 Eclipse

This workload executes the eclipse performance tests. Eclipse is a widely used IDE consisting of over 6 M lines of Java code. It has the highest concentration of hot code (BEF) and is one of the most sensitive workload to compiler configuration (PCC, PCS) and one of the most sensitive to last level cache size (PLS) and CPU frequency scaling (PFS). It suffers high bad speculation due to mispredicts (UBS, UBP).

Table 7. Complete nominal statistics for eclipse. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	7	84	6	28	58	211	nominal average object size (bytes)
AOL	8	88	4	24	56	200	nominal 90-percentile object size (bytes)
AOM	9	32	2	24	32	48	nominal median object size (bytes)
AOS	10	24	1	16	24	24	nominal 10-percentile object size (bytes)
ARA	3	1043	14	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	1	0	18	0	34	2204	nominal aaload per usec
BAS	4	0	13	0	1	126	nominal aastore per usec
BEF	10	29	1	1	4	29	nominal execution focus / dominance of hot code
BGF	0	0	20	0	527	32087	nominal getfield per usec
BPF	0	0	20	0	83	3863	nominal putfield per usec
BUB	0	1	20	1	34	177	nominal thousands of unique bytecodes executed
BUF	0	0	20	0	4	29	nominal thousands of unique function calls
GCA	2	83	19	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	5	997	11	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	1	77	20	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	5	2	12	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	7	1	8	0	0	120	nominal percent 10th iteration memory leakage
GMD	7	135	8	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	4	139	12	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	5	13	12	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	7	167	7	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	2	16	18	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	5	52	11	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	9	349	4	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	9	224	3	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	10	8	1	1	3	8	nominal execution time (sec)
PFS	9	18	4	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	9	224	3	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	6	6	9	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	7	23	7	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	5	5	12	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	5	5	13	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	3	0	16	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	5	3	11	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	5	92	12	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	8	36	5	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	5	25	11	5	23	41	nominal backend bound (memory)
UBP	9	97	3	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	5	994	13	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	9	98	3	5	39	137	nominal 1000 x bad speculation
UDC	5	11	13	2	12	27	nominal data cache misses per K instructions
UDT	7	283	8	14	174	576	nominal DTLB misses per M instructions
UIP	6	178	10	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	6	3108	10	335	2645	8506	nominal LLC misses per M instructions
USB	5	29	11	7	29	53	nominal 100 x back end bound
USC	3	30	16	1	52	351	nominal 1000 x SMT contention
USF	5	30	11	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 16. Lower bounds on the overheads [11] for eclipse for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

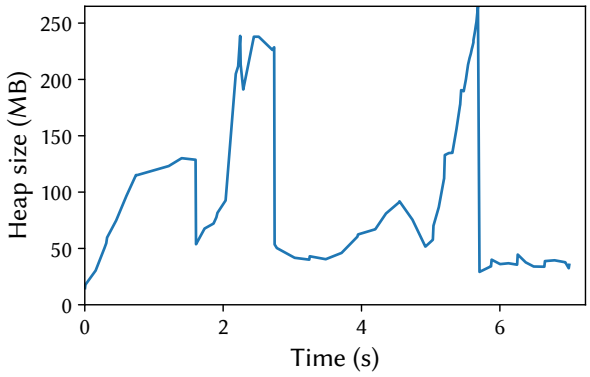


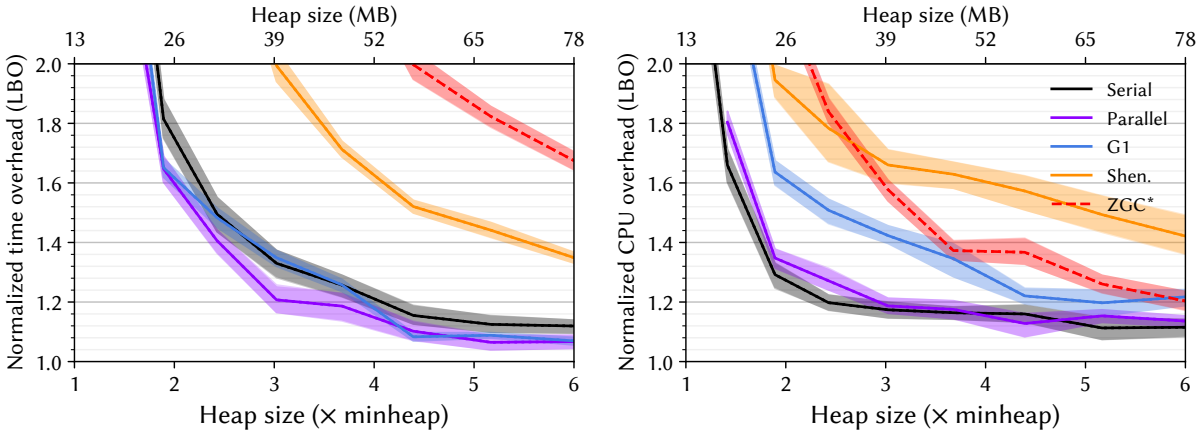
Figure 17. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.

B.6 Fop

This workload uses the Apache fop print formatter to render a number of XLS-FO files as pdfs. The Apache fop framework consists of over 400 K lines of Java code. fop has the largest number of unique bytecodes executed (BUB), and is one of the slowest benchmark to warm up (PWU). It has one of the highest percentages of time spent in GC pauses at a 2× heap (GCP), and is one of the most heap-size sensitive workloads (GSS). It suffers from bad speculation (UBS, UBP).

Table 8. Complete nominal statistics for fop. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	5	58	10	28	58	211	nominal average object size (bytes)
AOL	5	56	11	24	56	200	nominal 90-percentile object size (bytes)
AOM	9	32	2	24	32	48	nominal median object size (bytes)
AOS	10	24	1	16	24	24	nominal 10-percentile object size (bytes)
ARA	6	3340	9	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	5	34	11	0	34	2204	nominal aaload per usec
BAS	7	6	6	0	1	126	nominal aastore per usec
BEF	1	1	19	1	4	29	nominal execution focus / dominance of hot code
BGF	5	527	11	0	527	32087	nominal getfield per usec
BPF	6	95	9	0	83	3863	nominal putfield per usec
BUB	10	177	1	1	34	177	nominal thousands of unique bytecodes executed
BUF	9	26	3	0	4	29	nominal thousands of unique function calls
GCA	7	107	7	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	5	841	13	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	7	107	7	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	9	23	3	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	6	0	9	0	0	120	nominal percent 10th iteration memory leakage
GMD	1	13	20	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GMS	4	9	15	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	1	17	20	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	7	755	7	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	5	75	10	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	10	1083	1	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	2	23	18	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	3	1	17	1	3	8	nominal execution time (sec)
PFS	5	13	11	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	2	23	18	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	5	2	12	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	9	37	3	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	8	12	6	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	6	9	9	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	3	0	16	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	10	8	2	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	2	76	18	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	8	35	6	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	4	21	15	5	23	41	nominal backend bound (memory)
UBP	10	134	1	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	9	2653	3	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	10	137	1	5	39	137	nominal 1000 x bad speculation
UDC	6	14	10	2	12	27	nominal data cache misses per K instructions
UDT	5	174	12	14	174	576	nominal DTLB misses per M instructions
UIP	6	181	9	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	4	2138	15	335	2645	8506	nominal LLC misses per M instructions
USB	2	25	18	7	29	53	nominal 100 x back end bound
USC	2	19	18	1	52	351	nominal 1000 x SMT contention
USF	7	32	8	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 18. Lower bounds on the overheads [11] for fop for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

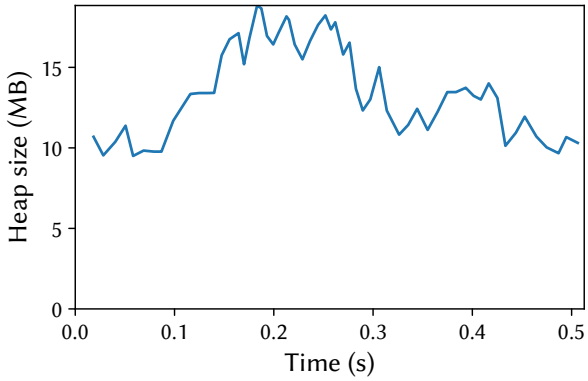


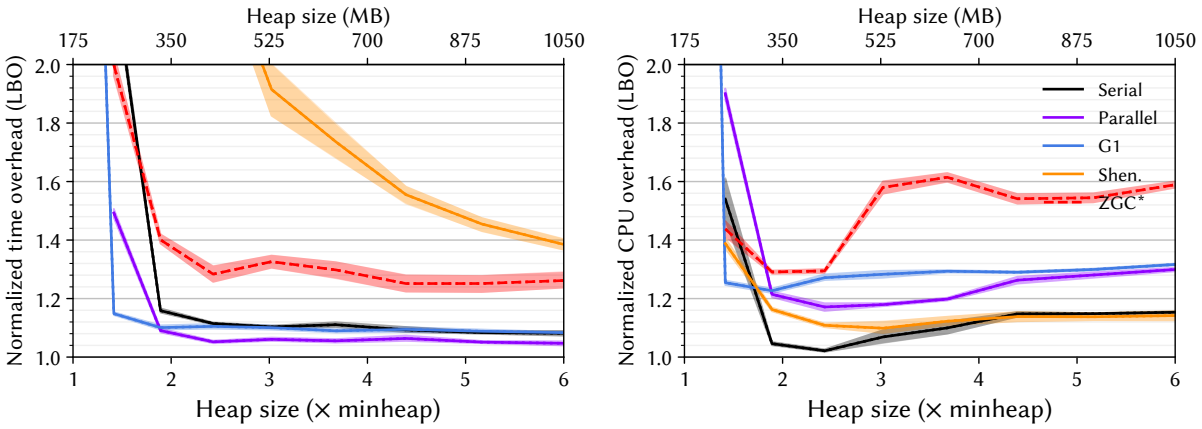
Figure 19. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.

B.7 Graphchi

(**New**) This workload performs ALS matrix factorization using the Netflix Challenge dataset with the Java port of the GraphChi engine [26]. It has one of the lowest number of unique bytecodes executed (BUB), one of the highest focuses of hot code (BEF) and one of the highest aaload and getfield rates (BAL, BGF). It is the most sensitive workload to compiler configuration (PCS). It has the lowest front end stalls (USF) and bad speculation (UBP), as well as low DTLB, and data cache miss rates (UDT, UDC), yielding one of the best IPCs (UIP) despite suffering SMT contention (USC) and being backend bound (USB). In its large configuration, it has a 1.1 GB minimum heap size.

Table 9. Complete nominal statistics for graphchi. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	8	110	4	28	58	211	nominal average object size (bytes)
AOL	9	160	2	24	56	200	nominal 90-percentile object size (bytes)
AOM	3	24	14	24	32	48	nominal median object size (bytes)
AOS	2	16	16	16	24	24	nominal 10-percentile object size (bytes)
ARA	5	2737	10	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	10	2204	1	0	34	2204	nominal aaload per usec
BAS	6	1	9	0	1	126	nominal aastore per usec
BEF	9	12	3	1	4	29	nominal execution focus / dominance of hot code
BGF	9	9217	3	0	527	32087	nominal getfield per usec
BPF	3	43	15	0	83	3863	nominal putfield per usec
BUB	1	8	19	1	34	177	nominal thousands of unique bytecodes executed
BUF	1	1	18	0	4	29	nominal thousands of unique function calls
GCA	9	113	4	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	6	1262	10	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	8	108	6	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	5	2	12	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	6	0	9	0	0	120	nominal percent 10th iteration memory leakage
GMD	8	175	5	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	8	1183	5	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	10	141	2	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	8	179	6	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	6	382	10	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	4	38	12	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	7	276	7	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	10	323	1	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	7	3	8	1	3	8	nominal execution time (sec)
PFS	6	14	10	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	10	323	1	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	3	1	16	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	4	5	14	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	7	10	7	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	6	9	9	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	7	1	7	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	5	2	13	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	8	112	6	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	8	35	6	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	2	19	18	5	23	41	nominal backend bound (memory)
UBP	0	5	22	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	3	704	16	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	0	5	22	5	39	137	nominal 1000 x bad speculation
UDC	1	3	20	2	12	27	nominal data cache misses per K instructions
UDT	1	45	20	14	174	576	nominal DTLB misses per M instructions
UIP	9	234	4	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	3	1746	17	335	2645	8506	nominal LLC misses per M instructions
USB	8	38	6	7	29	53	nominal 100 x back end bound
USC	9	192	4	1	52	351	nominal 1000 x SMT contention
USF	0	4	22	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 20. Lower bounds on the overheads [11] for graphchi for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

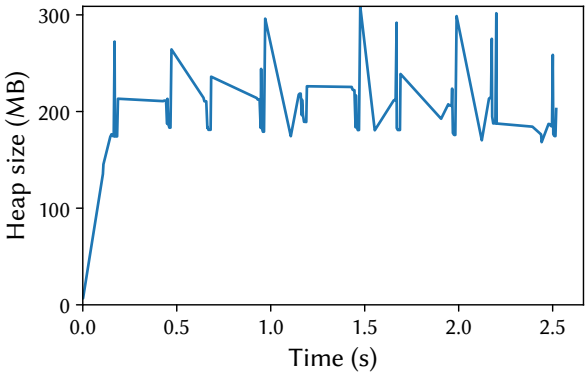


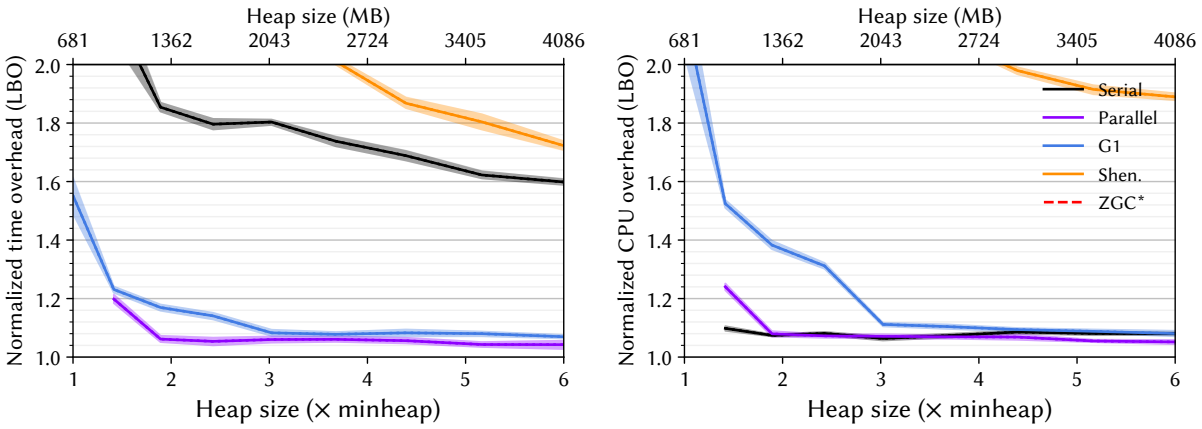
Figure 21. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.

B.8 H2

This workload is latency-sensitive. It executes a TPC-C-like transactional workload over the H2 database configured for in-memory operation. h2 has about 240 K lines of Java source code. It has very low memory turnover (GTO) and has high sensitivity to slower DRAM speeds (PMS). It has high DTLB and data cache miss rates (UDT, UDC), high SMT contention (USC) and is very backend memory bound (UBM). It spends very little time in kernel mode (PKP). It has the largest heap sizes for default, large, and vlarge configurations (681 MB, 10.2 GB, and 20.6 GB).

Table 10. Complete nominal statistics for h2. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	3	41	14	28	58	211	nominal average object size (bytes)
AOL	5	64	10	24	56	200	nominal 90-percentile object size (bytes)
AOM	9	32	2	24	32	48	nominal median object size (bytes)
AOS	10	24	1	16	24	24	nominal 10-percentile object size (bytes)
ARA	9	11858	2	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	8	234	5	0	34	2204	nominal aaload per usec
BAS	8	28	4	0	1	126	nominal aastore per usec
BEF	7	7	7	1	4	29	nominal execution focus / dominance of hot code
BGF	7	3677	6	0	527	32087	nominal getfield per usec
BPF	8	601	4	0	83	3863	nominal putfield per usec
BUB	1	17	18	1	34	177	nominal thousands of unique bytecodes executed
BUF	3	2	15	0	4	29	nominal thousands of unique function calls
GCA	5	98	13	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	3	552	17	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	2	82	18	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	5	4	11	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	6	0	9	0	0	120	nominal percent 10th iteration memory leakage
GMD	10	681	1	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	10	10201	1	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	8	69	6	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	10	903	1	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GMV	10	20641	1	371	1123	20641	nominal minimum heap size (MB) for vlarge size configuration (with compressed pointers)
GSS	3	38	16	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	2	30	16	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	2	87	18	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	4	55	14	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	5	2	13	1	3	8	nominal execution time (sec)
PFS	3	5	17	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	4	55	14	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	1	0	21	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	9	31	4	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	10	40	2	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	8	24	5	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	7	1	7	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	5	2	13	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	9	127	4	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	4	24	14	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	10	40	2	5	23	41	nominal backend bound (memory)
UBP	3	29	17	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	4	920	14	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	2	30	18	5	39	137	nominal 1000 x bad speculation
UDC	7	16	8	2	12	27	nominal data cache misses per K instructions
UDT	9	476	4	14	174	576	nominal DTLB misses per M instructions
UIP	5	135	13	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	7	4315	7	335	2645	8506	nominal LLC misses per M instructions
USB	9	43	4	7	29	53	nominal 100 x back end bound
USC	8	140	6	1	52	351	nominal 1000 x SMT contention
USF	3	17	17	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 22. Lower bounds on the overheads [11] for h2 for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

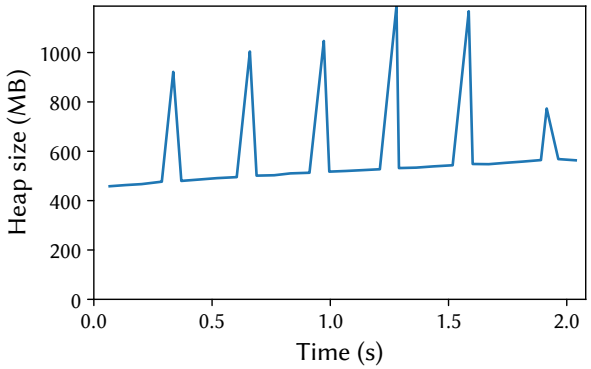
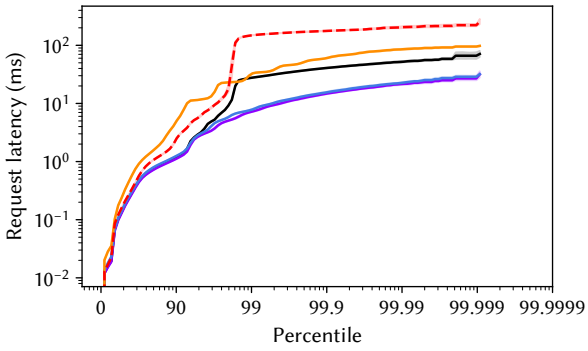
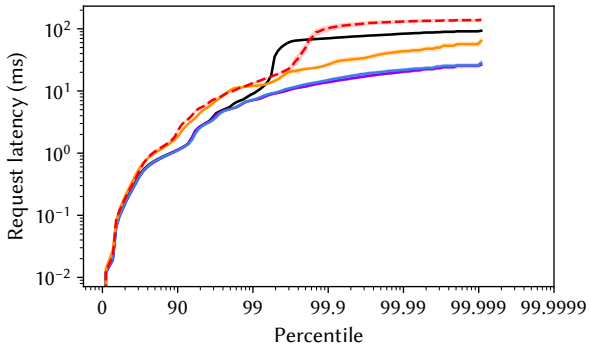


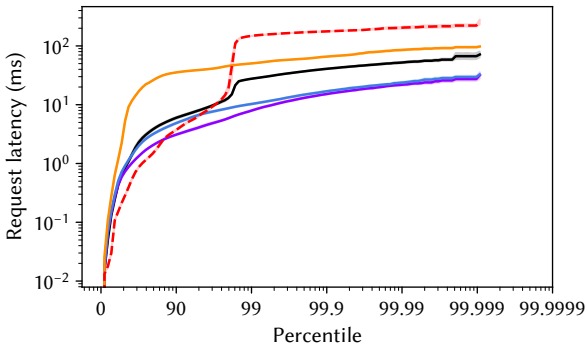
Figure 23. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.



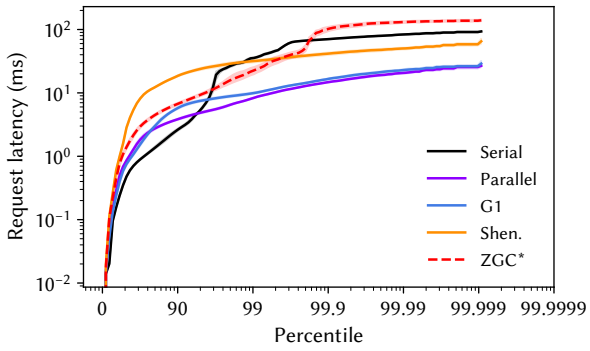
(a) Simple latency, 2.0× heap.



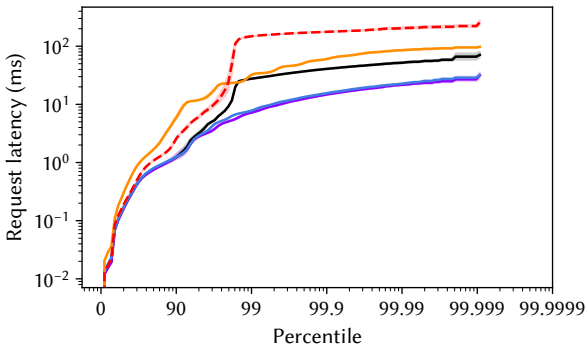
(b) Simple latency, 6.0× heap.



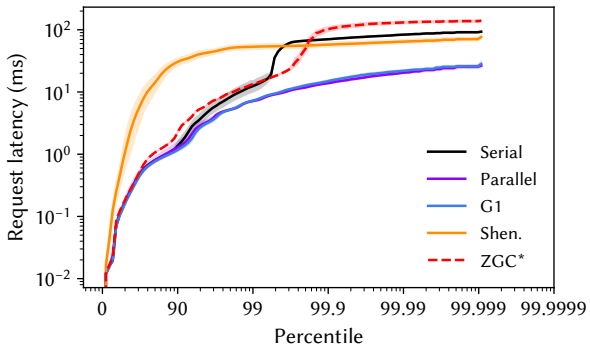
(c) Metered latency with 100ms smoothing, 2.0× heap.



(d) Metered latency with 100ms smoothing, 6.0× heap.



(e) Metered latency with full smoothing, 2.0× heap.



(f) Metered latency with full smoothing, 6.0× heap.

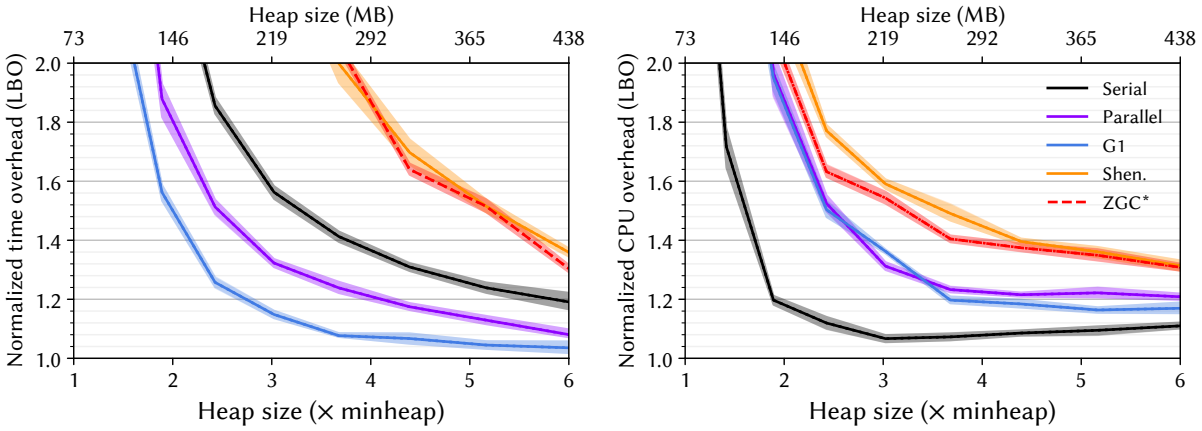
Figure 24. Distribution of request latencies for h2 for each of OpenJDK 21's six production collectors. The figures in the left top row simply plot the request latencies, while the figures in the middle and the bottom rows use DaCapo's metered latency, which models a request queue and the cascading effect of delays.

B.9 H2o

(**New**) This workloads performs machine learning using the H2O ML platform and the 201908-citibike-tripdata dataset. H2O consists of about 330 K lines of Java code. h2o is very sensitive to slower DRAM speeds (PMS) and exhibits one of the highest standard deviations among invocations (PSD). It has the one of the smallest median object sizes (AOM) but one of the largest average object sizes (AOA). It has the lowest IPC (UIP) and very high DTLB, last level cache and data cache miss rates (UDT, ULL, UDC) and is among the most back end bound (USB, UBM). Its large configuration has a 2.5 GB minimum heap size.

Table 11. Complete nominal statistics for h2o. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	9	142	2	28	58	211	nominal average object size (bytes)
AOL	9	152	3	24	56	200	nominal 90-percentile object size (bytes)
AOM	3	24	14	24	32	48	nominal median object size (bytes)
AOS	2	16	16	16	24	24	nominal 10-percentile object size (bytes)
ARA	7	5740	7	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	7	231	6	0	34	2204	nominal aaload per usec
BAS	9	31	3	0	1	126	nominal aastore per usec
BEF	6	6	8	1	4	29	nominal execution focus / dominance of hot code
BGF	7	3002	7	0	527	32087	nominal getfield per usec
BPF	6	142	8	0	83	3863	nominal putfield per usec
BUB	6	87	8	1	34	177	nominal thousands of unique bytecodes executed
BUF	6	11	8	0	4	29	nominal thousands of unique function calls
GCA	8	112	6	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	8	5118	5	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	8	111	5	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	8	12	6	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	9	17	4	0	0	120	nominal percent 10th iteration memory leakage
GMD	5	72	12	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	9	2543	3	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	7	29	8	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	5	73	12	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	5	249	12	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	7	187	6	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	5	207	11	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	5	57	13	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	7	3	8	1	3	8	nominal execution time (sec)
PFS	4	9	15	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	5	57	13	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	5	4	11	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	5	11	11	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	9	21	3	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	3	4	16	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	9	2	4	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	6	4	9	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	7	102	8	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	6	32	9	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	10	41	1	5	23	41	nominal backend bound (memory)
UBP	3	29	17	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	6	1126	10	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	2	30	18	5	39	137	nominal 1000 x bad speculation
UDC	9	23	3	2	12	27	nominal data cache misses per K instructions
UDT	9	499	3	14	174	576	nominal DTLB misses per M instructions
UIP	0	89	22	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	10	8506	1	335	2645	8506	nominal LLC misses per M instructions
USB	10	53	1	7	29	53	nominal 100 x back end bound
USC	7	102	7	1	52	351	nominal 1000 x SMT contention
USF	4	18	15	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 25. Lower bounds on the overheads [11] for h2o for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

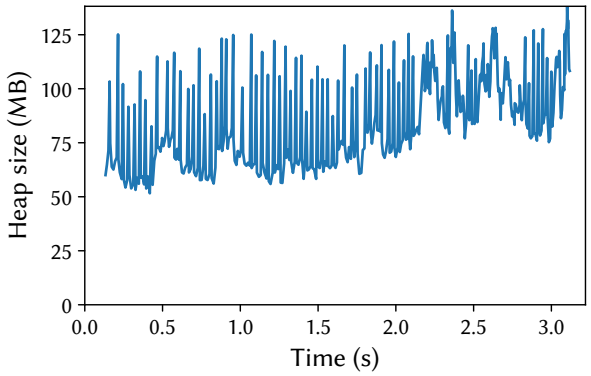


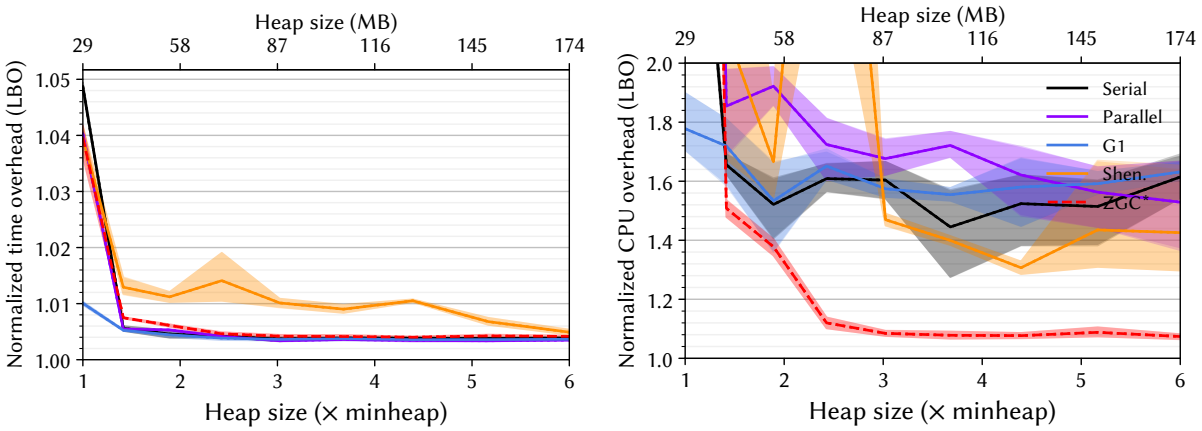
Figure 26. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.

B.10 Jme

(**New**) This workload is latency-sensitive, using jMonkey Engine, a 3-D game development suite, to render a series of video frames. jme has about 200 K lines of Java source code. It is one of the least GC-intensive workloads (GCA, GCC, GCM, GCP, GSS, GTO). It is insensitive to frequency scaling (PFS), compiler or interpreter choice (PCC, PCS, PIN), and warms up quickly (PWU). These factors are consistent with jme making extensive use of the GPU. It has the lowest SMT contention (USC) and is one of the most backend bound due to the CPU.

Table 12. Complete nominal statistics for jme. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	4	42	13	28	58	211	nominal average object size (bytes)
AOL	5	56	11	24	56	200	nominal 90-percentile object size (bytes)
AOM	3	24	14	24	32	48	nominal median object size (bytes)
AOS	10	24	1	16	24	24	nominal 10-percentile object size (bytes)
ARA	0	54	20	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	1	0	18	0	34	2204	nominal aaload per usec
BAS	4	0	13	0	1	126	nominal aastore per usec
BEF	5	4	11	1	4	29	nominal execution focus / dominance of hot code
BGF	1	26	19	0	527	32087	nominal getfield per usec
BPF	1	10	18	0	83	3863	nominal putfield per usec
BUB	5	34	11	1	34	177	nominal thousands of unique bytecodes executed
BUF	5	4	10	0	4	29	nominal thousands of unique function calls
GCA	1	24	21	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	0	31	22	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	1	24	21	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	1	0	21	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	6	0	9	0	0	120	nominal percent 10th iteration memory leakage
GMD	4	29	14	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	2	29	17	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	7	29	8	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	3	29	17	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	1	0	21	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	1	12	18	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	1	72	20	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	0	1	22	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	10	7	2	1	3	8	nominal execution time (sec)
PFS	1	0	21	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	0	1	22	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	8	8	6	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	2	0	19	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	2	0	19	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	2	3	19	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	3	0	16	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	1	1	20	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	0	2	22	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	1	1	20	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	2	19	18	5	23	41	nominal backend bound (memory)
UBP	9	89	4	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	7	1226	8	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	9	90	4	5	39	137	nominal 1000 x bad speculation
UDC	5	11	13	2	12	27	nominal data cache misses per K instructions
UDT	4	96	15	14	174	576	nominal DTLB misses per M instructions
UIP	7	204	7	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	2	1558	18	335	2645	8506	nominal LLC misses per M instructions
USB	4	27	14	7	29	53	nominal 100 x back end bound
USC	0	1	22	1	52	351	nominal 1000 x SMT contention
USF	7	32	8	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 27. Lower bounds on the overheads [11] for jme for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

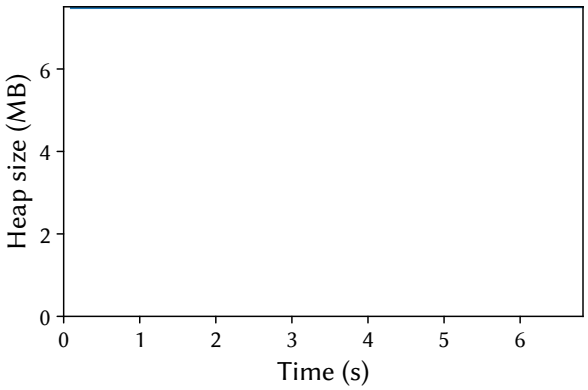
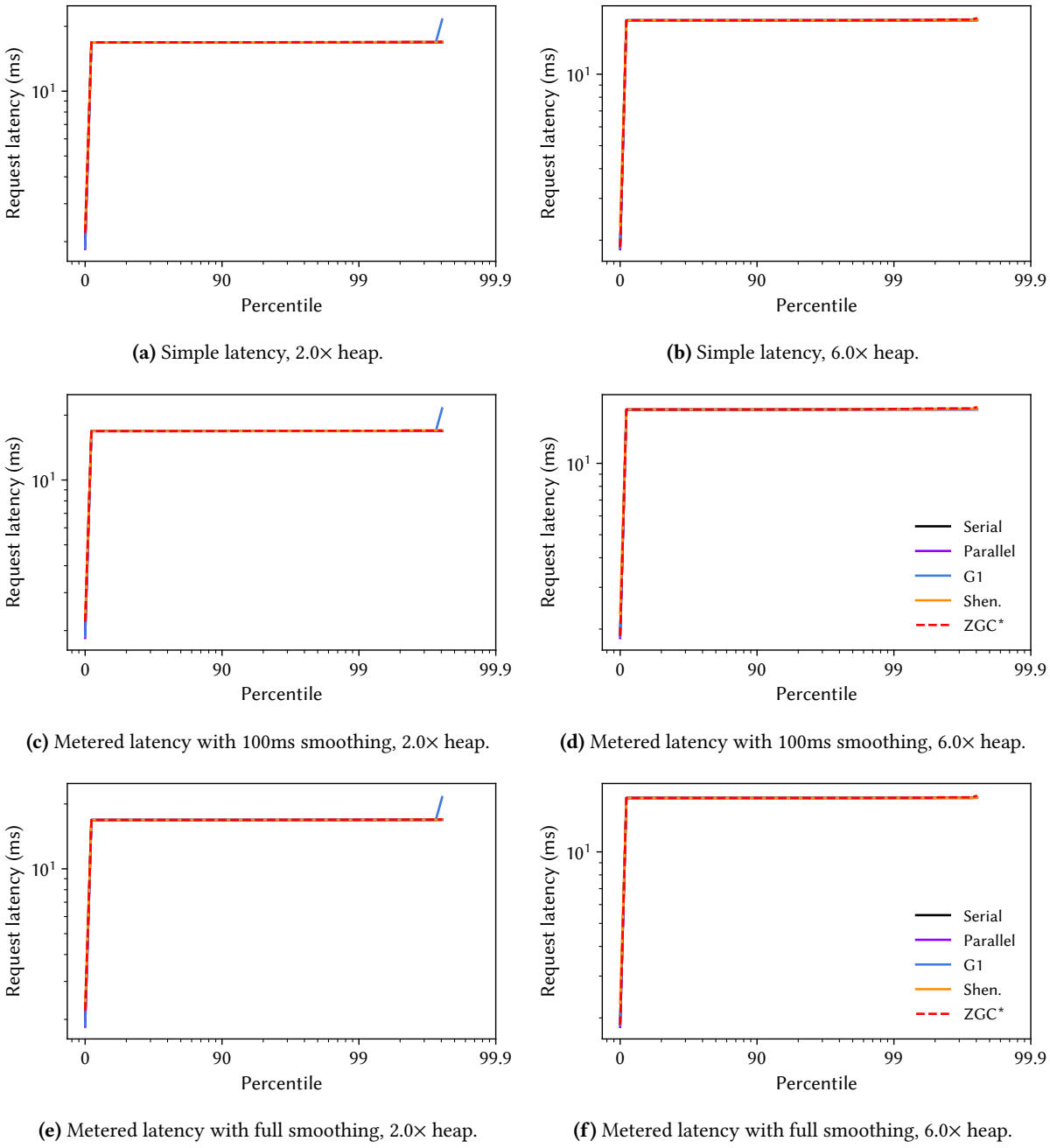


Figure 28. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.



(a) Simple latency, 2.0× heap.

(b) Simple latency, 6.0× heap.

(c) Metered latency with 100ms smoothing, 2.0× heap.

(d) Metered latency with 100ms smoothing, 6.0× heap.

(e) Metered latency with full smoothing, 2.0× heap.

(f) Metered latency with full smoothing, 6.0× heap.

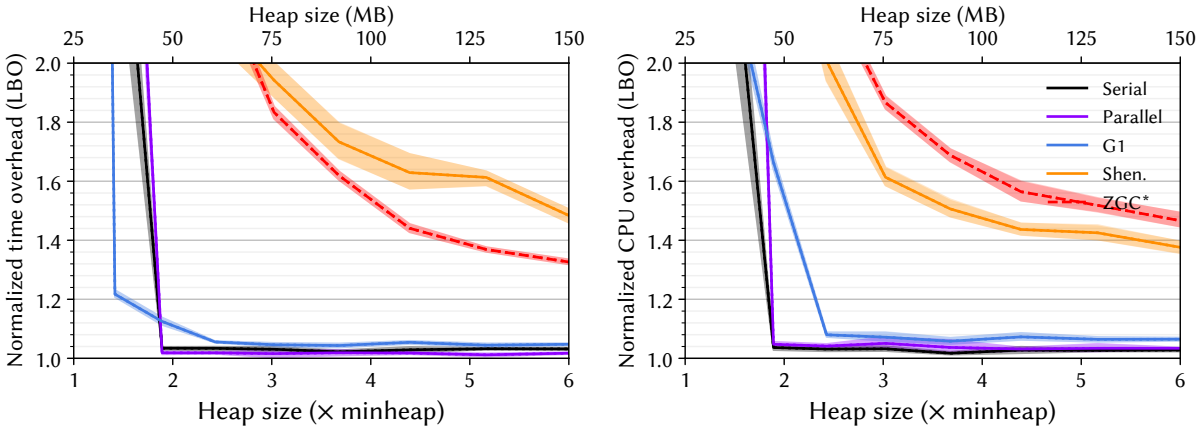
Figure 29. Distribution of request latencies for jme for each of OpenJDK 21's six production collectors. The figures in the left top row simply plot the request latencies, while the figures in the middle and the bottom rows use DaCapo's metered latency, which models a request queue and the cascading effect of delays.

B.11 Jython

The jython workload executes a standard Python performance test on top of Jython, a Java implementation of the Python programming language. Jython has about 310 K lines of Java code. It has the most unique function calls executed (BUF) and a large number of unique bytecodes executed (BUB). Consistent with this, it has the longest warmup (PWU) and is sensitive to compiler configuration (PCS, PIN). It is the most sensitive to frequency scaling (PFS), has very high IPC (UIP) and high bad speculation due to mispredicts (UBS, UBP).

Table 13. Complete nominal statistics for jython. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	2	37	17	28	58	211	nominal average object size (bytes)
AOL	3	48	15	24	56	200	nominal 90-percentile object size (bytes)
AOM	9	32	2	24	32	48	nominal median object size (bytes)
AOS	2	16	16	16	24	24	nominal 10-percentile object size (bytes)
ARA	4	1462	13	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	5	39	10	0	34	2204	nominal aaload per usec
BAS	8	13	5	0	1	126	nominal aastore per usec
BEF	7	8	6	1	4	29	nominal execution focus / dominance of hot code
BGF	3	256	15	0	527	32087	nominal getfield per usec
BPF	5	83	11	0	83	3863	nominal putfield per usec
BUB	8	149	4	1	34	177	nominal thousands of unique bytecodes executed
BUF	10	29	1	0	4	29	nominal thousands of unique function calls
GCA	6	104	9	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	8	3457	6	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	6	100	9	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	6	7	10	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	6	0	9	0	0	120	nominal percent 10th iteration memory leakage
GMD	3	25	17	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	1	25	18	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	6	25	10	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	4	31	14	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	8	2024	5	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	7	139	7	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	6	211	10	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	10	277	2	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	7	3	8	1	3	8	nominal execution time (sec)
PFS	10	20	1	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	10	277	2	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	3	1	16	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	3	1	17	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	2	0	19	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	5	5	13	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	7	1	7	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	10	9	1	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	7	102	8	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	6	32	9	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	1	17	20	5	23	41	nominal backend bound (memory)
UBP	8	85	5	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	5	1105	11	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	8	86	5	5	39	137	nominal 1000 x bad speculation
UDC	4	9	15	2	12	27	nominal data cache misses per K instructions
UDT	3	78	16	14	174	576	nominal DTLB misses per M instructions
UIP	10	268	2	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	1	1160	20	335	2645	8506	nominal LLC misses per M instructions
USB	1	20	20	7	29	53	nominal 100 x back end bound
USC	4	35	15	1	52	351	nominal 1000 x SMT contention
USF	5	21	13	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 30. Lower bounds on the overheads [11] for jython for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

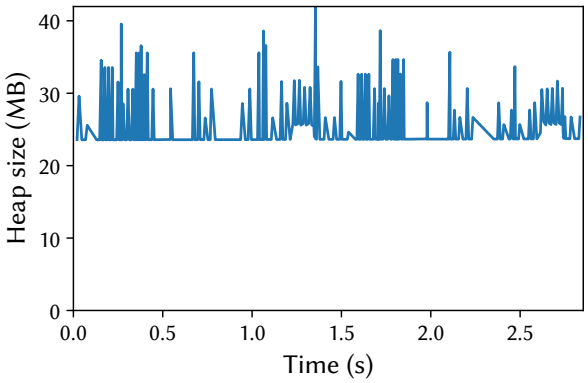


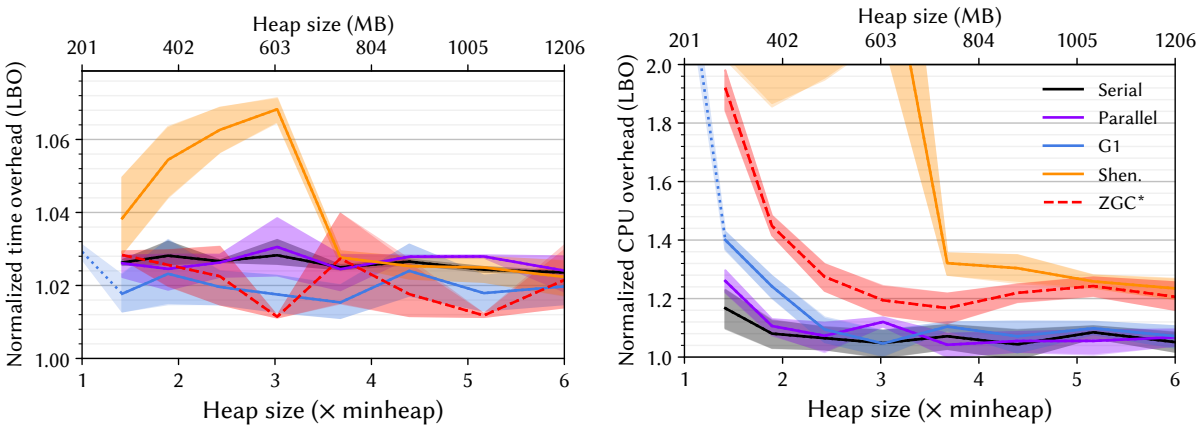
Figure 31. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.

B.12 Kafka

(**New**) This is a latency-sensitive workload that issues requests to the Apache Kafka framework for high-throughput publish-subscribe messaging. Kafka has about 840 K lines of Java and Scala source code. kafka has low garbage collection sensitivity (GSS, GCP). It is kernel-intensive (PKP) and insensitive to CPU frequency scaling (PFS) and memory speed (PMS). It has a very high data cache and last level cache miss rates (UDC, ULL) and is one of the most front end bound workloads (USF).

Table 14. Complete nominal statistics for kafka. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	4	54	12	28	58	211	nominal average object size (bytes)
AOL	5	56	11	24	56	200	nominal 90-percentile object size (bytes)
AOM	9	32	2	24	32	48	nominal median object size (bytes)
AOS	2	16	16	16	24	24	nominal 10-percentile object size (bytes)
ARA	2	803	17	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	2	1	17	0	34	2204	nominal aaload per usec
BAS	4	0	13	0	1	126	nominal aastore per usec
BEF	1	1	19	1	4	29	nominal execution focus / dominance of hot code
BGF	2	183	16	0	527	32087	nominal getfield per usec
BPF	3	55	14	0	83	3863	nominal putfield per usec
BUB	9	159	3	1	34	177	nominal thousands of unique bytecodes executed
BUF	9	28	2	0	4	29	nominal thousands of unique function calls
GCA	2	86	18	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	2	221	19	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	4	86	14	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	1	0	21	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	6	0	9	0	0	120	nominal percent 10th iteration memory leakage
GMD	10	201	2	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	6	345	8	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	10	157	1	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	9	208	4	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	1	0	21	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	2	19	17	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	7	255	8	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	4	34	15	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	9	6	3	1	3	8	nominal execution time (sec)
PFS	1	1	20	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	4	34	15	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	10	25	2	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	2	0	19	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	2	0	19	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	2	3	19	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	7	1	7	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	5	3	11	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	1	19	20	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	3	13	17	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	6	26	9	5	23	41	nominal backend bound (memory)
UBP	3	30	16	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	1	547	20	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	3	31	17	5	39	137	nominal 1000 x bad speculation
UDC	10	27	1	2	12	27	nominal data cache misses per K instructions
UDT	6	230	10	14	174	576	nominal DTLB misses per M instructions
UIP	4	127	14	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	10	6819	2	335	2645	8506	nominal LLC misses per M instructions
USB	6	30	10	7	29	53	nominal 100 x back end bound
USC	3	20	17	1	52	351	nominal 1000 x SMT contention
USF	9	43	3	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 32. Lower bounds on the overheads [11] for kafka for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

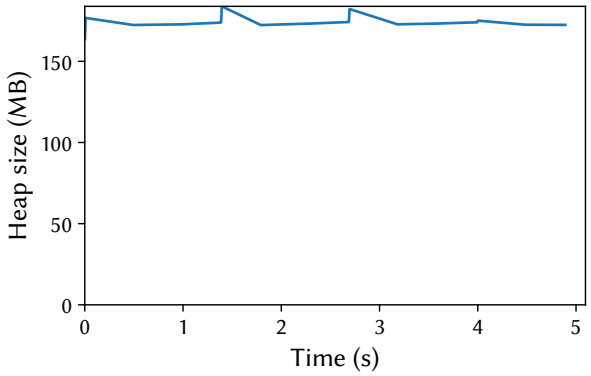
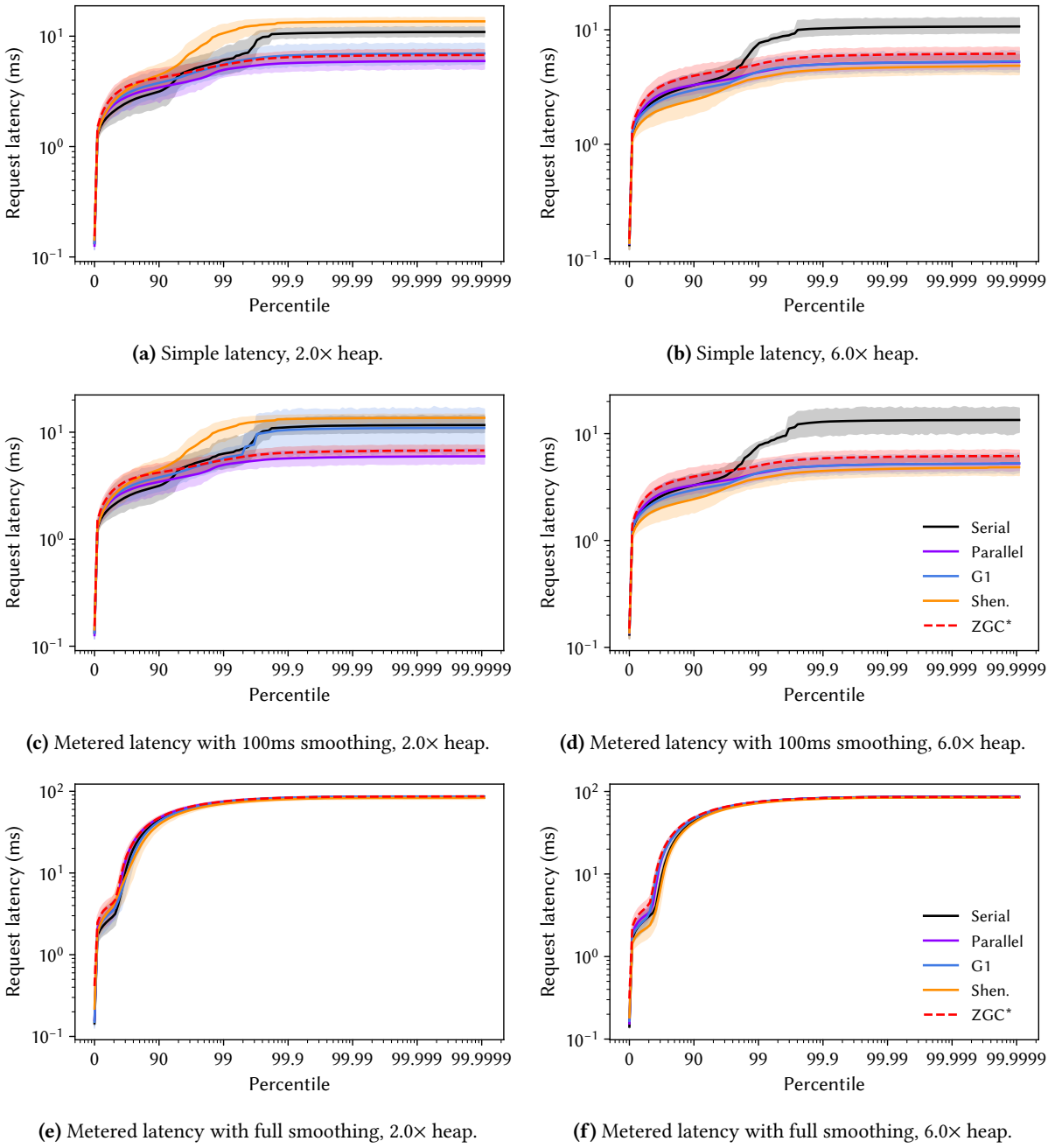


Figure 33. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.



(a) Simple latency, 2.0× heap.

(b) Simple latency, 6.0× heap.

(c) Metered latency with 100ms smoothing, 2.0× heap.

(d) Metered latency with 100ms smoothing, 6.0× heap.

(e) Metered latency with full smoothing, 2.0× heap.

(f) Metered latency with full smoothing, 6.0× heap.

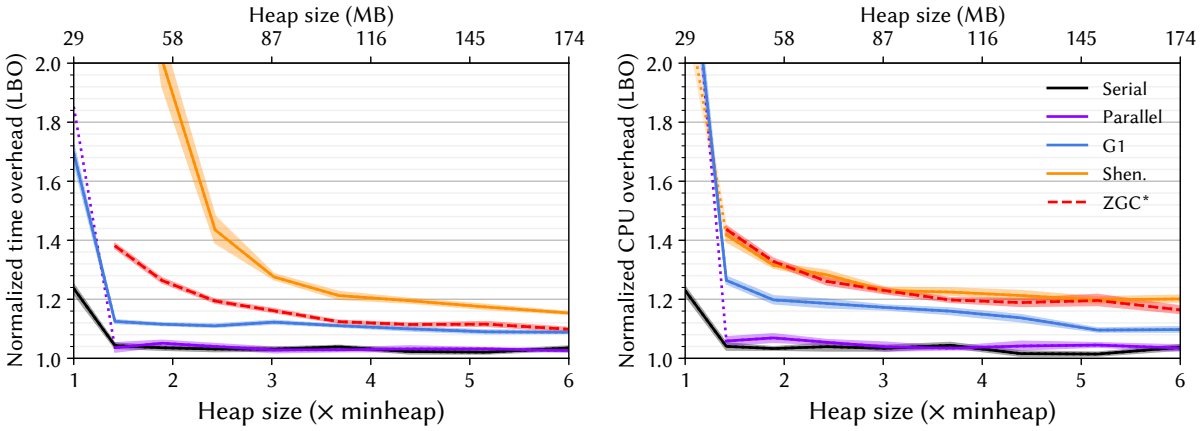
Figure 34. Distribution of request latencies for kafka for each of OpenJDK 21's six production collectors. The figures in the left top row simply plot the request latencies, while the figures in the middle and the bottom rows use DaCapo's metered latency, which models a request queue and the cascading effect of delays.

B.13 Luindex

This workload constructs a search index from a document corpus using the Apache Lucene search engine. Lucene has about 830 K lines of Java source code. luindex has the largest objects in the suite (AOA, AOL, AOM, AOS). It is one of the most sensitive to CPU frequency scaling (PFS) and last level cache size (PLS). It has one of the highest IPCs (UIP) but suffers one of the worst bad speculation rates (UBS, UBR, UBP), but has low cache miss rates (UDC, UDT, ULL).

Table 15. Complete nominal statistics for luindex. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	10	211	1	28	58	211	nominal average object size (bytes)
AOL	8	88	4	24	56	200	nominal 90-percentile object size (bytes)
AOM	9	32	2	24	32	48	nominal median object size (bytes)
AOS	10	24	1	16	24	24	nominal 10-percentile object size (bytes)
ARA	2	841	16	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	4	33	12	0	34	2204	nominal aaload per usec
BAS	6	1	9	0	1	126	nominal aastore per usec
BEF	3	3	15	1	4	29	nominal execution focus / dominance of hot code
BGF	6	1179	9	0	527	32087	nominal getfield per usec
BPF	7	306	6	0	83	3863	nominal putfield per usec
BUB	5	54	10	1	34	177	nominal thousands of unique bytecodes executed
BUF	6	5	9	0	4	29	nominal thousands of unique function calls
GCA	4	93	15	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	6	1459	9	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	6	100	9	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	4	1	15	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	6	0	9	0	0	120	nominal percent 10th iteration memory leakage
GMD	4	29	14	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	3	37	15	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	5	13	12	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	4	31	14	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	4	56	14	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	6	76	9	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	5	201	12	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	5	61	12	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	7	3	8	1	3	8	nominal execution time (sec)
PFS	9	18	4	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	5	61	12	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	5	2	12	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	10	38	2	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	4	2	15	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	2	3	19	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	7	1	7	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	5	2	13	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	5	90	13	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	5	25	12	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	8	31	6	5	23	41	nominal backend bound (memory)
UBP	10	109	2	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	10	3280	2	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	10	112	2	5	39	137	nominal 1000 x bad speculation
UDC	2	6	18	2	12	27	nominal data cache misses per K instructions
UDT	2	66	18	14	174	576	nominal DTLB misses per M instructions
UIP	9	263	3	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	1	930	21	335	2645	8506	nominal LLC misses per M instructions
USB	7	36	7	7	29	53	nominal 100 x back end bound
USC	1	4	21	1	52	351	nominal 1000 x SMT contention
USF	2	12	18	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 35. Lower bounds on the overheads [11] for luindex for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

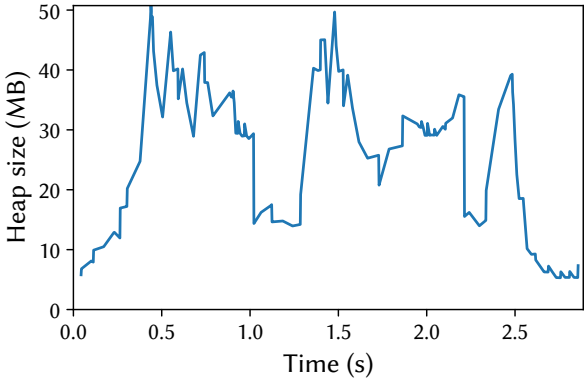


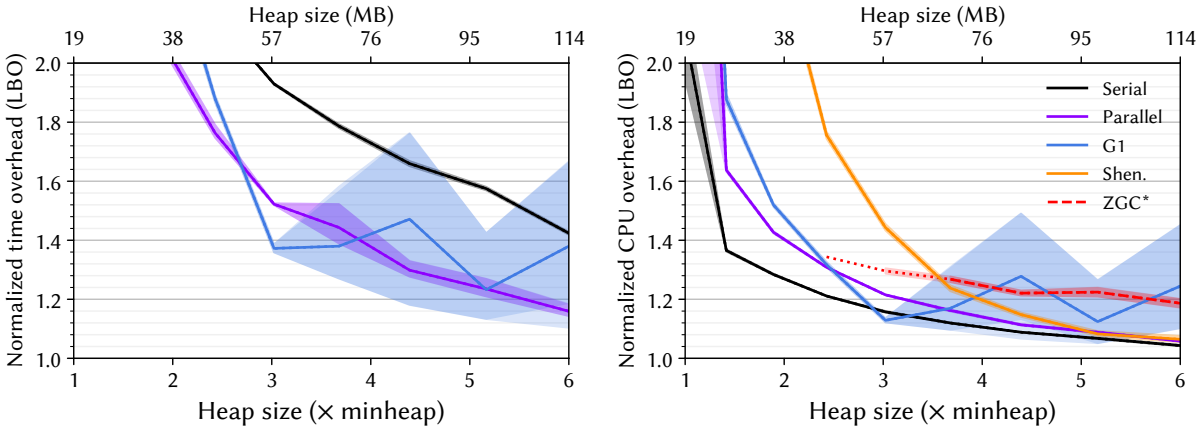
Figure 36. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.

B.14 Lusearch

This is a latency-sensitive workload that issues search requests to the Apache Lucene search engine. lusearch has the highest memory turn over (GTO), performs the most GCs (GCC), has the highest allocation rate (ARA), has the highest aastore and putfield rates (BAS, BPF), and one of the highest getfield rates (BGF). It uses a very small heap (GCA, GMD).

Table 16. Complete nominal statistics for lusearch. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	7	75	7	28	58	211	nominal average object size (bytes)
AOL	8	88	4	24	56	200	nominal 90-percentile object size (bytes)
AOM	3	24	14	24	32	48	nominal median object size (bytes)
AOS	10	24	1	16	24	24	nominal 10-percentile object size (bytes)
ARA	10	23556	1	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	8	252	4	0	34	2204	nominal aaload per usec
BAS	10	126	1	0	1	126	nominal aastore per usec
BEF	6	5	9	1	4	29	nominal execution focus / dominance of hot code
BGF	9	12289	2	0	527	32087	nominal getfield per usec
BPF	10	3863	1	0	83	3863	nominal putfield per usec
BUB	3	26	14	1	34	177	nominal thousands of unique bytecodes executed
BUF	3	3	14	0	4	29	nominal thousands of unique function calls
GCA	3	89	16	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	10	22408	1	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	4	84	15	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	10	32	2	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	6	0	9	0	0	120	nominal percent 10th iteration memory leakage
GMD	2	19	19	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	4	109	13	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	2	5	18	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	2	21	19	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	9	2159	4	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	10	1211	1	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	4	172	14	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	9	202	4	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	5	2	13	1	3	8	nominal execution time (sec)
PFS	5	11	13	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	9	202	4	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	7	7	7	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	7	19	8	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	7	9	8	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	9	34	4	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	9	3	3	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	10	8	2	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	4	87	14	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	10	56	1	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	3	20	17	5	23	41	nominal backend bound (memory)
UBP	5	40	11	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	2	596	18	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	5	41	11	5	39	137	nominal 1000 x bad speculation
UDC	5	12	12	2	12	27	nominal data cache misses per K instructions
UDT	5	154	13	14	174	576	nominal DTLB misses per M instructions
UIP	5	149	12	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	5	2830	11	335	2645	8506	nominal LLC misses per M instructions
USB	5	29	11	7	29	53	nominal 100 x back end bound
USC	9	198	3	1	52	351	nominal 1000 x SMT contention
USF	5	23	12	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 37. Lower bounds on the overheads [11] for lusearch for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

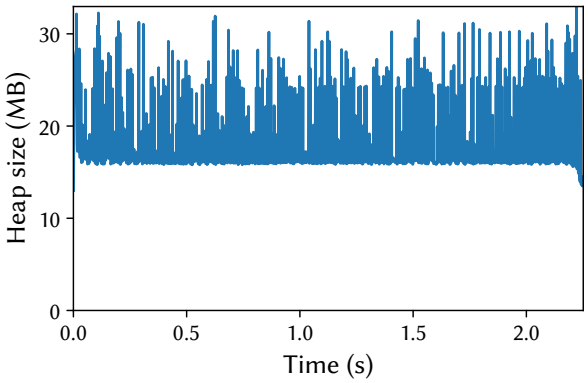
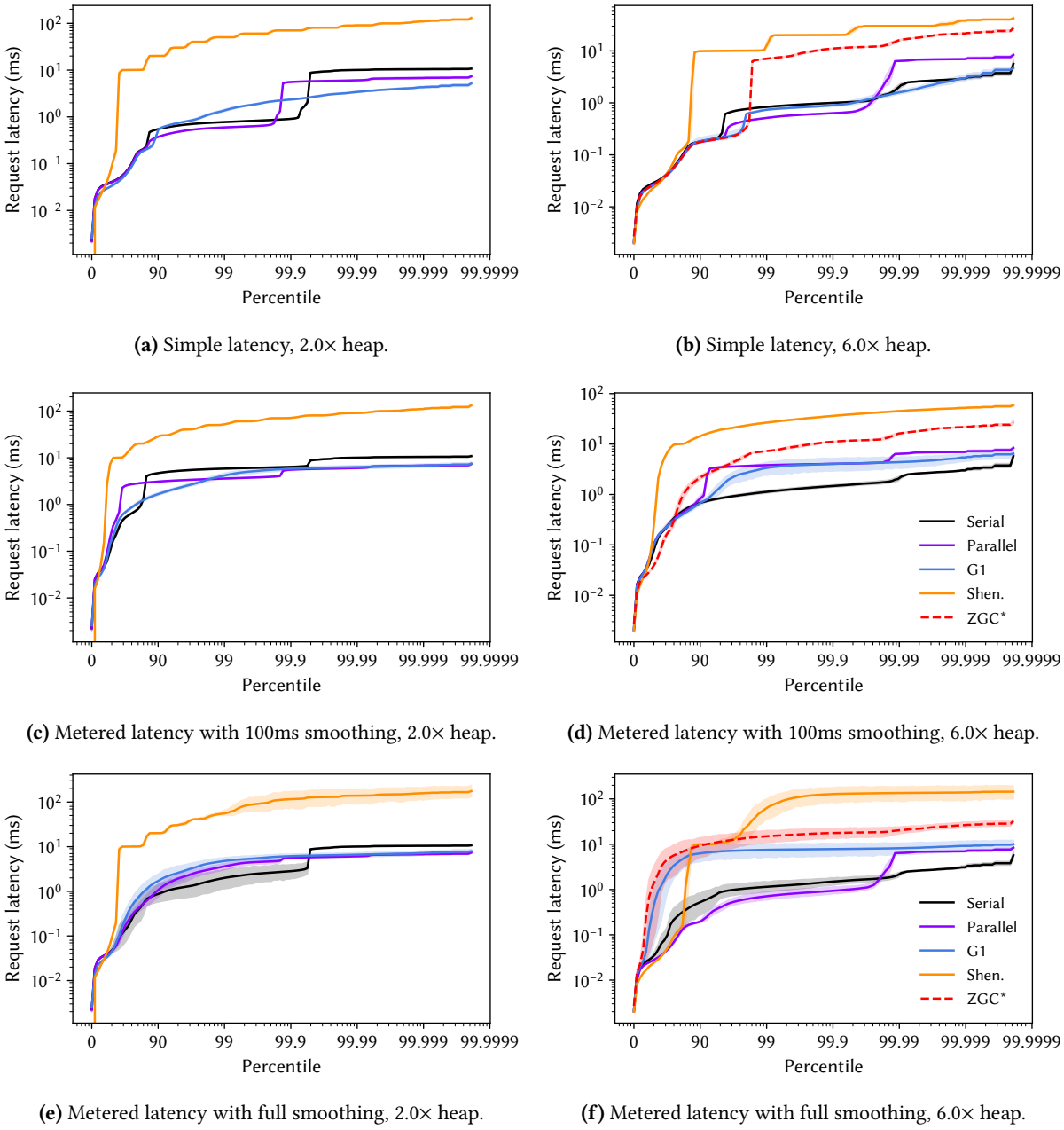


Figure 38. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.



(a) Simple latency, 2.0× heap.

(b) Simple latency, 6.0× heap.

(c) Metered latency with 100ms smoothing, 2.0× heap.

(d) Metered latency with 100ms smoothing, 6.0× heap.

(e) Metered latency with full smoothing, 2.0× heap.

(f) Metered latency with full smoothing, 6.0× heap.

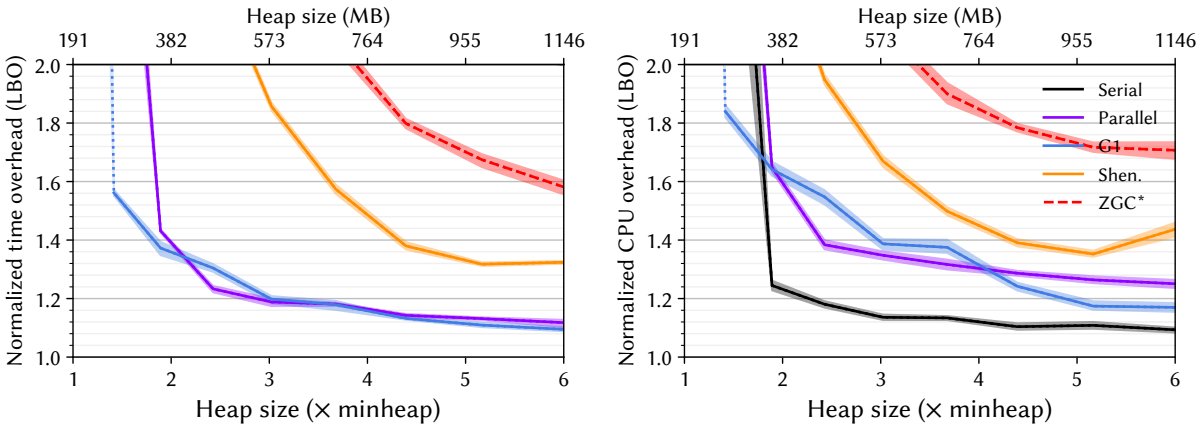
Figure 39. Distribution of request latencies for lusearch for each of OpenJDK 21's six production collectors. The figures in the left top row simply plot the request latencies, while the figures in the middle and the bottom rows use DaCapo's metered latency, which models a request queue and the cascading effect of delays.

B.15 Pmd

This workload uses the PMD static code analyzer to check a corpus of source code. PMD has about 120 K lines of Java code. pmd is one of the most last level cache size-sensitive workloads (PLS) and is sensitive to memory speed (PMS). It is one of the least generational workloads (GCM), and is one of the slowest to warm up (PWU). It is among the most back end bound (USB, UBM), with high SMT contention (USC) and high last level cache miss rate (ULL).

Table 17. Complete nominal statistics for pmd. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	1	32	19	28	58	211	nominal average object size (bytes)
AOL	3	48	15	24	56	200	nominal 90-percentile object size (bytes)
AOM	3	24	14	24	32	48	nominal median object size (bytes)
AOS	2	16	16	16	24	24	nominal 10-percentile object size (bytes)
ARA	8	6721	5	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	6	82	8	0	34	2204	nominal aaload per usec
BAS	6	1	9	0	1	126	nominal aastore per usec
BEF	5	4	11	1	4	29	nominal execution focus / dominance of hot code
BGF	6	1719	8	0	527	32087	nominal getfield per usec
BPF	8	583	5	0	83	3863	nominal putfield per usec
BUB	7	95	7	1	34	177	nominal thousands of unique bytecodes executed
BUF	7	15	7	0	4	29	nominal thousands of unique function calls
GCA	10	133	1	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	4	781	14	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	10	144	1	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	8	16	5	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	7	5	7	0	0	120	nominal percent 10th iteration memory leakage
GMD	9	191	3	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	9	3519	2	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	3	7	16	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	10	269	2	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	7	467	8	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	3	32	15	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	5	179	13	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	6	74	10	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	3	1	17	1	3	8	nominal execution time (sec)
PFS	5	11	13	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	6	74	10	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	3	1	16	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	9	31	4	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	8	19	5	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	7	10	8	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	7	1	7	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	9	7	4	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	8	112	6	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	10	47	2	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	8	35	5	5	23	41	nominal backend bound (memory)
UBP	5	38	13	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	7	1295	7	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	5	39	12	5	39	137	nominal 1000 x bad speculation
UDC	7	16	8	2	12	27	nominal data cache misses per K instructions
UDT	6	258	9	14	174	576	nominal DTLB misses per M instructions
UIP	2	109	18	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	8	4478	6	335	2645	8506	nominal LLC misses per M instructions
USB	8	40	5	7	29	53	nominal 100 x back end bound
USC	8	155	5	1	52	351	nominal 1000 x SMT contention
USF	5	21	13	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 40. Lower bounds on the overheads [11] for pmd for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

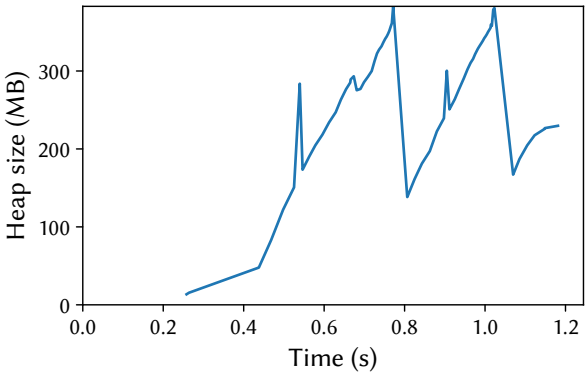


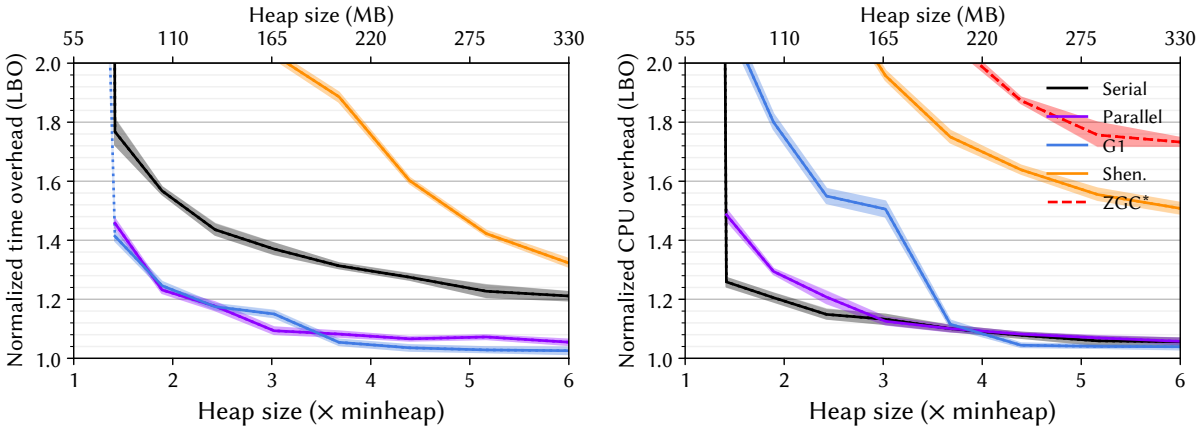
Figure 41. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.

B.16 Spring

(**New**) This is a latency-sensitive workload that runs the petclinic workload over the Spring Boot microservices web framework. DaCapo replaces petclinc's synthetic load generator with a deterministic request workload. Spring Boot has about 580 K lines of Java source code. spring is sensitive to memory speed (PMS). It has one of the highest number of unique bytecodess executed (BUB) and unique function calls (BUF) and is sensitive to choice of compiler (PCS).

Table 18. Complete nominal statistics for spring. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	6	70	9	28	58	211	nominal average object size (bytes)
AOL	10	200	1	24	56	200	nominal 90-percentile object size (bytes)
AOM	9	32	2	24	32	48	nominal median object size (bytes)
AOS	10	24	1	16	24	24	nominal 10-percentile object size (bytes)
ARA	9	10849	3	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	3	11	14	0	34	2204	nominal aaload per usec
BAS	7	2	7	0	1	126	nominal aastore per usec
BEF	1	2	18	1	4	29	nominal execution focus / dominance of hot code
BGF	4	395	12	0	527	32087	nominal getfield per usec
BPF	5	94	10	0	83	3863	nominal putfield per usec
BUB	9	170	2	1	34	177	nominal thousands of unique bytecodes executed
BUF	9	26	3	0	4	29	nominal thousands of unique function calls
GCA	4	94	14	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	7	2770	7	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	3	83	17	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	8	12	6	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	6	0	9	0	0	120	nominal percent 10th iteration memory leakage
GMD	5	55	13	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	3	65	14	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	7	43	7	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	5	70	13	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	6	397	9	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	8	283	5	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	4	162	15	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	7	110	8	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	5	2	13	1	3	8	nominal execution time (sec)
PFS	3	8	16	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	7	110	8	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	7	7	7	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	5	6	13	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	9	20	4	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	9	36	3	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	7	1	7	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	5	2	13	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	4	87	14	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	5	30	11	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	7	28	8	5	23	41	nominal backend bound (memory)
UBP	7	60	7	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	8	1475	6	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	7	61	7	5	39	137	nominal 1000 x bad speculation
UDC	5	13	11	2	12	27	nominal data cache misses per K instructions
UDT	8	392	6	14	174	576	nominal DTLB misses per M instructions
UIP	4	122	15	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	7	4264	8	335	2645	8506	nominal LLC misses per M instructions
USB	6	32	9	7	29	53	nominal 100 x back end bound
USC	7	100	8	1	52	351	nominal 1000 x SMT contention
USF	7	32	8	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 42. Lower bounds on the overheads [11] for spring for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

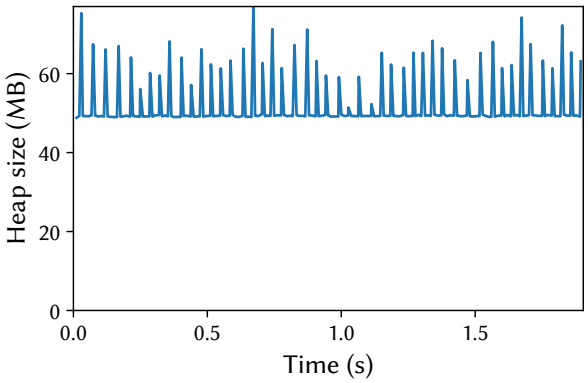
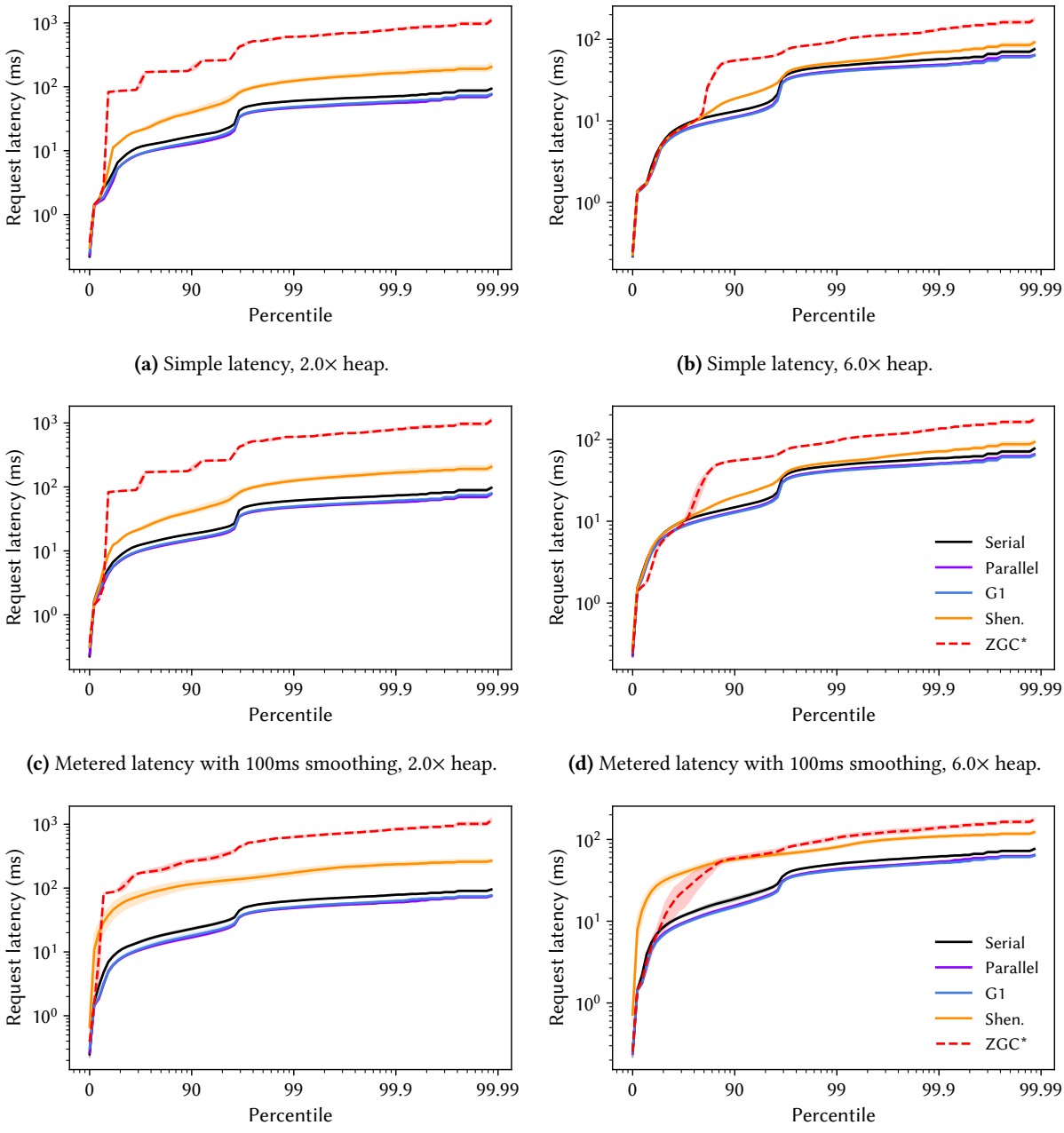


Figure 43. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.



(a) Simple latency, 2.0× heap.

(b) Simple latency, 6.0× heap.

(c) Metered latency with 100ms smoothing, 2.0× heap.

(d) Metered latency with 100ms smoothing, 6.0× heap.

(e) Metered latency with full smoothing, 2.0× heap.

(f) Metered latency with full smoothing, 6.0× heap.

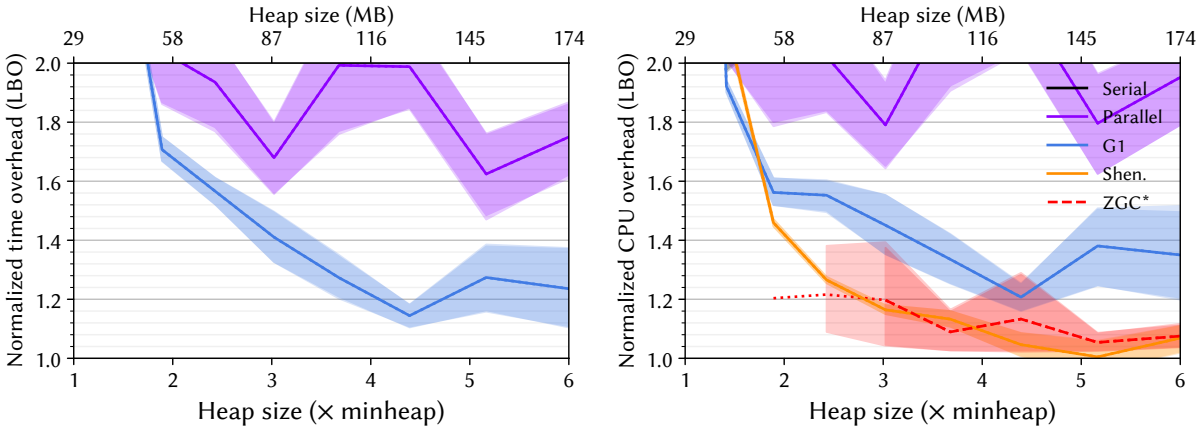
Figure 44. Distribution of request latencies for spring for each of OpenJDK 21's six production collectors. The figures in the left top row simply plot the request latencies, while the figures in the middle and the bottom rows use DaCapo's metered latency, which models a request queue and the cascading effect of delays.

B.17 Sunflow

This workload uses the Sunflow photorealistic renderer to render a series of images. Sunflow consists of about 25 K lines of Java code. sunflow has a high allocation rate (ARA), and the highest aaload and getfield rates (BAL, BGF). It is the slow to warm up (PWU) and has the highest execution variance (PSD). It is the least sensitive to last level cache size (PLS). It is one of the least front end bound (USF) and one of the most back end bound (USB) and suffers high SMT contention (USC).

Table 19. Complete nominal statistics for sunflow. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	3	40	15	28	58	211	nominal average object size (bytes)
AOL	3	48	15	24	56	200	nominal 90-percentile object size (bytes)
AOM	10	48	1	24	32	48	nominal median object size (bytes)
AOS	10	24	1	16	24	24	nominal 10-percentile object size (bytes)
ARA	8	10518	4	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	10	2204	1	0	34	2204	nominal aaload per usec
BAS	7	2	7	0	1	126	nominal aastore per usec
BEF	3	3	15	1	4	29	nominal execution focus / dominance of hot code
BGF	10	32087	1	0	527	32087	nominal getfield per usec
BPF	9	3200	2	0	83	3863	nominal putfield per usec
BUB	2	20	16	1	34	177	nominal thousands of unique bytecodes executed
BUF	1	1	18	0	4	29	nominal thousands of unique function calls
GCA	9	113	4	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	10	14139	2	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	9	113	4	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	9	20	4	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	6	0	9	0	0	120	nominal percent 10th iteration memory leakage
GMD	4	29	14	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	5	149	11	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	2	5	18	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	4	31	14	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	9	6329	3	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	9	711	3	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	3	92	17	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	2	14	19	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	7	3	8	1	3	8	nominal execution time (sec)
PFS	7	16	8	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	2	14	19	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	3	1	16	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	0	-2	22	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	4	3	14	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	8	24	5	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	10	13	1	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	8	6	6	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	5	98	11	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	4	19	15	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	9	37	3	5	23	41	nominal backend bound (memory)
UBP	1	21	20	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	8	2380	5	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	1	24	20	5	39	137	nominal 1000 x bad speculation
UDC	3	8	16	2	12	27	nominal data cache misses per K instructions
UDT	3	75	17	14	174	576	nominal DTLB misses per M instructions
UIP	3	114	16	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	4	2333	14	335	2645	8506	nominal LLC misses per M instructions
USB	10	49	2	7	29	53	nominal 100 x back end bound
USC	10	240	2	1	52	351	nominal 1000 x SMT contention
USF	1	5	21	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 45. Lower bounds on the overheads [11] for sunflow for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

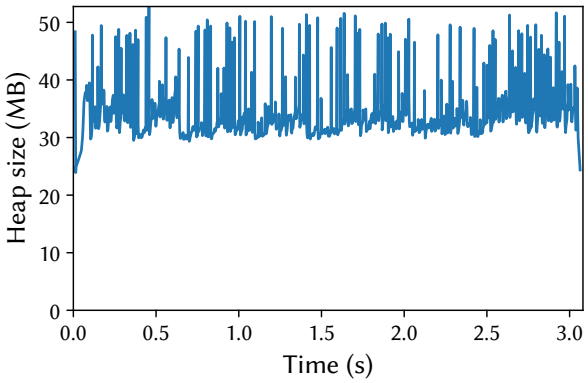


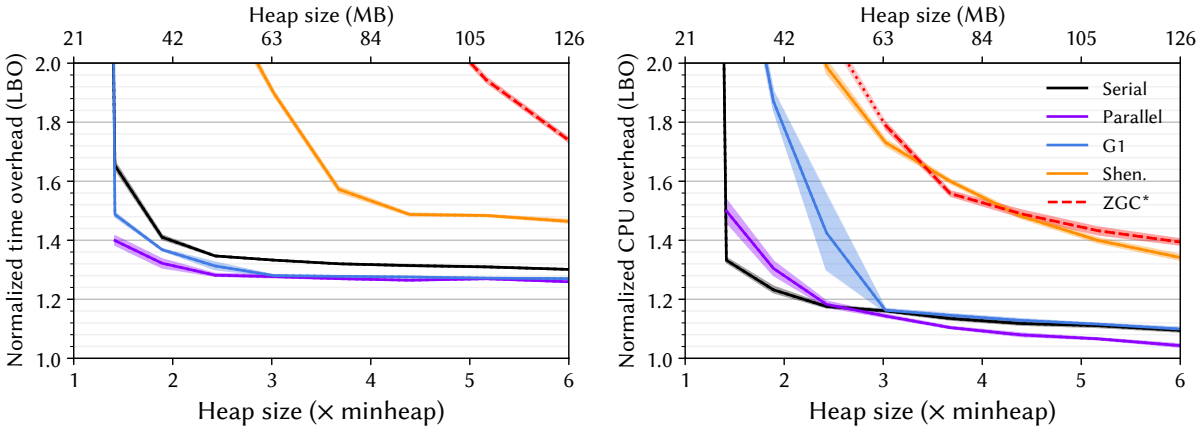
Figure 46. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.

B.18 Tomcat

This is a latency-sensitive workload that issues requests to the Apache Tomcat web server. Tomcat consists of about 380 K lines of Java code. Tomcat has the highest parallel efficiency (PPE). It is sensitive to heap size (GSS) and has a high GC turnover (GTO) and GC count (GCC). It spends a relatively large amount of time in the kernel (PKP), which is unsurprising for a web server. It is sensitive to compiler choice (PCC, PIN). It has one of the highest data cache, last level cache, and DTLB miss rates (UDC, ULL, UDT), is front end bound (USF) and has one of the lowest IPCs (UIP).

Table 20. Complete nominal statistics for tomcat. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	7	75	7	28	58	211	nominal average object size (bytes)
AOL	7	80	7	24	56	200	nominal 90-percentile object size (bytes)
AOM	9	32	2	24	32	48	nominal median object size (bytes)
AOS	10	24	1	16	24	24	nominal 10-percentile object size (bytes)
ARA	6	5290	8	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	3	9	15	0	34	2204	nominal aaload per usec
BAS	4	0	13	0	1	126	nominal aastore per usec
BEF	8	9	5	1	4	29	nominal execution focus / dominance of hot code
BGF	3	272	14	0	527	32087	nominal getfield per usec
BPF	4	75	12	0	83	3863	nominal putfield per usec
BUB	8	118	5	1	34	177	nominal thousands of unique bytecodes executed
BUF	7	16	6	0	4	29	nominal thousands of unique function calls
GCA	5	100	12	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	9	6029	4	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	5	95	13	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	6	8	9	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	6	0	9	0	0	120	nominal percent 10th iteration memory leakage
GMD	2	21	18	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	2	35	16	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	5	13	12	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	2	24	18	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	8	1227	6	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	9	810	2	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	10	465	2	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	8	149	6	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	8	4	6	1	3	8	nominal execution time (sec)
PFS	2	2	18	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	8	149	6	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	9	19	3	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	5	8	12	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	4	2	15	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	10	87	1	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	3	0	16	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	5	2	13	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	1	14	21	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	2	4	19	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	4	22	14	5	23	41	nominal backend bound (memory)
UBP	6	44	10	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	2	584	19	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	6	45	10	5	39	137	nominal 1000 x bad speculation
UDC	8	18	5	2	12	27	nominal data cache misses per K instructions
UDT	10	519	2	14	174	576	nominal DTLB misses per M instructions
UIP	1	102	20	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	9	5119	4	335	2645	8506	nominal LLC misses per M instructions
USB	4	26	15	7	29	53	nominal 100 x back end bound
USC	5	76	11	1	52	351	nominal 1000 x SMT contention
USF	10	45	2	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 47. Lower bounds on the overheads [11] for tomcat for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

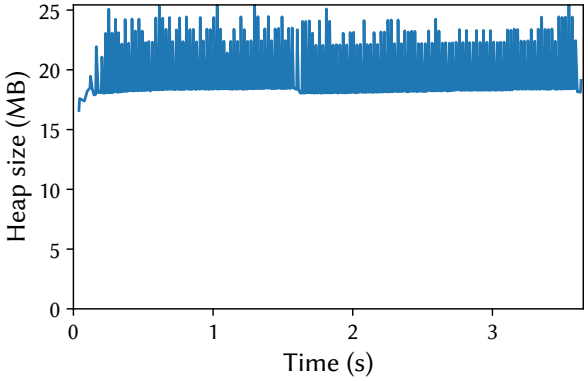
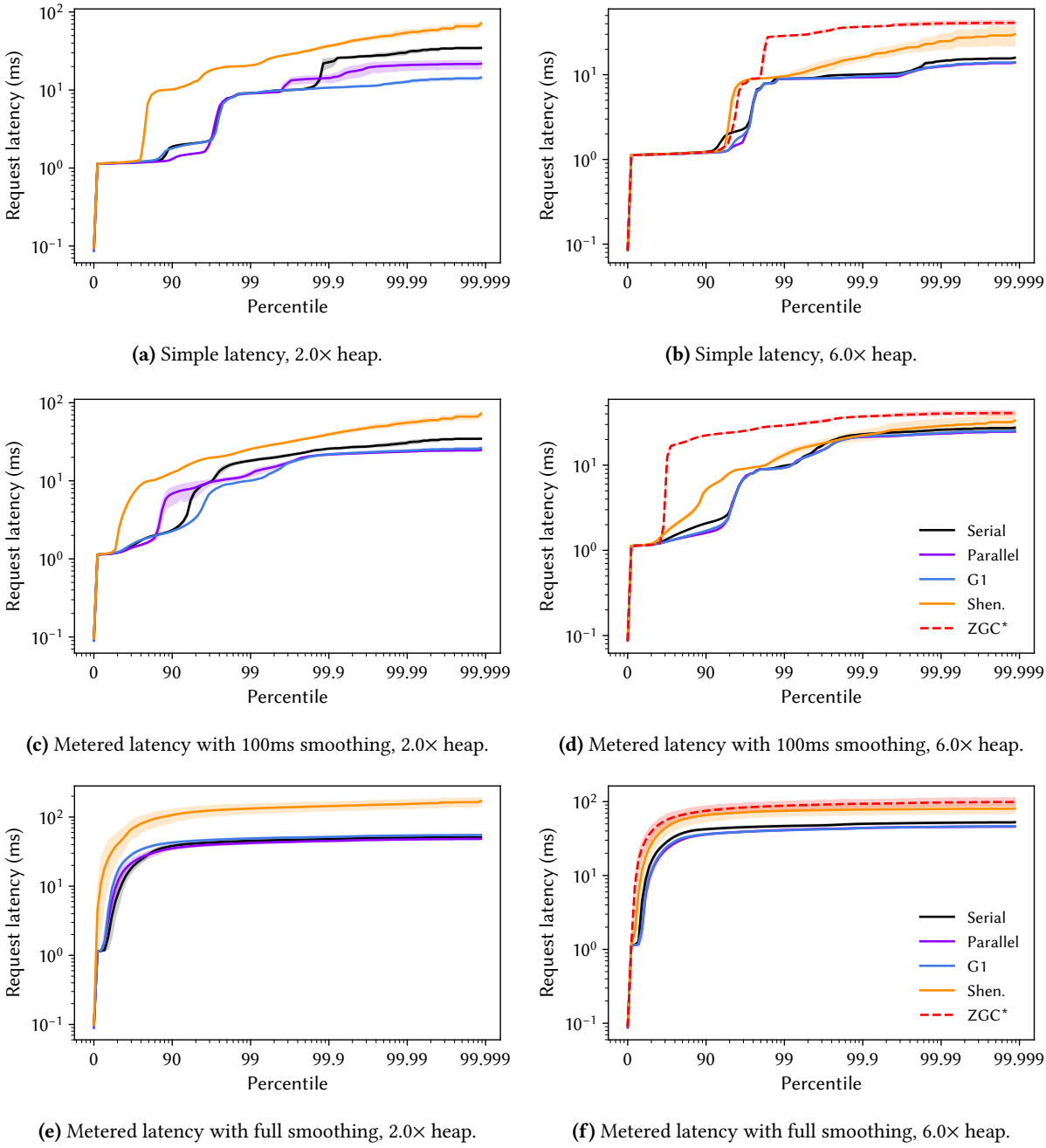


Figure 48. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.



(a) Simple latency, 2.0× heap.

(b) Simple latency, 6.0× heap.

(c) Metered latency with 100ms smoothing, 2.0× heap.

(d) Metered latency with 100ms smoothing, 6.0× heap.

(e) Metered latency with full smoothing, 2.0× heap.

(f) Metered latency with full smoothing, 6.0× heap.

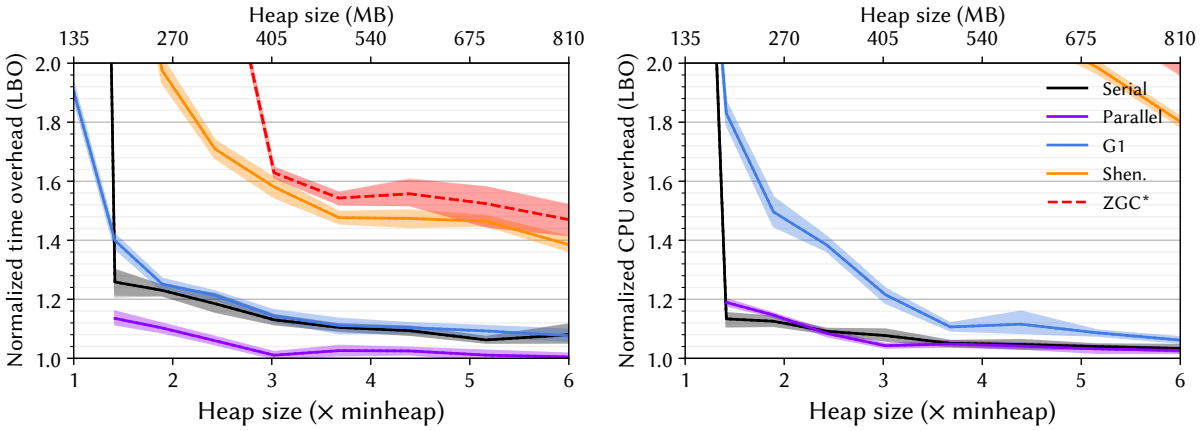
Figure 49. Distribution of request latencies for tomcat for each of OpenJDK 21's six production collectors. The figures in the left top row simply plot the request latencies, while the figures in the middle and the bottom rows use DaCapo's metered latency, which models a request queue and the cascading effect of delays.

B.19 Tradebeans

This is a latency-sensitive workload that executes the DayTrader workload over the Wildfly application server. Wildfly has about 4.2 M lines of Java source code. The DayTrader workload was originally developed by IBM Research to model customer applications on their production application server [34, 35]. DaCapo replaces the DayTrader synthetic load generator with a deterministic load. tradebeans is sensitive to compiler configuration (PCS) and memory speed (PMS). It is slow to warm up (PWU) and has high variance (PSD). It has a minimum heap size of 1.1 GB in its vlarge configuration. It is one of the least back end bound workloads (USB).

Table 21. Complete nominal statistics for tradebeans. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
GCA	3	87	17	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	5	948	12	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	4	84	15	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	5	2	12	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	9	26	3	0	0	120	nominal percent 10th iteration memory leakage
GMD	7	135	8	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	7	605	7	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	8	73	5	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	6	141	9	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GMV	4	1123	2	371	1123	20641	nominal minimum heap size (MB) for vlarge size configuration (with compressed pointers)
GSS	5	67	13	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
PCC	3	104	16	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	8	150	5	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	3	1	17	1	3	8	nominal execution time (sec)
PFS	7	17	7	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	8	150	5	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	5	2	12	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	8	25	6	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	6	7	9	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	3	4	16	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	9	2	4	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	8	6	6	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	9	144	3	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	9	42	3	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	4	21	15	5	23	41	nominal backend bound (memory)
UBP	5	38	13	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	6	1187	9	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	5	39	12	5	39	137	nominal 1000 x bad speculation
UDC	3	8	16	2	12	27	nominal data cache misses per K instructions
UDT	5	177	11	14	174	576	nominal DTLB misses per M instructions
UIP	7	199	8	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	5	2352	13	335	2645	8506	nominal LLC misses per M instructions
USB	2	23	19	7	29	53	nominal 100 x back end bound
USC	5	42	13	1	52	351	nominal 1000 x SMT contention
USF	8	38	5	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 50. Lower bounds on the overheads [11] for tradebeans for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

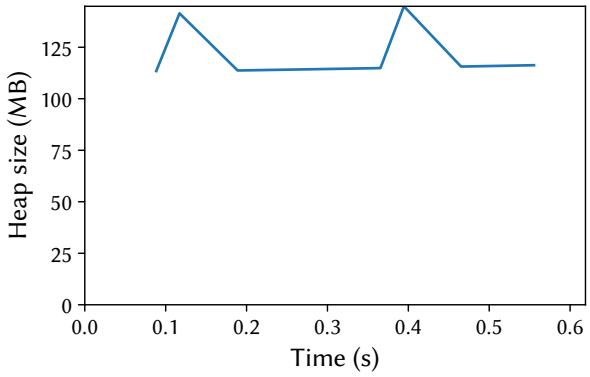
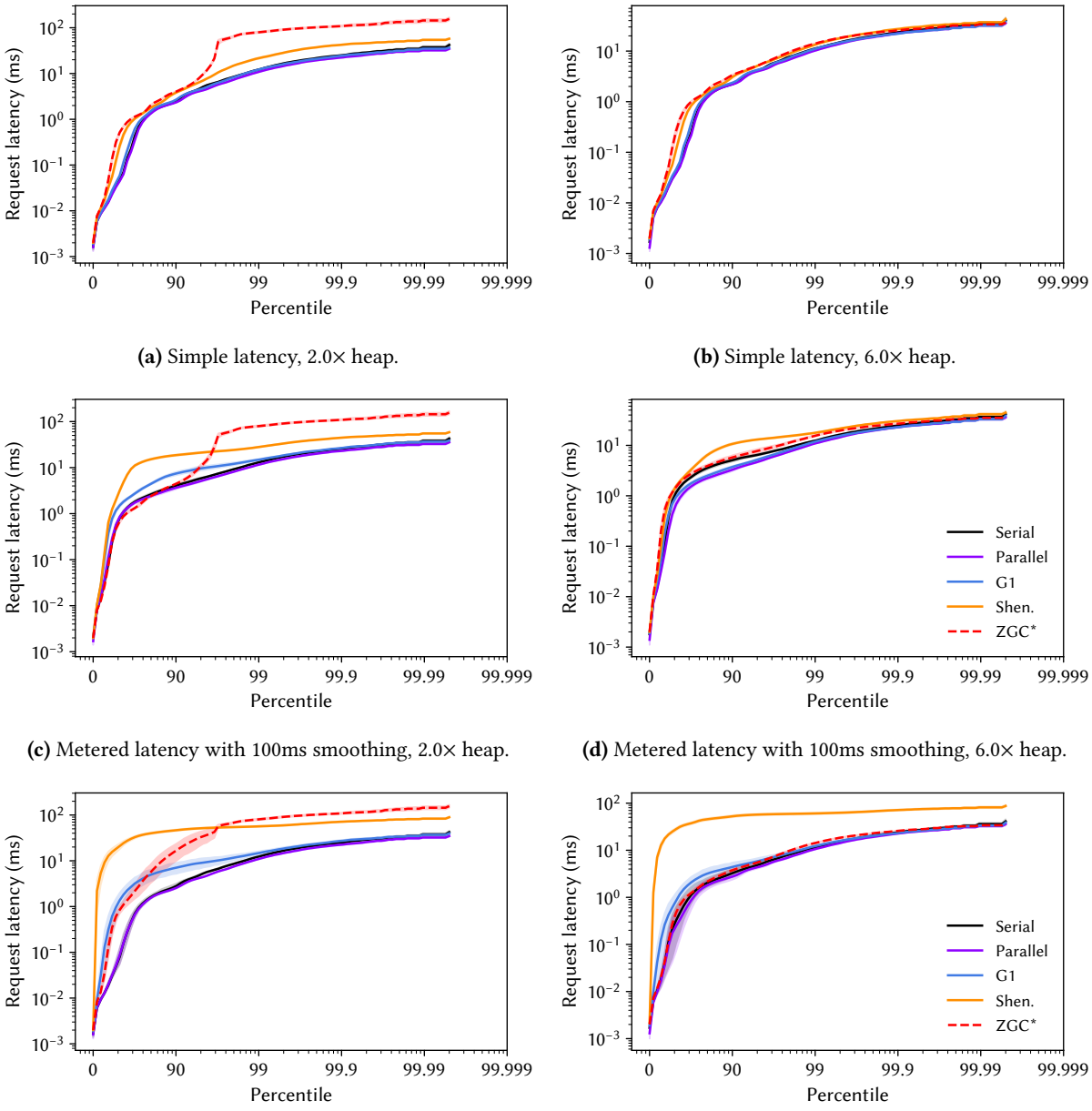


Figure 51. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.



(a) Simple latency, 2.0× heap.

(b) Simple latency, 6.0× heap.

(c) Metered latency with 100ms smoothing, 2.0× heap.

(d) Metered latency with 100ms smoothing, 6.0× heap.

(e) Metered latency with full smoothing, 2.0× heap.

(f) Metered latency with full smoothing, 6.0× heap.

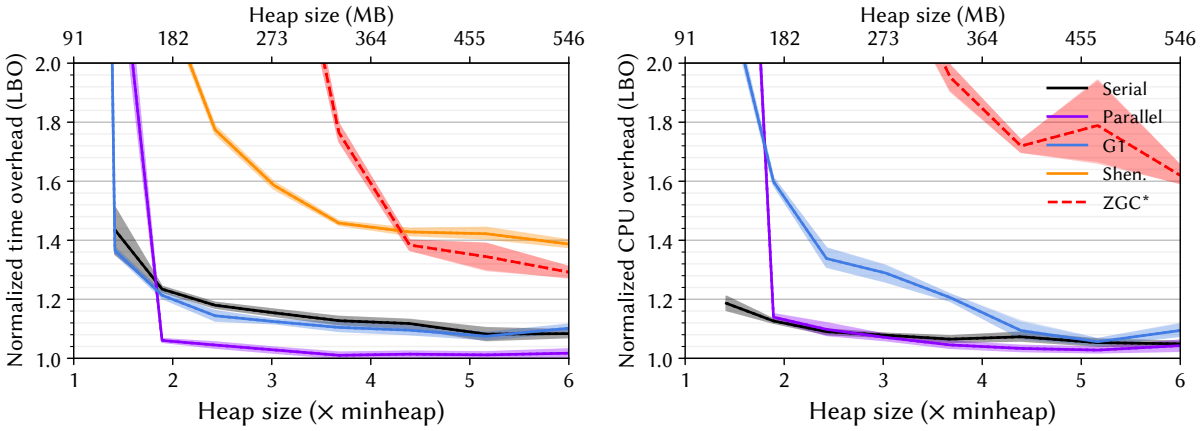
Figure 52. Distribution of request latencies for tradebeans for each of OpenJDK 21's six production collectors. The figures in the left top row simply plot the request latencies, while the figures in the middle and the bottom rows use DaCapo's metered latency, which models a request queue and the cascading effect of delays.

B.20 Tradesoap

This is a latency-sensitive workload that executes the DayTrader workload over the Wildfly application server. It differs from tradebeans in that it uses the full SOAP protocol to communicate with the server. DaCapo includes the two variants of the DayTrader workload on the recommendation of the authors of the original work that pointed to the inefficiencies of such web frameworks [36]. It is sensitive to CPU frequency scaling (PFS) and is the most sensitive to last level cache size (PLS). It has a high DTLB miss rate (UDT), but is not particularly back end bound (USB, UBM).

Table 22. Complete nominal statistics for tradesoap. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
GCA	5	101	11	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	4	774	15	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	6	100	9	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	4	1	15	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	8	6	6	0	0	120	nominal percent 10th iteration memory leakage
GMD	5	91	11	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	6	229	9	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	9	75	4	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	5	115	11	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GMV	0	371	3	371	1123	20641	nominal minimum heap size (MB) for vlarge size configuration (with compressed pointers)
GSS	5	284	11	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
PCC	9	400	3	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	7	117	7	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	3	1	17	1	3	8	nominal execution time (sec)
PFS	7	16	8	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	7	117	7	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	5	2	12	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	10	40	1	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	6	6	10	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	5	8	11	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	9	2	4	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	7	5	8	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	10	147	2	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	7	34	8	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	5	23	12	5	23	41	nominal backend bound (memory)
UBP	8	73	6	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	5	1087	12	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	8	74	6	5	39	137	nominal 1000 x bad speculation
UDC	7	17	7	2	12	27	nominal data cache misses per K instructions
UDT	7	385	7	14	174	576	nominal DTLB misses per M instructions
UIP	5	163	11	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	5	2645	12	335	2645	8506	nominal LLC misses per M instructions
USB	4	26	15	7	29	53	nominal 100 x back end bound
USC	5	52	12	1	52	351	nominal 1000 x SMT contention
USF	7	35	7	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 53. Lower bounds on the overheads [11] for tradesoap for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

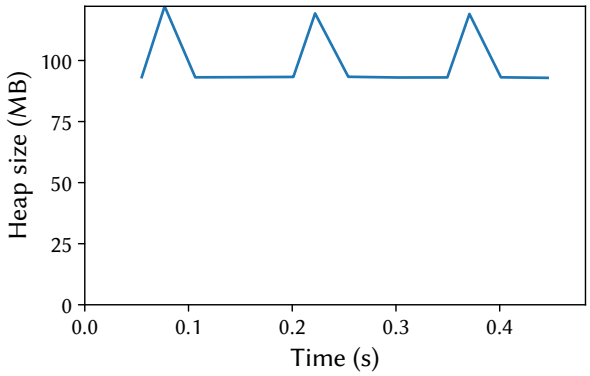
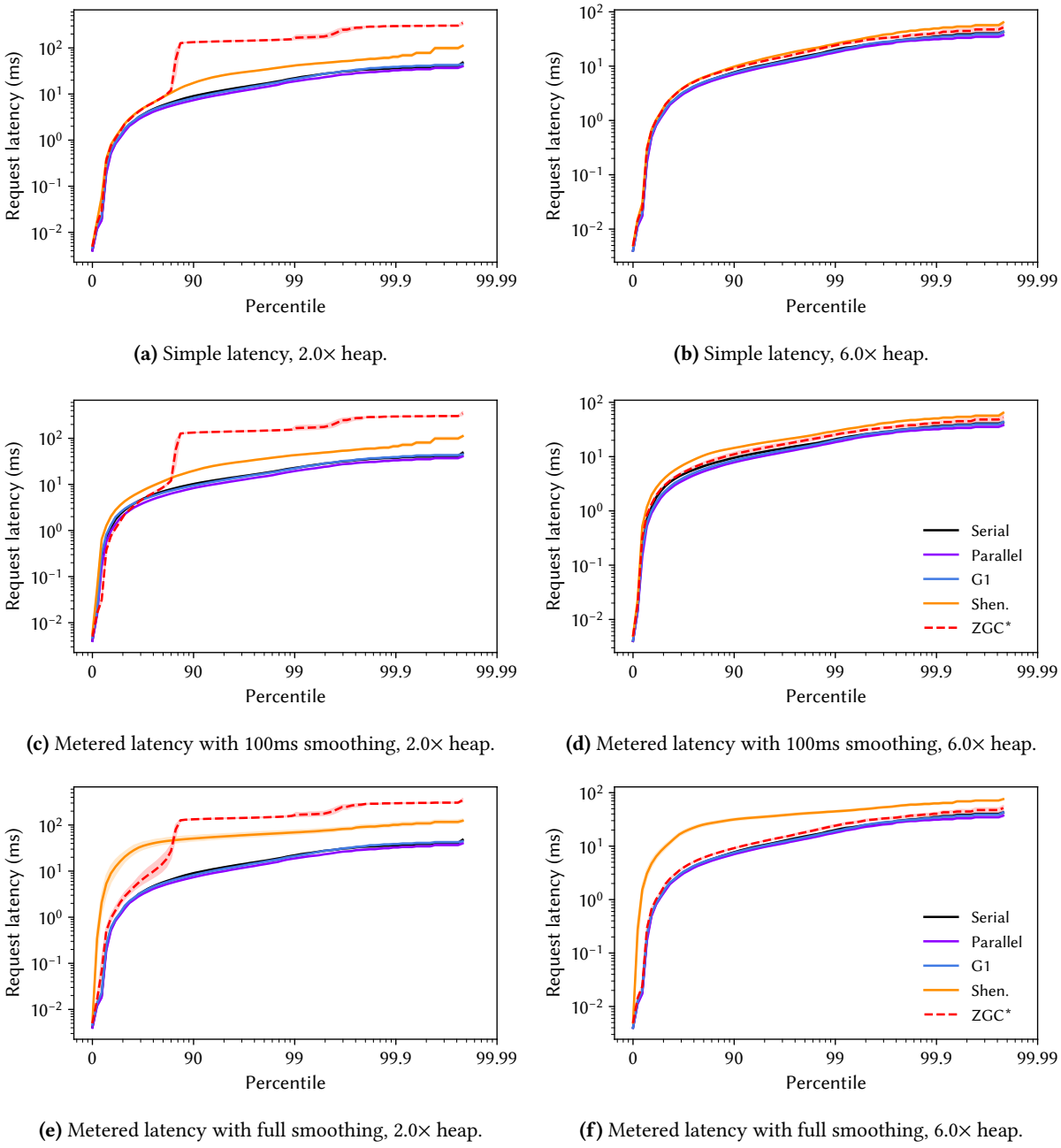


Figure 54. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.



(a) Simple latency, 2.0× heap.

(b) Simple latency, 6.0× heap.

(c) Metered latency with 100ms smoothing, 2.0× heap.

(d) Metered latency with 100ms smoothing, 6.0× heap.

(e) Metered latency with full smoothing, 2.0× heap.

(f) Metered latency with full smoothing, 6.0× heap.

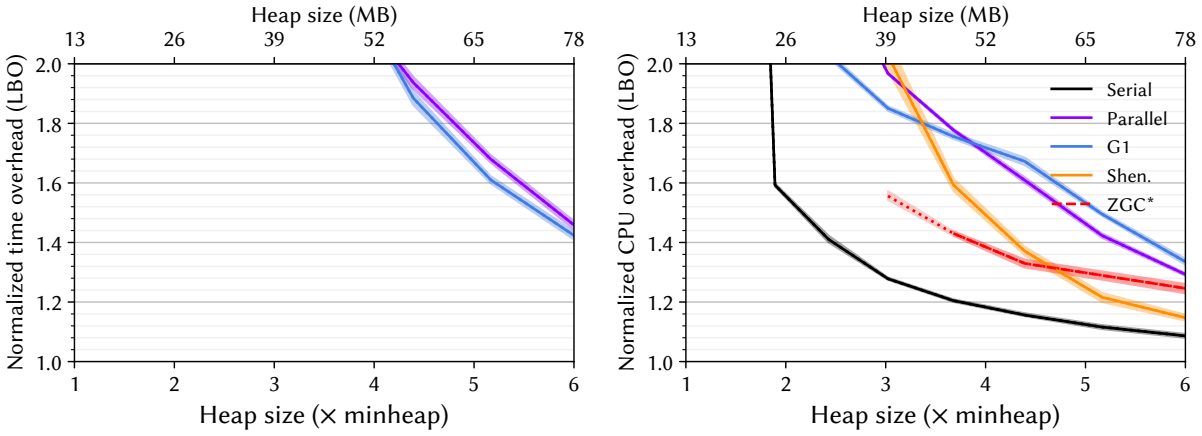
Figure 55. Distribution of request latencies for tradesoap for each of OpenJDK 21's six production collectors. The figures in the left top row simply plot the request latencies, while the figures in the middle and the bottom rows use DaCapo's metered latency, which models a request queue and the cascading effect of delays.

B.21 Xalan

This workload uses the Apache Xalan XSLT processor to transform a set of documents. xalan is the workload most sensitive to heap size (GSS, GCA, GCC, GCM, GCP, GTO). It has a high allocation rate (ARA), and very high aastore, aaload, putfield, and getfield rates (BAS, BAL, BPF, BGF). It is very insensitive to compiler configuration (PCC, PCS, PIN). It has one of the worst data cache miss rates (UDC), and one of the lowest IPCs (UIP).

Table 23. Complete nominal statistics for xalan. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	8	107	5	28	58	211	nominal average object size (bytes)
AOL	3	48	15	24	56	200	nominal 90-percentile object size (bytes)
AOM	3	24	14	24	32	48	nominal median object size (bytes)
AOS	10	24	1	16	24	24	nominal 10-percentile object size (bytes)
ARA	7	6682	6	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	9	294	3	0	34	2204	nominal aaload per usec
BAS	9	90	2	0	1	126	nominal aastore per usec
BEF	5	4	11	1	4	29	nominal execution focus / dominance of hot code
BGF	8	6389	4	0	527	32087	nominal getfield per usec
BPF	9	807	3	0	83	3863	nominal putfield per usec
BUB	3	21	15	1	34	177	nominal thousands of unique bytecodes executed
BUF	3	2	15	0	4	29	nominal thousands of unique function calls
GCA	9	114	3	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	9	13484	3	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	9	114	3	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	10	78	1	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	8	7	5	0	0	120	nominal percent 10th iteration memory leakage
GMD	1	13	20	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GML	0	13	20	13	149	10201	nominal minimum heap size (MB) for large size configuration (with compressed pointers)
GMS	2	5	18	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	1	17	20	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	10	7638	1	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	8	285	4	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	0	0	22	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	1	13	20	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	3	1	17	1	3	8	nominal execution time (sec)
PFS	5	12	12	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	1	13	20	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	9	14	4	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	7	19	8	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	5	4	13	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	5	6	12	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	7	1	7	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	1	1	20	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	6	101	10	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	3	13	17	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	7	29	7	5	23	41	nominal backend bound (memory)
UBP	5	39	12	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	4	785	15	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	5	39	12	5	39	137	nominal 1000 x bad speculation
UDC	9	20	4	2	12	27	nominal data cache misses per K instructions
UDT	8	441	5	14	174	576	nominal DTLB misses per M instructions
UIP	1	98	21	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	8	5045	5	335	2645	8506	nominal LLC misses per M instructions
USB	7	34	8	7	29	53	nominal 100 x back end bound
USC	6	97	9	1	52	351	nominal 1000 x SMT contention
USF	8	36	6	4	23	51	nominal 100 x front end bound



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 56. Lower bounds on the overheads [11] for xalan for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

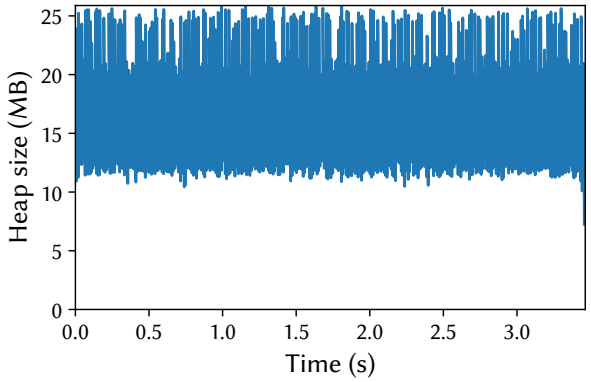


Figure 57. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.

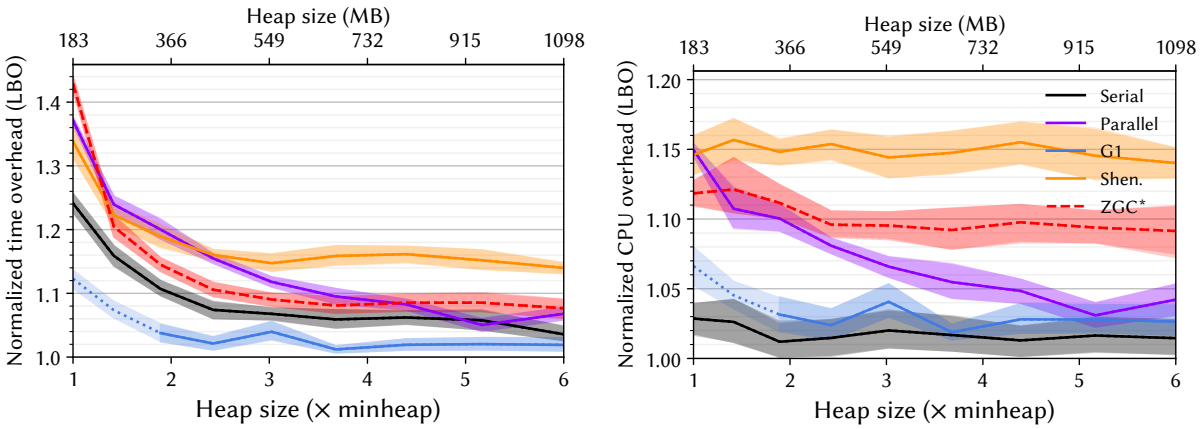
B.22 Zxing

(**New**) This workload uses the ZXing[9] barcode reader to read a series of 1D and 2D barcodes. ZXing has about 48 K lines of Java source code. zxing has one of the highest parallel efficiency among the workloads (PPE). It is one of the slowest to warm up (PWU), has one of the largest average and median object sizes (AOA, AOM) and is one of the least sensitive to garbage collection (GCA, GCC, GCM, GCP). It has the highest SMT contention (USC), is among the least back end bound (USB), and has among the lowest data cache, last level cache, and DTLB miss rates (UDC, ULL, UDT).

Table 24. Complete nominal statistics for zxing. Value represents the concrete value for that metric with respect to Description. Min, Median, and Max are the summary statistics for that metric across all benchmarks. For each metric, the benchmark obtains a Score between 0 and 10 (10 being the largest concrete value for that metric). Similarly, the benchmark obtains a Rank between 1 and the number of benchmarks having that metric (1 being the largest).

Metric	Score	Value	Rank	Min	Median	Max	Description
AOA	9	115	3	28	58	211	nominal average object size (bytes)
AOL	7	80	7	24	56	200	nominal 90-percentile object size (bytes)
AOM	9	32	2	24	32	48	nominal median object size (bytes)
AOS	10	24	1	16	24	24	nominal 10-percentile object size (bytes)
ARA	5	2097	11	54	2097	23556	nominal allocation rate (bytes / usec)
BAL	7	129	7	0	34	2204	nominal aaload per usec
BAS	4	0	13	0	1	126	nominal aastore per usec
BEF	8	11	4	1	4	29	nominal execution focus / dominance of hot code
BGF	8	6226	5	0	527	32087	nominal getfield per usec
BPF	2	31	16	0	83	3863	nominal putfield per usec
BUB	6	55	9	1	34	177	nominal thousands of unique bytecodes executed
BUF	5	4	10	0	4	29	nominal thousands of unique function calls
GCA	0	16	22	16	100	133	nominal average post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCC	1	58	21	31	948	22408	nominal GC count at 2X heap size (G1)
GCM	0	14	22	14	98	144	nominal median post-GC heap size as percent of min heap, when run at 2X min heap with G1
GCP	4	1	15	0	2	78	nominal percentage of time spent in GC pauses at 2X heap size (G1)
GLK	10	120	1	0	0	120	nominal percent 10th iteration memory leakage
GMD	9	183	4	5	72	681	nominal minimum heap size (MB) for default size configuration (with compressed pointers)
GMS	2	5	18	5	13	157	nominal minimum heap size (MB) for small size configuration (with compressed pointers)
GMU	6	127	10	7	73	903	nominal minimum heap size (MB) for default size without compressed pointers
GSS	1	6	20	0	249	7638	nominal heap size sensitivity (slowdown with tight heap, as a percentage)
GTO	1	7	19	3	52	1211	nominal memory turnover (total alloc bytes / min heap bytes)
PCC	8	277	6	0	201	1083	nominal percentage slowdown due to aggressive c2 compilation compared to baseline (compiler cost)
PCS	5	64	11	1	61	323	nominal percentage slowdown due to worst compiler configuration compared to best (sensitivty to compiler)
PET	3	1	17	1	3	8	nominal execution time (sec)
PFS	0	-1	22	-1	12	20	nominal percentage speedup due to enabling frequency scaling (CPU frequency sensitivity)
PIN	5	64	11	1	61	323	nominal percentage slowdown due to using the interpreter (sensitivty to interpreter)
PKP	6	5	10	0	2	56	nominal percentage of time spent in kernel mode (as percentage of user plus kernel time)
PLS	6	18	10	-2	8	40	nominal percentage slowdown due to 1/16 reduction of LLC capacity (LLC sensitivity)
PMS	10	46	1	0	5	46	nominal percentage slowdown due to slower memory (memory speed sensitivity)
PPE	10	63	2	3	6	87	nominal parallel efficiency (speedup as percentage of ideal speedup for 32 threads)
PSD	3	0	16	0	1	13	nominal standard deviation among invocations at peak performance (as percentage of performance)
PWU	9	7	4	1	3	9	nominal iterations to warm up to within 1.5 % of best
UAA	3	77	17	2	92	168	nominal percentage change (slowdown) when running on ARM Neoverse N1 (Ampere Altra Q80-30) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UAI	9	42	3	-19	25	56	nominal percentage change (slowdown) when running on Intel Golden Cove (i9-12900KF) v AMD Zen 4 (Ryzen 9 7950X) on a single core (taskset 0)
UBM	0	5	22	5	23	41	nominal backend bound (memory)
UBP	7	52	8	5	39	134	nominal 1000 x bad speculation: mispredicts
UBR	1	374	21	164	1087	3487	nominal 1000000 x bad speculation: pipeline restarts
UBS	6	52	9	5	39	137	nominal 1000 x bad speculation
UDC	1	3	20	2	12	27	nominal data cache misses per K instructions
UDT	0	14	22	14	174	576	nominal DTLB misses per M instructions
UIP	8	211	6	89	149	476	nominal 100 x instructions per cycle (IPC)
ULL	0	335	22	335	2645	8506	nominal LLC misses per M instructions
USB	0	7	22	7	29	53	nominal 100 x back end bound
USC	10	351	1	1	52	351	nominal 1000 x SMT contention
USF	4	18	15	4	23	51	nominal 100 x front end bound

[9]Pronounced *zebra crossing*.



(a) Wall clock.

(b) Total CPU overheads (TASK_CLOCK).

Figure 58. Lower bounds on the overheads [11] for zxing for each of OpenJDK 21's six production garbage collectors as a function of heap size. The figure on the left shows the overhead in terms of wall clock time while the figure on the right shows the overhead using the Linux perf TASK_CLOCK, which sums the running time of all threads in the process, giving the total computation overhead.

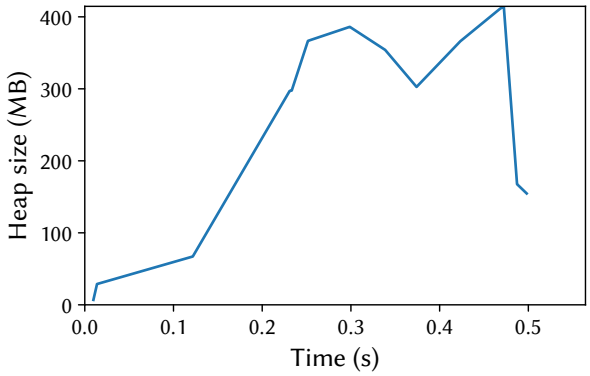


Figure 59. Heap size post each garbage collection, with the time relative to the start of the last benchmark iteration. The benchmark is running with OpenJDK 21's G1 collector at 2.0× heap.